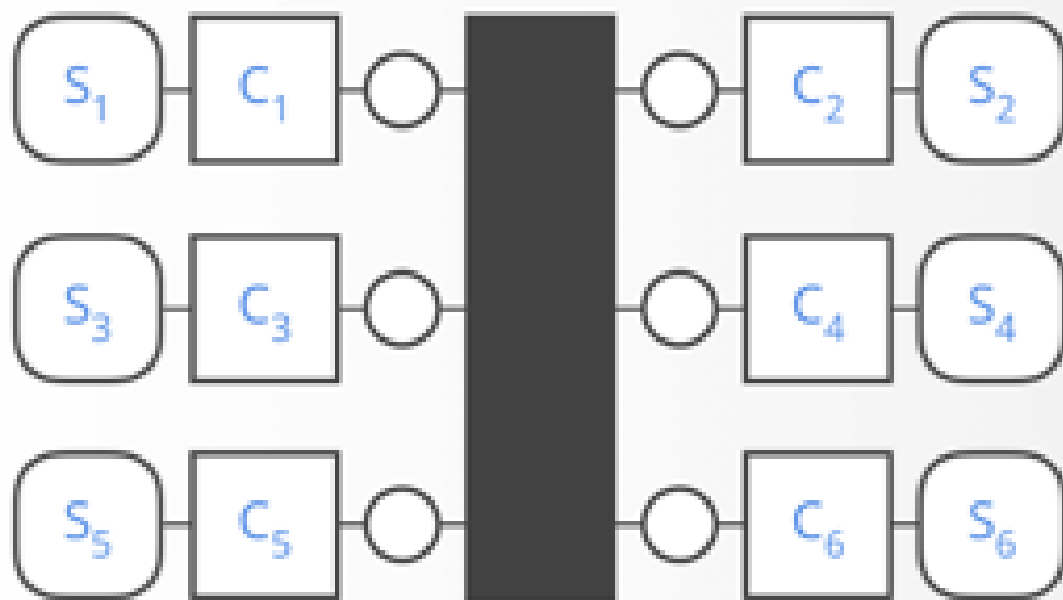


Distributed Systems



Dominik Tornow

PREFACE



Software engineering can be a delightful profession, but at the same time, software engineering can be a dreadful profession. For me, software engineering is particularly dreadful when I experience confusion.

Unfortunately, in this industry, confusion is a constant state of mind. Our ability to think about software systems and our ability to talk about software systems lacks the maturity of other professions that have developed strong mental models and means of communication. For example, ask yourself: What is a micro service, what is the cloud, what is cloud native, or what is serverless - an endless list of questions. Do you know the answers?

Often we have a fuzzy notion, a vague idea, that we cannot communicate effectively and efficiently.

For me, this is a source of great frustration. I do not want a fuzzy notion of a system, I do not want to probe the system, poke it with a stick, observe and rationalize its behavior

until the next poke unexpectedly yet inadvertently invalidates what I think I knew.

I want an accurate and concise mental model to reason confidently about the system.

In this book, we will develop accurate and concise mental models to reason confidently about distributed systems. Additionally, you will learn to construct your own mental models to replace confusion with certainty and hesitance with confidence.

THINKING IN DISTRIBUTED SYSTEMS



There is a distinction between knowing about complex distributed systems and truly understanding complex distributed systems.

The distinction between knowing and understanding is not limited to complex, distributed systems, but can also be seen elsewhere, for example in board games. While it may be easy to learn the basic rules of a game, it takes a much greater investment of time to master the strategies and tactics required for full understanding. This process may take months, years, or even a lifetime.

The game of chess showcases the divide between knowing and understanding perfectly. While it only takes a short amount of time to learn the rules of the game, it takes much longer to truly understand the strategies and tactics necessary to excel at it. This is why chess is often seen as a measure of intelligence, both for humans and machines. Despite its simple setup of a board, six types of pieces, and two players, chess is a remarkably complex and sophisticated game.

The same concept applies to distributed systems. So as we progress through this book, our goal is to not only build a deeper knowledge but also to build a deeper understanding of distributed systems.

AHA! moments

On my own journey to understanding distributed systems, I had many moments of realization, or “AHA!” moments. These moments always left me excited and increased my confidence.

In this book, I hope to share my AHA! moments with you. Some may be obvious to you, while others may be more profound. However, I don't think that I'm the only one who struggled to gain a deeper understanding of these concepts, and I believe that sharing my own experiences, whether they seem obvious or profound, may be helpful to you as well.

Distributed System Incorporated

A distributed system is made up of multiple components that operate concurrently and communicate with each other by sending and receiving messages over a network.

- The overall behavior of a distributed system is a result of the behavior of its individual components and how they interact with each other.
- The overall complexity of a distributed system is a result of the complexity of its components and the complexity of the interactions between them.

As we dive into the topic of distributed systems, I encourage you to picture them as a cooperation situated in an office building.

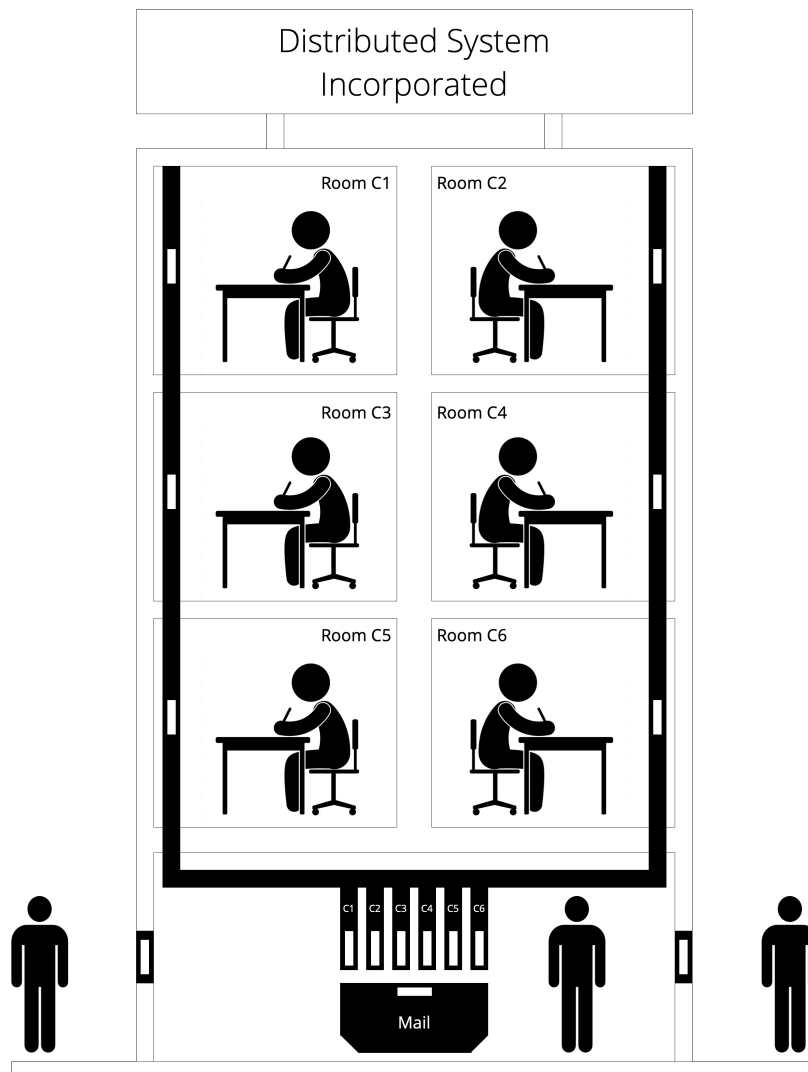


Figure 1. Distributed Systems Incorporated

The building represents the system, with each room within it representing a component. These rooms are connected by pneumatic tubes, which act as the network for communication. The office building is connected to the rest of the world via a mailbox where all incoming and outgoing messages are processed.

I find this mental model helpful because it allows me to think about and talk about distributed systems: It takes an intangible, abstract cyber system and maps it onto a tangible, concrete physical system, while faithfully capturing the core mechanics of the system.

In addition, this model is able to represent a wide range of concerns. For example, what happens to Distributed Systems Incorporated if Bob the mailroom attendant loses messages, duplicates messages, or rearranges messages? This reflects different types of message delivery semantics. What happens when employees take a 15 min break, take vacation, or leave the company? This reflects different types of crashes.

Through these examples, we will analyze the consequences of these actions and explore potential countermeasures.

You may be surprised at how far this model can take you. Try using it to model Kubernetes Inc., Kafka Inc., or a multi-cluster deployment involving multiple buildings and a public mail service like USPS.

Although we won't be covering Kubernetes or Kafka in this book, I am eager to use insights gained from Distributed Systems Inc. to accurately and concisely define the terms *service* and dare I say *microservice* .

Some AHA! Moments

Even without delving deeply into the complexities of distributed systems, we can already begin to have some AHA! moments.

Let's preview a few informally and explore them further in future chapters:

AHA! Moment • Simplicity vs Complexity

While each member of our staff may only handle a simple set of tasks and follow a simple set of rules, the resulting behavior of Distributed Systems Inc. is complex. Simple components do not necessarily result in simple systems!

AHA! Moment • Emergent Behavior

Interesting behavior of Distributed System Inc. cannot be traced back to individual employees. For example, one employee cannot be held responsible for the scalability of the company, as there is a limit to how much work they can accomplish in a day. Additionally, one employee cannot be held responsible for the reliability of the company, as they may be absent due to illness or leave the company.

Instead, the interesting behavior of Distributed System Inc. is emergent, resulting from the behavior of individual employees and their interactions. This is the very foundation of the idea to build reliable systems from unreliable components.

AHA! Moment • Changing Perspectives

For me, this was one of my most profound AHA! moments - and also one of the most vulnerable moments in my career. To this day, I am almost embarrassed that it took me so long to realize it.

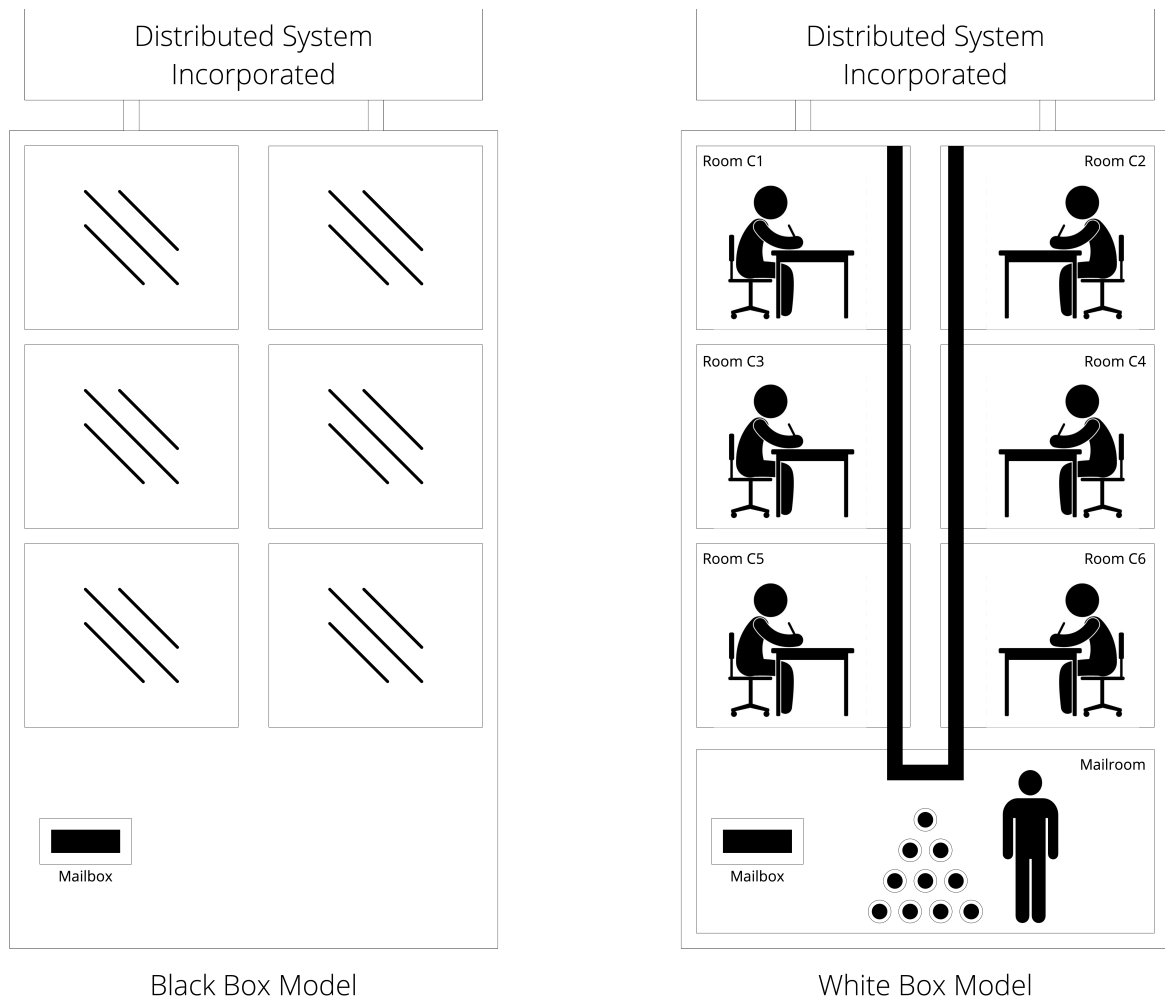


Figure 2. Black Box vs. White Box, global point of view

We often easily and instinctively think about systems from a black box and a white box perspective. However, moving from a black box to a white box perspective or vice versa is a change in resolution, not a change in perspective.

As shown in Figure 2., whether we are looking at the black box or the white box side, we are considering the system from the perspective of an all-knowing observer, meaning that we have the ability to observe the state of the entire system, **we have a global view.**

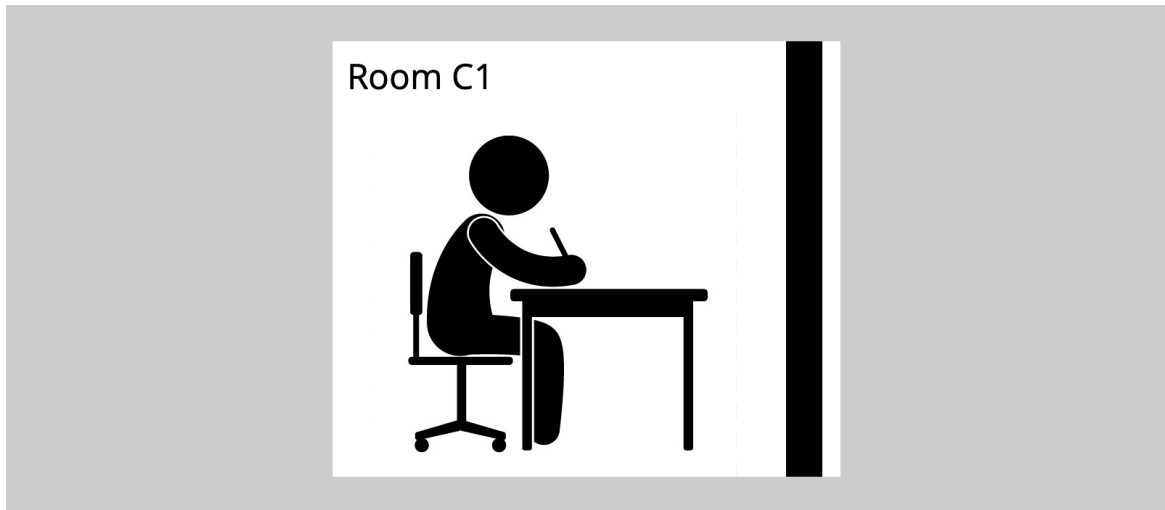


Figure 3. Local point of view

However, Figure 3. illustrates that a component does not have the same luxury as an all-knowing observer. A component can only observe its own state, **giving it a limited, local view.**

As a side note: While the depiction in Figure 3 may seem lonely and depressing, I like to think that Distributed System Incorporated has a positive work culture, strong team dynamics, and the highest levels of employee satisfaction.

On my own journey of understanding and thinking in distributed systems, it took me a long time to change my perspective. In hindsight, I think it would have been beneficial to make this shift at the outset.

With this shift in perspective, we are able to accurately and concisely identify the core challenge of distributed systems: thinking globally while acting locally.

Think globally act locally

The core challenge of distributed systems is to design a global algorithm, while each component of the system implements a local algorithm.

In other words, the challenge is to create a system that functions as a cohesive whole, despite the fact that each component is only aware of and able to access local information.

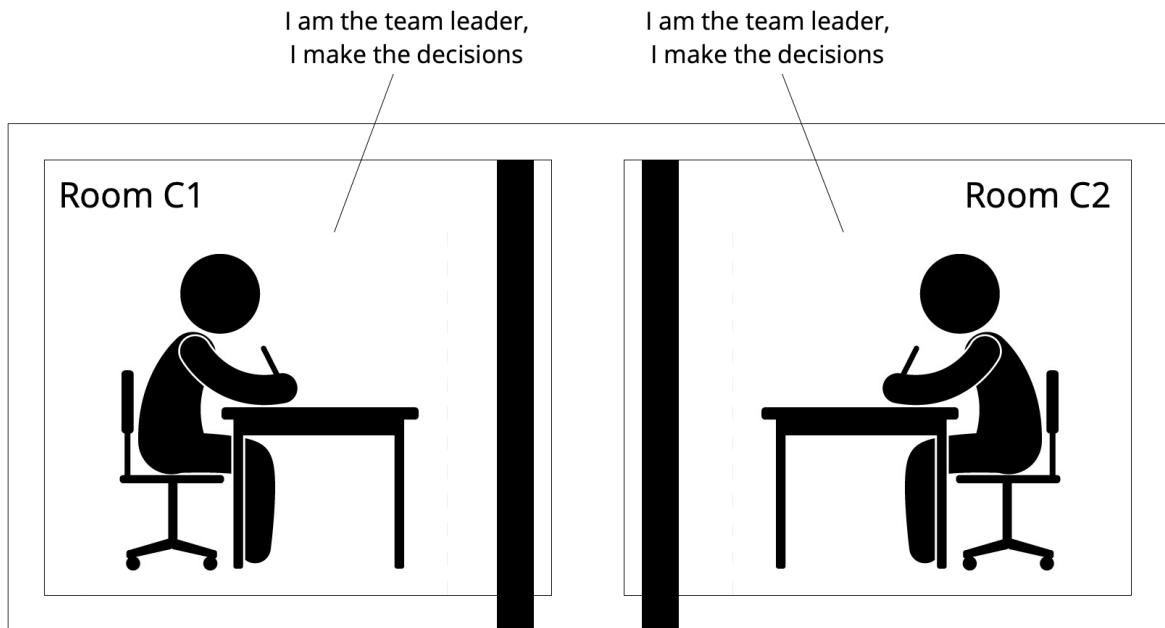


Figure 4. Splitbrain

Figure 4. illustrates a common example of the difficulty in ensuring global correctness with only limited knowledge: Splitbrain. In this scenario, global correctness depends on one person being the leader and making decisions. But how can you ensure that only one person believes they are the team lead?

What are the participants involved in this process, what local knowledge do they possess, and what local steps do they take to ensure global guarantees are met?

Conclusion

In the chapters ahead, we will utilize the analogy of Distributed System Incorporated to map cyber systems to physical systems, making abstract concepts more concrete while still maintaining the core mechanics of the system.

We have briefly touched upon several important concepts related to distributed systems. In future chapters, we will dive deeper into each of these points:

- The complexity of distributed systems: We will explore how simple components can combine to create complex systems and how this complexity arises from emergent behavior.
- Building reliable systems from unreliable components: We will delve into the foundation of emergent behavior and how it allows us to construct reliable systems from unreliable components.
- Thinking globally while acting locally: We will examine the central challenge of distributed systems and the importance of shifting perspective from an all-knowing observer to the limited, local view of an individual component.

I am excited to embark on this journey with you.

INTRODUCTION



Every modern software system is a distributed system. Whether you are building a web app, mobile app, or cloud service, understanding the fundamental principles of distributed systems is essential for any software engineer.

In this chapter, we will delve into an informal discussion to build a *comprehensible* mental model of distributed systems and their mechanics. In the next chapter, we will delve into a formal discussion to build a *comprehensive* mental model of distributed systems and their mechanics.

Objectives:

- Be able to define and to think about the structure and behavior of a distributed system.
- Be able to define and to think about the correctness, scalability, and reliability of a distributed system.
- Be comfortable to reason about a system in different terms and from different perspectives.
- Understand the need for building distributed systems.

Definition

A distributed system is a set of concurrent, communicating components that communicate by sending and receiving messages over a network. Each component has exclusive access to its own local state, which is not accessible by any other components. Additionally, the network has exclusive access to its own local state, which is not accessible by any other components, capturing messages that are in flight.

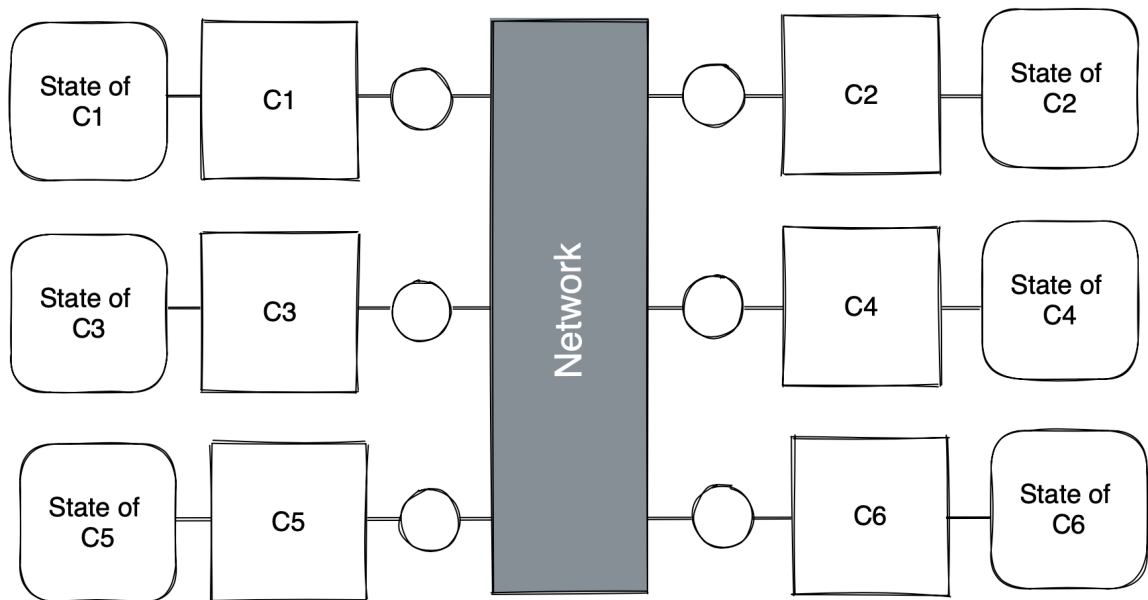


Figure 1. A distributed system as a set of concurrent, communicating components (local state of network not shown)

An accurate and concise way to think about the behavior of a distributed system is as a state machine: the behavior is a sequence of states and a step transitions the distributed system from the previous state to its next state.

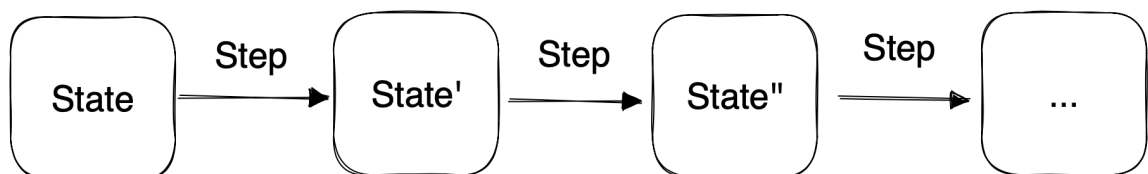


Figure 2. Behavior of a system as a sequence of states

In the most general terms, the system proceeds in discrete steps, with either a component or the network taking a step. A step can be either an external step, such as receiving a message or sending a message, or an internal step, such as performing a local computation including accessing the local state.

To summarize: We will think about the system as exactly one component or the network will take exactly one step at a time.

In different blog posts, papers, or books, components are often referred to as "actors", "agents", "processes", or "nodes". Similarly, steps are often referred to as "actions" or "events".

Alternative Definitions or Models

Over the years, I have extensively read about distributed systems. However, I found myself becoming increasingly frustrated that every blog, book, or paper defined and described distributed systems differently. I felt lost in a sea of inconsistencies and struggled to reconcile the differences.

There is not just one way to describe a distributed system. Instead, there are countless ways; for example, informally or formally, as action machines, state machines, state action machines, process algebras, the list goes on and on. Over time, I came to two realizations: *Different models but same aspects* and *different models and different aspects*.

Same Aspects

The first realization was that sometimes different ways of describing a distributed system are indeed equivalent, as a description of one kind can be transformed into a description of another kind.

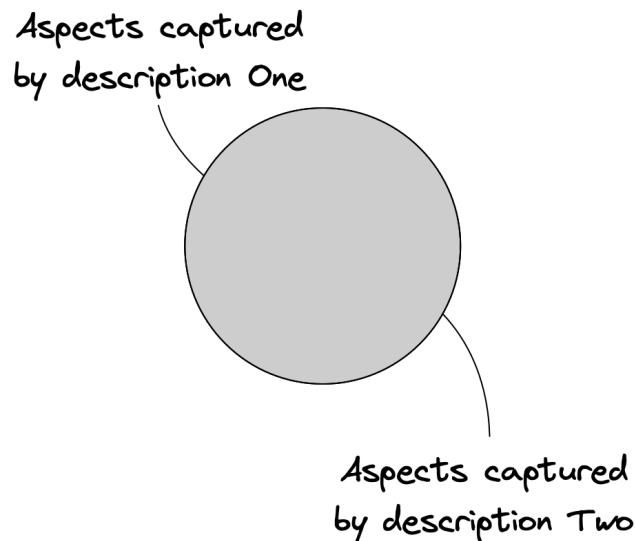


Figure 3. Different models describing the same aspects of a system

For example, we described a distributed system as if the network has a buffer to track inflight messages. However, we could have described the distributed system as if each component has a buffer to track inflight messages.

Both these descriptions are equivalent, both are able to express message loss, message duplication, or message reordering.

For example, on the one hand, in his lecture [Introduction to Distributed Systems](#), Martin Kleppmann describes a distributed system with the network being the buffer for inflight messages.

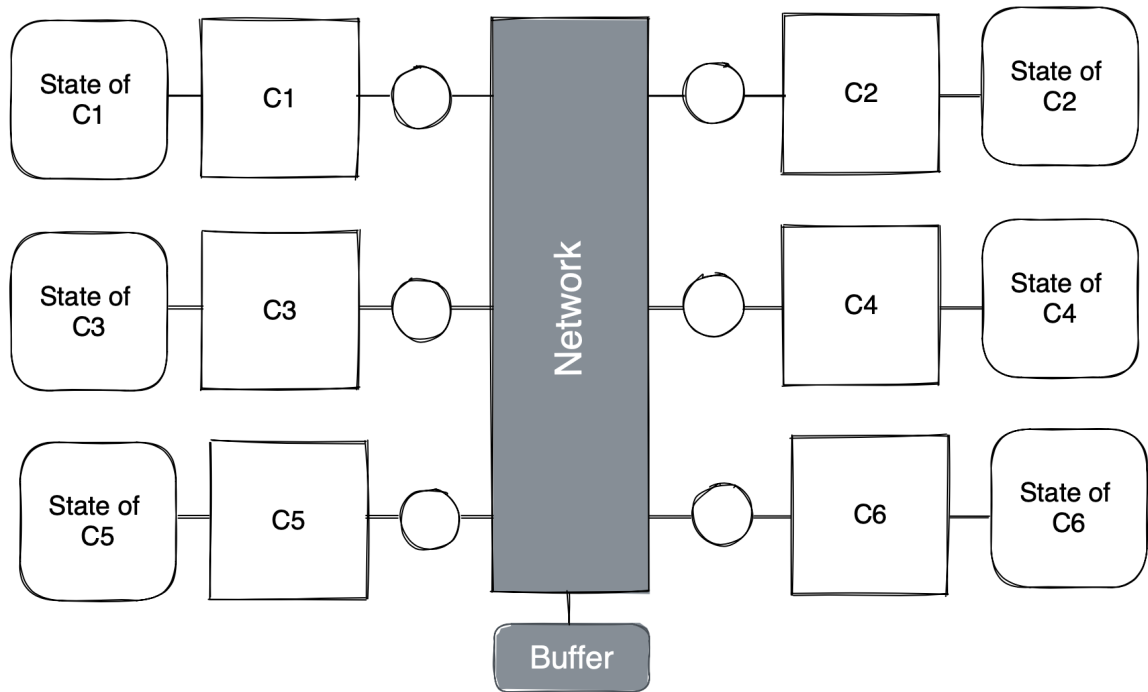


Figure 4. The network as the buffer of inflight messages

On the other hand, in his lecture [Introduction to Distributed Systems](#), Seif Haridi describes a distributed system with the components being the buffer for inflight messages.

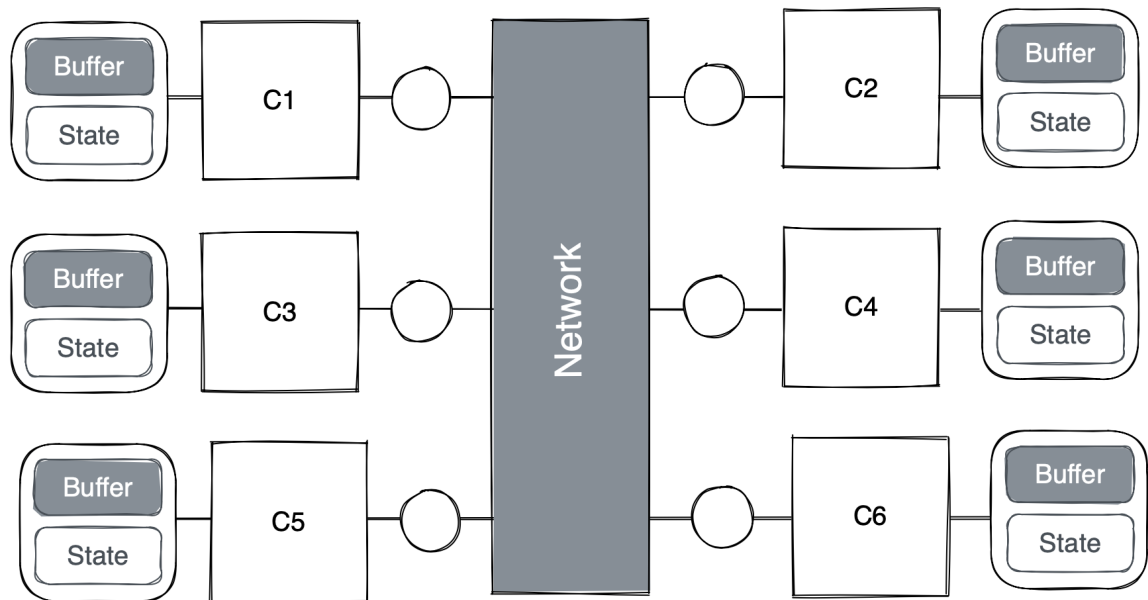


Figure 5. The components as the buffer for inflight messages

Neither model is incorrect, both are capable of capturing the relevant aspects of the distributed system such as messages getting lost, duplicated, or rearranged. Each model is a reflection on how the author thinks about distributed systems, their mental models.

Different Aspects

The second realization was that sometimes different ways of describing a distributed system are indeed not equivalent, as they have a different focus. Some descriptions express certain aspects of a system that others deliberately choose to omit.

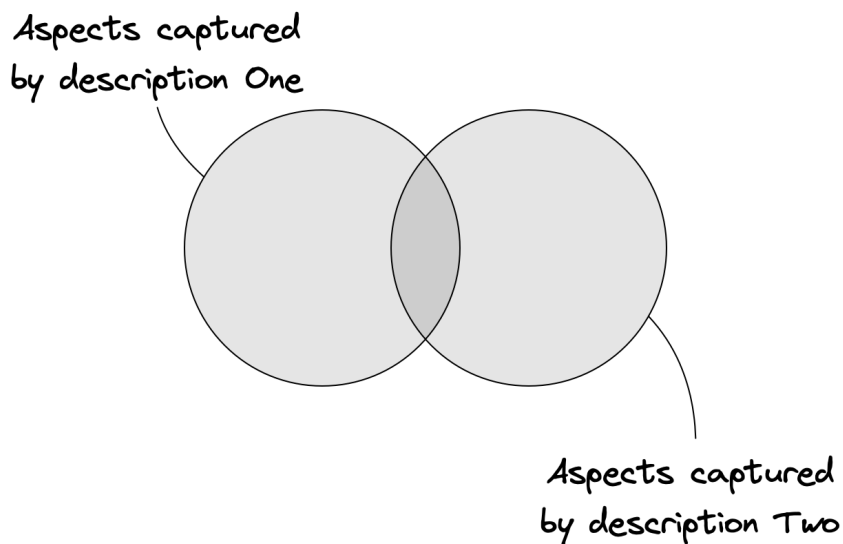


Figure 6. Different models describing different aspects of a system

For example, in his Introduction to TLA+, the Temporal Logic of Actions, Leslie Lamport describes the Transaction Commit Protocol, which is an abstraction of all distributed transaction protocols, and the Two Phase Commit Protocol, which is an implementation of a distributed transaction protocol.

The first model ignores message exchange all together, while the second model takes message exchange into account. As a result, the first model is unable to represent messages getting lost, duplicated, or rearranged, while the second model is able to do so.

In a later chapter of this book, we will delve deeper into Distributed Transaction Protocols, Consensus Protocols, and the topic of different models of distributed systems, examining their respective characteristics in greater detail.

AHA! Moment • Alternative Definitions or Models

The well-known parable of the blind men and an elephant is often used as an allegory to illustrate the importance of considering multiple perspectives when trying to understand a complex or abstract topic. The story goes that six blind men come across an elephant for the first time in their lives. Each man touches a different part of the elephant, such as the trunk, the ear, the tail, etc., and each man describes the elephant based on the part they touched. The moral of the story is that each person's understanding of the elephant is incomplete and limited by their perspective, and that a more complete understanding can only be gained by considering all of the perspectives together.

In the same way, studying multiple mental models, different ways of thinking, can help you gain a more holistic understanding of the topic. Additionally, by being exposed to multiple perspectives, you can also identify any misconceptions that may be present in your own thinking.

So, if you encounter different models of a distributed system, it does not mean that some of them are incorrect. When reading

posts, articles, or papers, try to not only understand the model chosen by the author, but also the reasons behind their choice – what makes the model suitable for the point they are trying to convey?

Please keep in mind that this can be quite challenging and even outright frustrating: You may have to set aside your hard-earned mental models in favor of an author's mental model. However, you will gain a more holistic understanding in return.

Correctness

As I progressed in my career, I realized that my understanding of correctness of a software system was fuzzy at best: a system was correct if it did what it was supposed to do, duh. `\\_(\ツ)/`

However, this definition does not provide any objective criteria for determining if a system is correct or not. I wanted to be able to rigorously think about the system's guarantees, capabilities, and limitations.

My favorite definition was coined by Leslie Lamport in [Proving the Correctness of Multiprocess Programs](#). Leslie Lamport defines the correctness of a system in terms of its safety and liveness properties:

- **Safety** Informally, a safety property guarantees that something bad will never happen.
- **Liveness** Informally, a liveness property guarantees that something good will eventually happen.

A system is correct if every possible behavior of the system is correct, that is, every possible behavior meets the system's

safety and liveness guarantees.

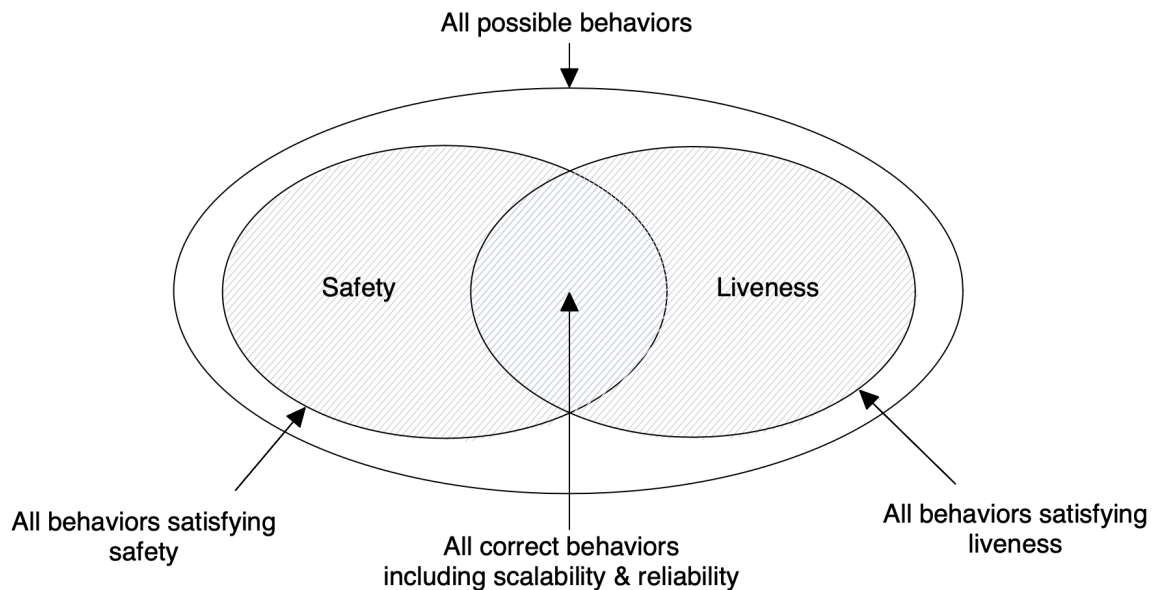


Figure 7. Safety and Liveness

Example

In the context of distributed transactions, a safety guarantee asserts that no two participants in a transaction arrive at a conflicting decision, while a liveness guarantee asserts that every participant will eventually arrive at a decision.

Let's assume a distributed transaction that spans two participants, two key value stores, P_1 and P_2 .

```
Begin Transaction
```

```
  AT participant  $P_1$  SET b TO "foo"
```

```
  AT participant  $P_2$  SET a TO "bar"
```

```
Commit Transaction
```

Figure 8. illustrates the behavior space, that is, all possible behaviors of the distributed transaction: Here, a state reflects the status of a transaction at participants P1 and P2. The transaction can be in one of three possible states: working, committed, or aborted. A state transition updates one participant from working to either committed or aborted (remember, exactly one component exactly one step at a time).

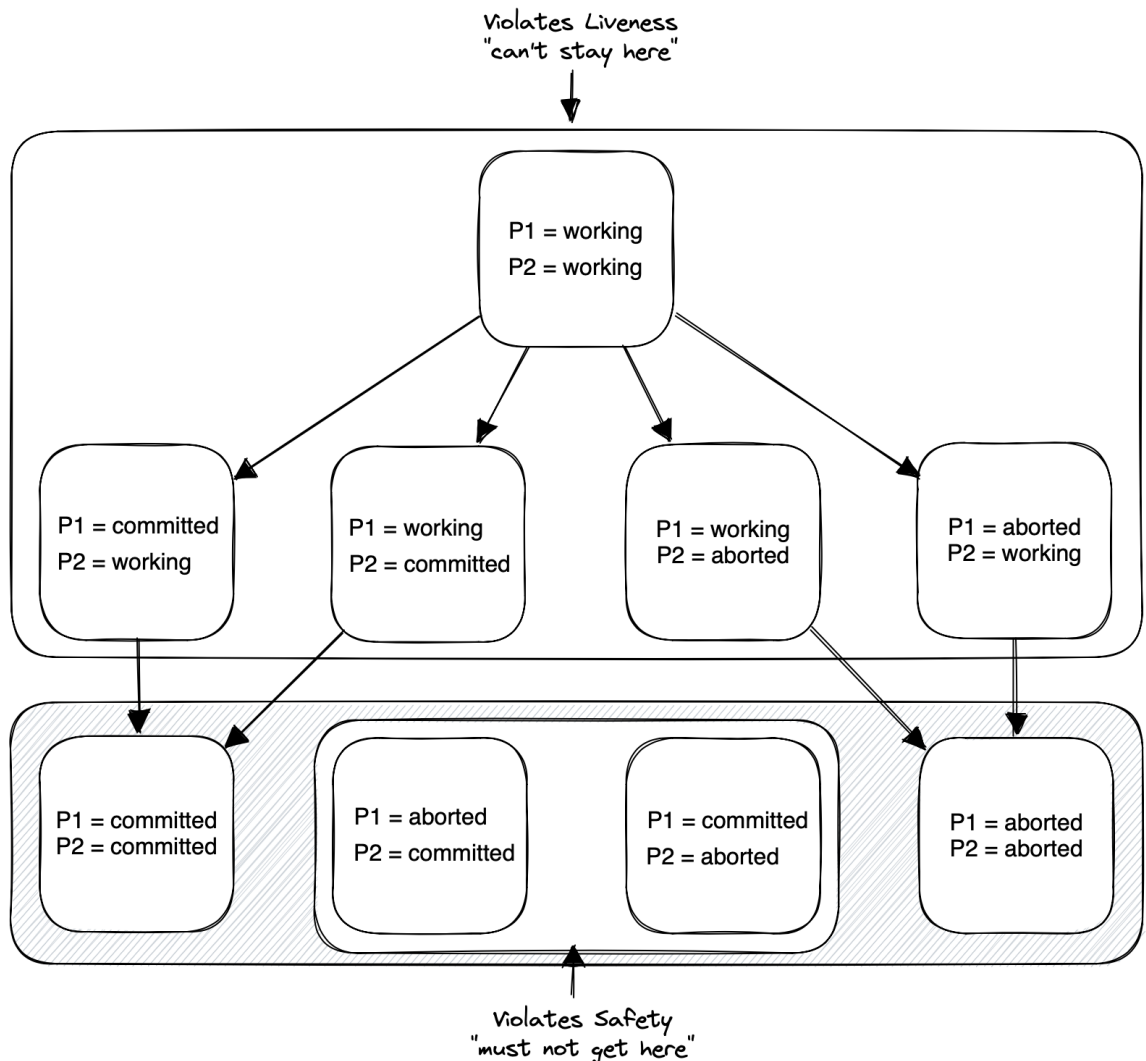


Figure 8. Behavior space of a distributed transaction with two participants

The safety guarantee of the system asserts that neither P_1 nor P_2 will commit while the other aborts. The liveness guarantee asserts that both P_1 and P_2 will eventually commit or abort. The

combination of safety and liveness guarantees asserts that eventually both P_1 and P_2 will reach the same decision, either committing or aborting.

In other words, the safety guarantee prevents the system from reaching an inconsistent state, the liveness guarantee prevents the system from getting stuck.

At first, I found the concept of safety and liveness guarantees to be overly academic - the nice way of saying *OMG, could you be more annoying?* - but I quickly came to appreciate the value of safety and liveness as a thinking tool, allowing me to effectively and efficiently reason about systems.

Note • Revisiting Alternative Models

Note how we think about the system: We have two key value stores yet we do not think in terms of keys and values. We are only interested in the status of a transaction at participant P_1 and P_2 . We ignore all other aspects. For example, we ignore the keys that are involved in the transaction and we ignore the values that are proposed by the transaction.

Of course, we could think about the system in terms of keys and values, and we could include other aspects, like locks on the keys that are involved in the transaction.

By presenting this model, I am asserting that other aspects are irrelevant to this discussion. You may disagree with this assertion. Neither of us is wrong; our mental models, our focus, our interest is simply different.

Scalability & Reliability

In Defining Liveness, Bowen Alpern and Fred Schneider provide a proof that every property of a system is an intersection of a safety property and a liveness property.

However, I do not find myself thinking about all aspects of a system in terms of safety and liveness: I do not find myself thinking about *scalability* and *reliability* guarantees in terms of safety and liveness guarantees.

That's not uncommon: Different mental models compete for our attention. Even when these mental models are equivalent, capable of expressing the same aspects but in different ways, we often gravitate towards one over the other.

So I find myself thinking about scalability and reliability guarantees in terms of responsiveness, where responsiveness is *informally* defined as a system's ability to meet its Service Level Objectives:

- **Scalability** Scalability of a systems is defined as its ability to be responsive in the presence of load.
- **Reliability** Reliability of a system is defined as its ability to be responsive in the presence of failure.

Excursion

Responsiveness is *formally* defined through four related concepts:

- **Service Level Indicator** A service level indicator is a quantitative observation about the behavior of a system.
- **Service Level Objective** A service level objective is a predicate (a function that yields true or false) on a service level indicator that determines whether the behavior of a system meets an objective.

- **Error Rate** The error rate is the ratio of the number of observations that do not meet their objectives to the number of observations in total for a fixed time interval.
- **Error Budget** The error budget is an upper limit on the error rate.

Based on the above, we can now define responsiveness of a system as the ability of the system to keep its error rate below its error budget. For a detailed discussion of these concepts, I recommend Google's book on [Site Reliability Engineering](#).

AHA! Moment • Application Specific

Correctness guarantees, including scalability and reliability, are specific to the application. As a software engineer, you have the ability to define the guarantees of your system and decide what behavior is desirable, tolerable, or intolerable. In other words, what you may consider to be broken, I may consider to be acceptable.

AHA! Moment • Emergence

Correctness guarantees, including scalability and reliability are **emergent** properties. Neither correctness, scalability, nor reliability trace back to an individual component. Instead, they are the result of a set of components and their interactions.

Discussion

At this point, I want to emphasize two "Big Ideas" that have greatly influenced my thinking in distributed systems: *Systems of Systems* and *Global vs. Local Viewpoints*.

These concepts will serve as the foundation for a clear and precise definition of services and, dare I say, microservices in a later chapter of the book.

Systems of Systems

According to its definition, a distributed system is a set of concurrent, communicating components. While not explicitly stated, I argue this definition motivates us to think of components as atomic entities.

However, in software systems, most components are not atomic entities but rather higher-order entities, or subsystems.

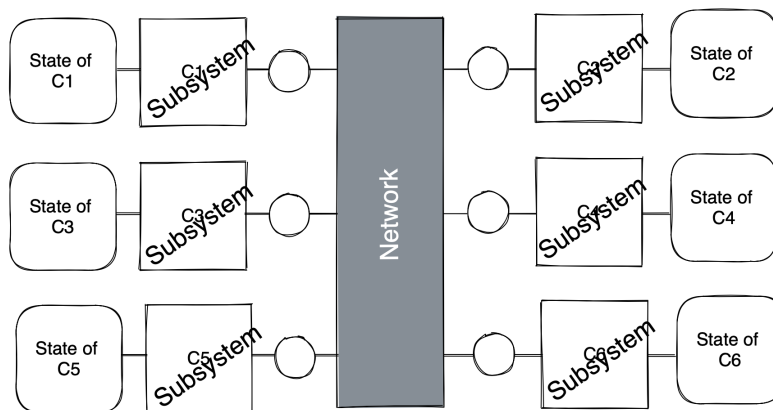


Figure 9. A distributed system as a set of concurrent, communicating subsystems

How should we approach the idea that a group of concurrent and communicating components exhibit a specific behavior, and that when viewed at a higher level of abstraction, the group behaves as if it were a single component?

In his 1967 book, "The Ghost in the Machine," the Hungarian philosopher Arthur Koestler introduced the term *holon* to describe an entity that is both a whole in itself and a part of a larger whole. Depending on the level of observation, a holon can

be viewed either as an atomic entity or as an organization of holons, a higher-order entity. This hierarchical structure composed of holons is referred to as a *holarchy*.

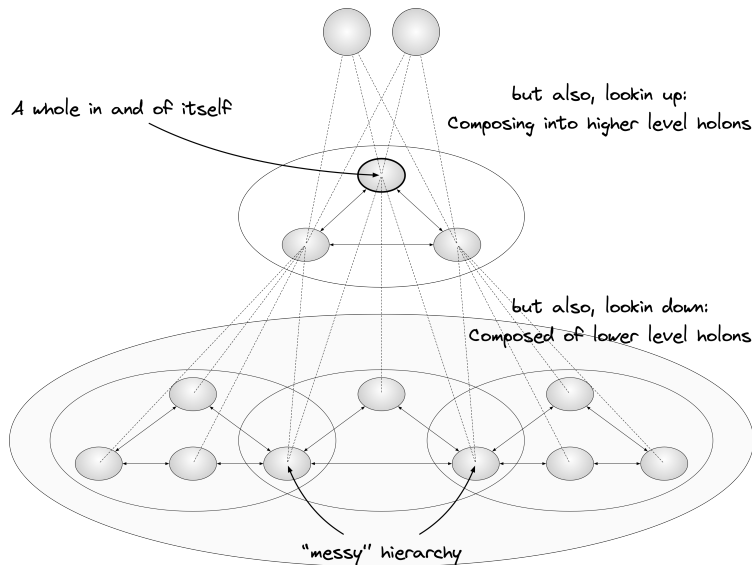


Figure 10. Holons and holarchies

Holons and holarchies can be found in various fields, such as physics, biology, and sociology, and are often used to describe complex systems composed of smaller, interconnected parts.

While not as common, holons and holarchies have been an incredibly useful tool for me in thinking about distributed systems. Most complex distributed systems do not have clean-cut hierarchies, they exhibit complex forms of organization. Holons and holarchies can express these complex and "messy" forms of organization, allowing us to group, zoom in, or zoom out to capture the relevant aspects of a system accurately and concisely.

Example

As an example, let's consider a multi-tenant, replicated database management system. The database management system

replicates data through a consensus algorithm, such as Paxos, Raft, or Viewstamped Replication.

Let's assume there are two tenants, each provisioned one database. Let's also assume there are three database nodes.

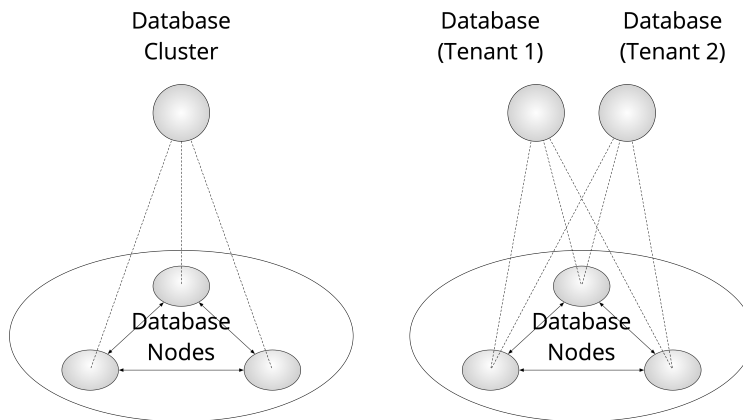


Figure 11. Two different holarchies representing the same system

Depending on our perspective or the point we want to make, we may view the system in different ways:

On the one hand, we may think in terms of one atomic entity, the database cluster. Alternatively, we may think of the database cluster as a higher order entity, composed of three interacting database nodes.

On the other hand, we may think in terms of two atomic entities, two databases, one per tenant. Alternatively, we may think of each database as higher order entities, composed of the interacting database nodes.

The choice of which components to consider atomic or higher-order entities, and which components to include in our analysis, is made by us and can vary even for the same distributed system, depending on the context.

Question 1. Can you already guess how **this** realization will help us later to define services and microservices?

Global View vs. Local View

In the preface of this book we briefly outlined the idea of a global view and a local view. Recall that we are considering the system from the perspective of an all-knowing observer, meaning that we have the ability to observe the state of the entire system, **we have a global view**. A component does not have the same luxury, a component can only observe its own state and its own channel to the network, **giving it a limited, local view**.

From a global point of view, we can observe the entire system, that is, we can determine the system state.

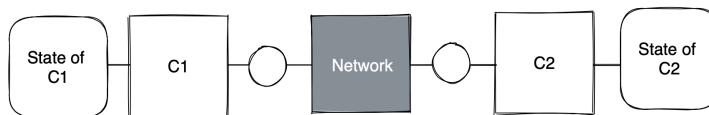


Figure 12. Global point of view, there is C1 and C2.

From a local point of view, a component can only observe itself, that is, it can only determine its own state. To determine a global system state, a component must enlist the cooperation of other components that must send their recorded local states via the network.

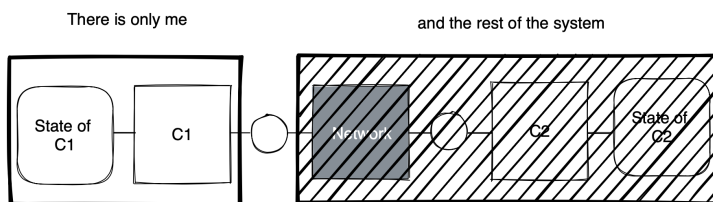


Figure 13. C1's point of view, there is only C1 and the rest of the system

AHA! Moment • Who's There?!

Typically, we think about communication as a direct message exchange between two components, C_1 and C_2 . However, it is important to understand that C_1 is not communicating directly with C_2 , but rather with rest of the system: C_1 sends a message that is addressed to C_2 to the network. The network may or may not forward the message to C_2 or any other component.

*Question 2. Can you already guess how **this** realization will help us later to define services and microservices?*

Why Distributed Systems

At this point we are well equipped to answer the question why and when we need to transition from a non-distributed system to a distributed system. After all, distributed systems are hard to build, so why go through the trouble of building them?

We assess the fitness of a system in terms of correctness, scalability, and reliability. In other words, a system shall provide the intended function even in the presence of load and in the presence of failure.

A single component can be functional, but a single component cannot handle an infinite amount of load nor a single crash failure. We need more than one component to be functional *and* scalable and reliable!

We need a distributed system. We need redundancy, that is, duplication and coordination to handle an increased amount of load and an increased amount of failure.

Conclusion

A distributed system is a set of concurrent, communicating components that communicate by sending and receiving messages over a network.

Even for the same system, depending on our point of view, the set of components may be viewed differently; for example, as individual nodes, clusters, or databases. These components can be thought of as atomic entities or higher-order entities.

However, from the perspective of each component, there is only itself and the rest of the system.

Ultimately, we are interested in the guarantees a system provides. We reason about these guarantees in terms of correctness, that is, in terms of safety and liveness guarantees as well as scalability and reliability guarantees - even though, strictly speaking, we are inviting some redundancy into our thinking.

That redundancy is a reminder that we need to be open to reason about distributed systems in different terms and different models, first to be able to communicate efficiently and effectively with others, and second, to see the entire elephant.

Outlook

In the next chapter, we will examine various system, component, and network models, including synchronous, partially

synchronous, and asynchronous systems. We will also delve into different failure scenarios, such as component and network failures, and introduce the concepts of physical time, logical time, and causal consistency.

SYSTEM MODELS, ORDER & TIME



In the last chapter, we delved into an informal discussion to build a *comprehensible* mental model of distributed systems and their mechanics. In this and the next chapters, we will delve into a formal discussion to build a *comprehensive* mental model of distributed systems and their mechanics.

In this chapter we will define the notion of system models and explore widely used system models like synchronous and asynchronous distributed systems and explore the concepts of order, physical time, and logical time.

The goal of this chapter is to gain an understanding of the *state of affairs* or *forces at play* that determine and constrain the behavior of distributed systems.

Objectives:

- Understand synchronous and asynchronous systems
- Understand component and network behavior
- Understand the concepts of order, physical time, and logical time

System Models

We can't reasonably talk about a distributed algorithm or protocol without talking about the assumptions we made about the underlying distributed system. This set of assumptions is known as the system model.

A system model is a set of assumptions about the system's set of components, its network, and its timing behavior.

Algorithms and protocols that are correct under one system model may not be correct under another system model - *any* deviation may render an algorithm or protocol incorrect.

Examples of assumptions about the system

One System Model	Another System Model
Components may not fail	Components may fail
Messages may not get lost	Messages may get lost

One System Model

Another System Model

Clocks are perfectly
synchronized

Clocks are not perfectly
synchronized

AHA! Moment • Board Game

You can think of a system model as a board game: The game sets the stage and sets the rules while the players have to devise a strategy to achieve the objective of the game within the constraints of the rules. Even a slight change to the rules may render the players' strategy completely ineffective. Your carefully devised strategy, executed over many rounds, may unravel in that instant.

Theory & Practice

System models can take many forms, from more theoretical to more practical and everything in between.

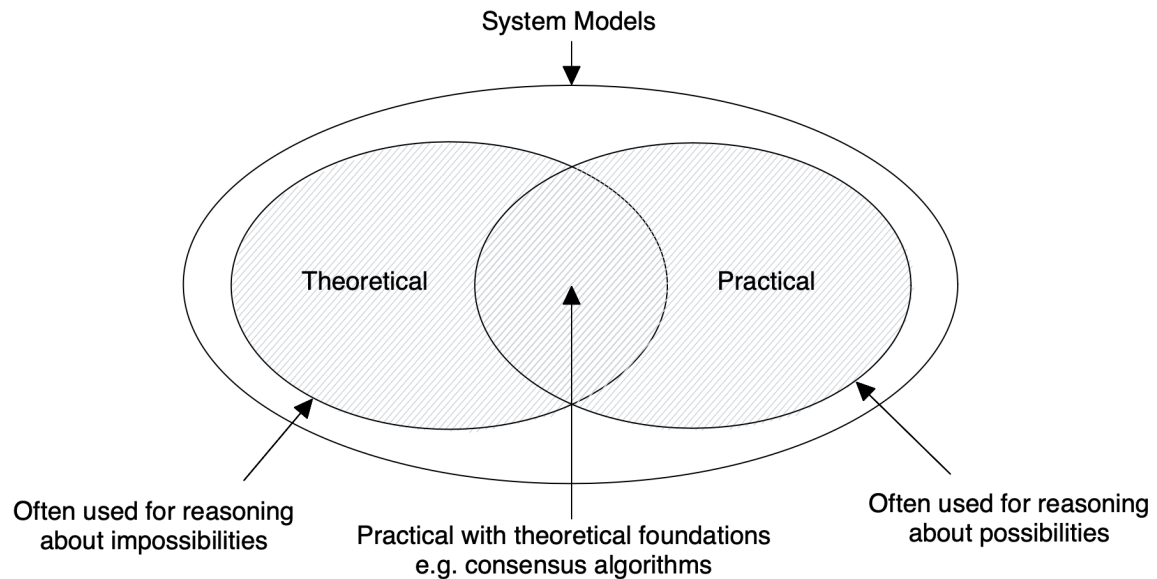


Figure 1. System Models

Theoretical system models are commonly used to reason about *impossibilities*. Computer scientists explore the theoretical limitations of a problem by investigating the feasibility of a solution and assessing the minimum and maximum bounds of the possible solutions in terms of the number of components involved or the number of messages exchanged.

Practical system models are commonly used to reason about *possibilities*. Engineers explore the practical considerations by assessing possible solutions, taking into account critical factors such as development costs, operational costs, and scalability and reliability properties.

Finally, there is the intersection of practical and theoretical system models, that is, system models that are equally relevant for theory and practice, e.g. system models that underlie popular consensus algorithms like Viewstamped Replication, Paxos, or Raft.

Synchronous Distributed Systems

Different authors think differently about synchronous distributed systems but each author assumes a *strong* notion of physical time.

In this theoretical system model, each component is equipped with a clock that is either perfectly synchronized with other clocks or has a known upper-bound deviation. Furthermore, internal and external events unfold in strict time or occur with a bounded delay.

So in effect, some authors prefer to think about a synchronous system as a system without *any* uncertainty about its timing and some authors prefer to think about a synchronous system as a system with *some bounded* uncertainty about its timing.

Note that the synchronous system model does not predicate component failure or message loss. These are separate assumptions which will be discussed in the sections to follow.

Asynchronous Distributed Systems

As with synchronous systems, different authors think differently about asynchronous distributed systems, but here the difference is significant:

- Some authors assume *no* notion of time.
- Some authors assume *a weak* notion of time.

Some problems do not have a solution assuming no notion of time yet do have a solution assuming a weak notion of time. So when an author mentions the term asynchronous system, first try to determine their notion of time and if their notion of time has a significance to their statements.

No notion of time

In this theoretical system model, no component has access to any kind of clock. Additionally, internal and external events occur in arbitrary time or internal and external events occur with arbitrary delay.

In this model, the notion of time does not exist, the most significant consequence being that a component cannot use timeouts. Timeouts are a crucial mechanism for failure detection and mitigation.

Weak notion of time

In this theoretical system model, a component has access to a clock that is not synchronized to other clocks and has an unknown deviation. Additionally, internal and external events occur in arbitrary time or internal and external events occur with arbitrary delay.

In this model, the notion of time does exist, the most significant consequence being that a component can use timeouts.

Note that as with synchronous systems, the asynchronous system model does not predicate component failure or message

loss. There are a different set of assumptions which will be discussed below.

Partially Synchronous Systems

Both, synchronous and asynchronous system models are theoretical system models. In reality, no distributed system behaves as entirely synchronous or entirely asynchronous. Instead, the system's behavior lies somewhere in between.

The partially synchronous system model acknowledges this reality and postulates that a distributed system is synchronous most of the time and asynchronous sometimes. This model matches both our experience as well as our expectations: components and networks are generally well behaved but sometimes they do act up.

A typical approach to designing protocols is to ensure both safety and liveness guarantees when the system operates synchronously, and to prioritize safety over liveness when the system operates asynchronously.

We will talk more about failure tolerance, that is, failure detection and failure mitigation in great detail in the next chapter of the book.

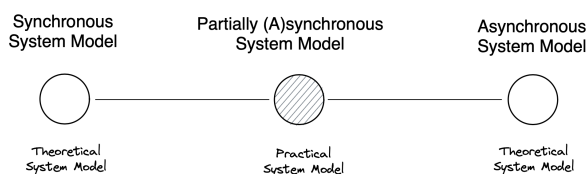


Figure 2. From Synchronous to Asynchronous System Models

Component & Network Behavior

Components and networks are frequently characterized by their failure behavior. We will discuss failure in the next chapter, so we will defer the definition for now.

Component Failure

System models for components vary depending on the types of failures that may occur in a component. Figure 3. illustrates typical failure models, but it is important to note that the definitions of failure models can vary across different authors.

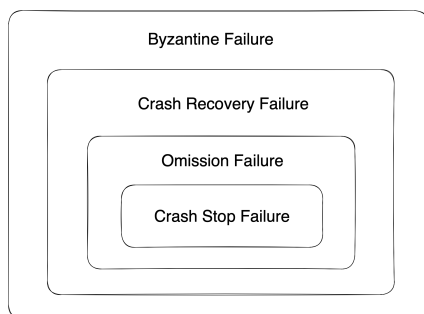


Figure 3. Component Failures

Crash-Stop Failures

Crash-Stop failures are the most basic failure model. When a component experiences a Crash-Stop failure, it stops to execute any internal or external steps at an arbitrary moment in time and never performs any further internal or external steps again.

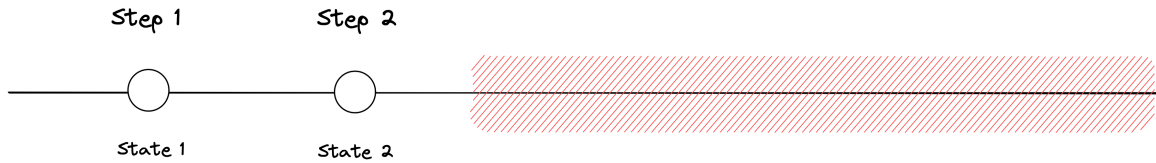


Figure 4. Crash-Stop Failure

To put it simply, in effect, a component ceases to exist.

Omission Failure

If a component experiences an Omission failure, it stops to perform any internal or external step for an arbitrary duration at an arbitrary moment in time. Then the process resumes to perform internal and external steps again.

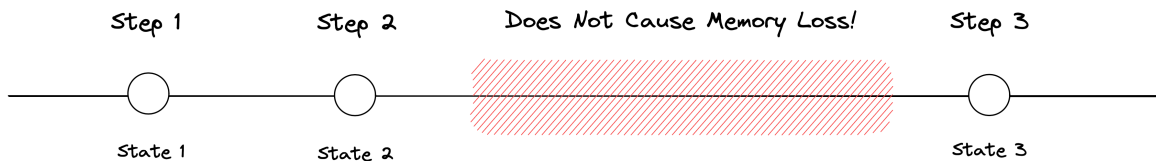


Figure 5. Omission Failure

To put it simply, in effect, a component takes a break.

Crash-Recovery Failure

In a Crash-Recovery failure, it stops to perform any internal or external step for an arbitrary duration at an arbitrary moment in time. Then the process resumes to perform internal and external steps again. However, unlike in the case of Omission Failure, the component may lose its state.

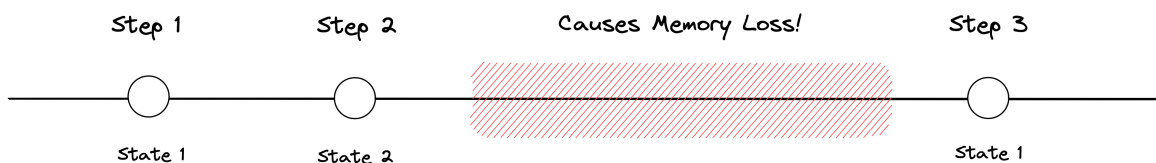


Figure 6. Crash Recovery Failure

To put it simply, in effect, a component may suffer memory loss.

Byzantine Failure

A Byzantine failure occurs when a component behaves in an arbitrary manner, which includes actions of deception like intentionally deviating from its intended algorithm.



Figure 7. Byzantine Failure

To put it simply, *anything* goes.

Network Failure

Like in the case of components, system models for the network differ according to the failures that may occur in a network.

Message Reordering

A network may not respect the order of messages, that is, a network may deliver messages in a different order than they are received.

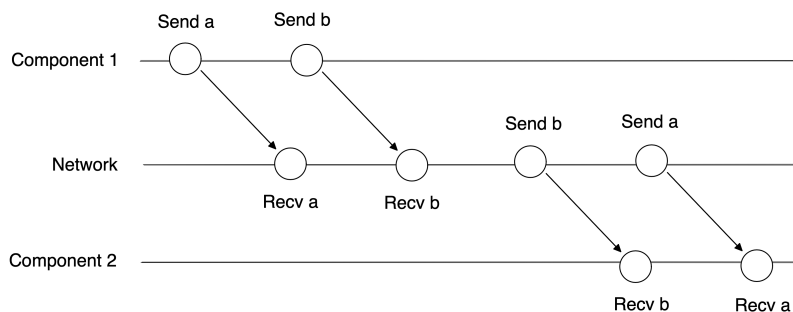


Figure 8. Message Reordering

Message Duplication

A network may send a message to the receiving component more than once, that is, the component may receive duplicate messages.

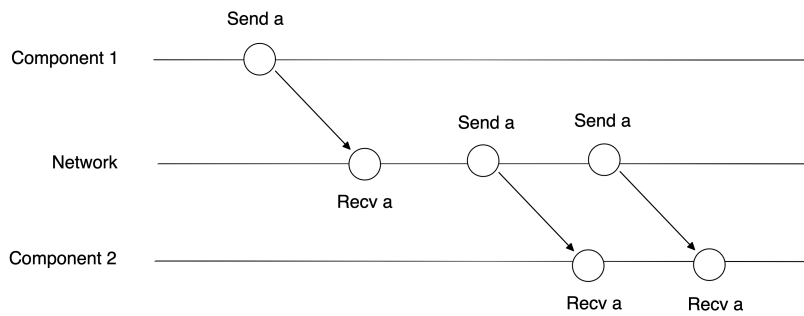


Figure 9. Message Duplication

Message Loss

A network may not send a message to the receiving component at all.

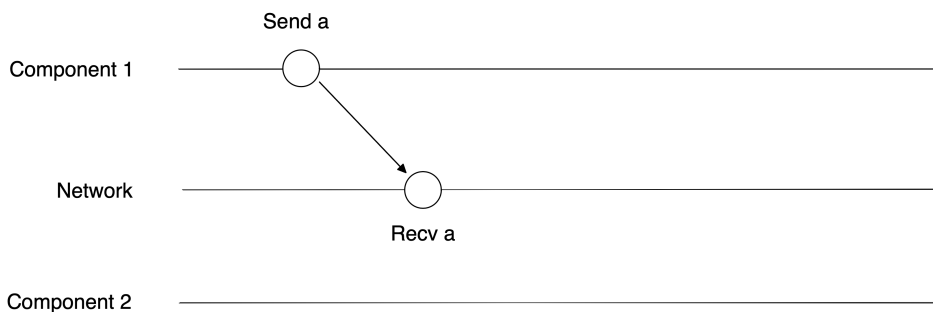


Figure 10. Message Loss

However, in general, it is assumed that a network cannot lie. If a network delivers a message to a component, it is presumed that the message was actually sent by some component in the system.

A network that may experience Message Reordering, Message Duplication, and Message Loss is commonly referred to as an

unreliable network.

Realistic System Models

With the knowledge we have acquired thus far, we can now identify practical system models - a set of practical assumptions regarding distributed systems.

In practical terms, distributed systems are generally synchronous most of the time and asynchronous at times. Components may fail due to crashes, but they may also recover, forgetting some state (volatile) but remembering other state (durable). The network may reorder, drop or duplicate messages.

More formally, distributed systems are partially synchronous consisting of components subject to Crash-Stop failure, Omission failure, and Crash-Recovery failure communicating over an unreliable network.

Although Byzantine failures are relevant for some domains like cryptocurrency, we will largely ignore them in this book.

This is the system model I have in mind when I think about distributed systems.

Order & Time

While the section on order and time may not be the most exciting read for some, it's essential to grasp the concepts of event ordering, physical time, and logical time as they will

accompany you throughout your entire journey in the distributed system realm - both in theory and in practice. A solid foundational understanding of these concepts is invaluable for success.

Additionally, I've included two of my favorite AHA! Moments to look forward to: a clear definition of Race Conditions and a definition of Concurrency versus Parallelism.

Let's take a moment to revisit the definition of a distributed system: A distributed system is a set of concurrent, communicating components that communicate by sending and receiving messages over a network.

However, components are not *merely* communicating, they are *collaborating and coordinating*. Here, collaboration and coordination refer to the management of dependencies between the steps of components.

In general, actions are not commutative: The result of collaboration and coordination depends on the order that actions are applied.

Therefore, establishing the correct order of actions is crucial for correct results. The question then arises, how do we determine this correct order?

Let's examine a practical example to illustrate the concept. Consider two proposers, P1 and P2, and two acceptors, A1 and A2, with both acceptors starting in the initial state of $A1 = A2 = 0$. The proposers broadcast requests to apply simple arithmetic operations, such as $(+2)$ or $(\times 2)$, to the acceptors. The acceptors apply the requests to their local state in the order they are received.

If both P1 and P2 broadcast (+2), the order of application does not affect the result since the operations are commutative. However, if P1 broadcasts (+2) and P2 broadcasts ($\times 2$), the order of application does affect the result since the operations are not commutative.

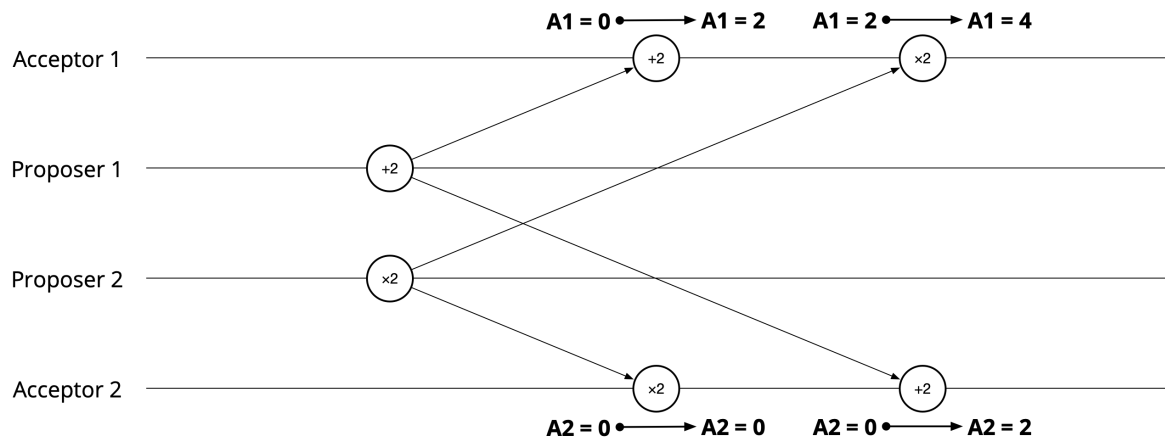


Figure 11. Proposers & Acceptors

Let's take a closer look at Figure 11. Even as all knowing observers, we cannot immediately tell which order of application is correct since both P1 and P2 broadcast their request *at the same time*. However, this result is unintuitive, this result feels wrong! Intuitively, we expect A1 and A2 to show the same value, we expect A1 and A2 to apply the operations in the same order. Since P1 and P2 broadcast their requests at the same time, the order of the operations does not matter as long as the same order is guaranteed.

AHA! Race Conditions

This is a great example of a **race condition**. While we may have a general understanding of a race condition, providing an accurate and concise definition can be challenging. Typically, we tend to rely on examples involving processes, threads, fibers, and shared memory.

In the context of this book a system contains a race condition if the system has multiple *possible* executions, some of which are considered correct and some considered incorrect.

For instance, if proposers P1 and P2 can broadcast either only (+n) **or** only (×n), the system does not contain a race condition because the delivery order does not affect the outcome at the acceptors. However, if P1 and P2 can broadcast both (+n) and (×n), the system does contain a race condition because only specific delivery orders can produce the same result at the acceptors.

Personally, I think this is the most abstract and broadly applicable definition of a race condition.

A potential solution to the race condition in this example is to introduce a Coordinator C. Instead of broadcasting their requests to A1 and A2, P1 and P2 send their requests to C, which initially has a state of $C = 0$. When C receives a request, C tags each request with its current state, broadcasts the request to A1 and A2, and then increments the value by 1. A1 and A2 keep track of the latest request they have applied. If an acceptor receives a request with a larger tag, they realize they have skipped some steps and can delay applying the requests until they receive the missing requests.

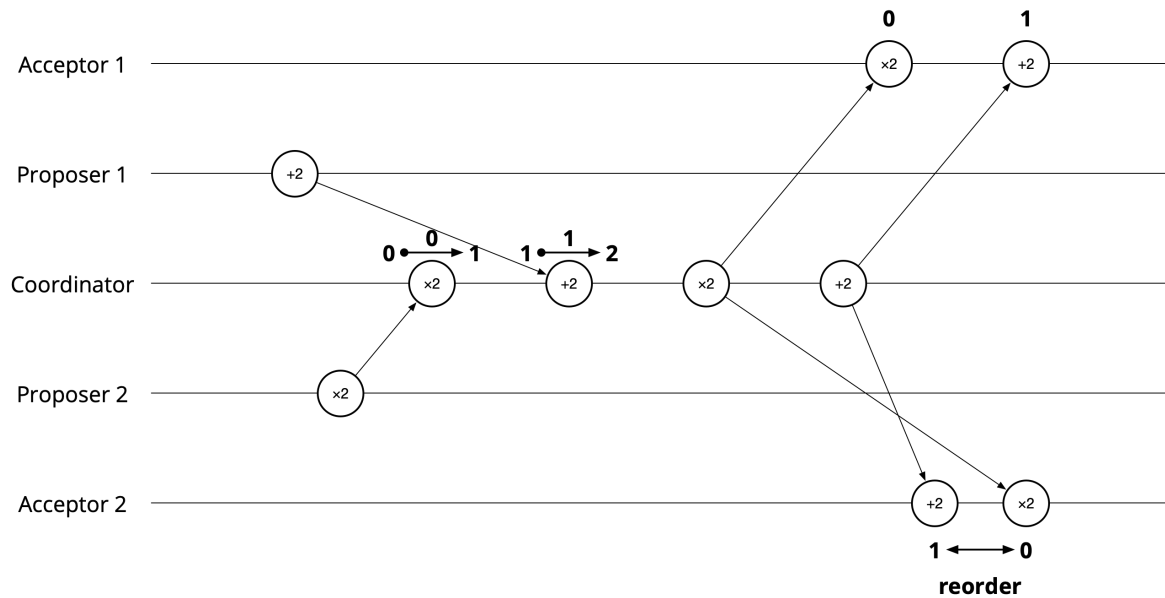


Figure 12. Proposers, Acceptors, & a Coordinator

Let's take a closer look at Figure 12. As all-knowing observers, we see that P1 sends its request (+2) before P2 sends its request ($\times 2$). However, the network delivers P2's request ($\times 2$) before P1's request (+2). Is that a problem? P1 and P2 did not coordinate their requests, and neither can expect to be first, as they can only observe themselves, not each other. Since C cannot observe P1 or P2 sending their requests, any delivery order is acceptable. C tags P2's request with 0 and P1's request with 1 before broadcasting $\langle (\times 2), 0 \rangle$ and $\langle (+2), 1 \rangle$ to A1 and A2. This way, regardless of the order in which the requests arrive at A1 and A2, the acceptors can reorder them before applying them to their local state.

Both order and the ability to establish order are fundamental to many distributed system problems and their solutions.

The Happened-before Relationship

In his seminal and groundbreaking paper [Time, Clocks, and the Ordering of Events in a Distributed System](#) from 1978, Leslie Lamport recognizes the importance of the order of events and explores its relationship to physical time and logical time. In this paper, Leslie Lamport introduced a formal framework that is widely used today: **The happened-before relationship**.

Leslie Lamport defines the happened-before relationship in terms of intra and inter component relations of events:.

Intra Component

If events a and b occur **at the same component** and **the occurrence of a precedes the occurrence of b**, then event a happened-before event b.

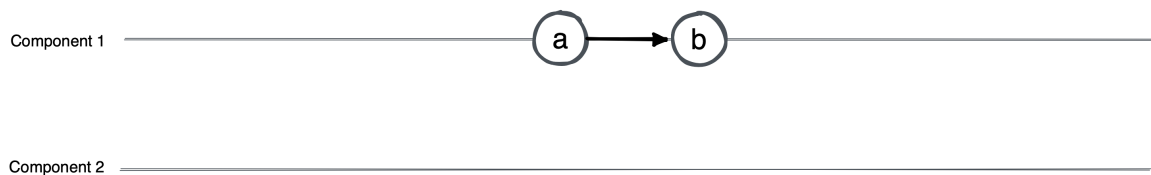


Figure 13. Happened-before, intra component

Inter Component

If event a and b occur **at different components** and **a is a Send Message event and b is the corresponding Receive Message event**, then event a happened-before event b.

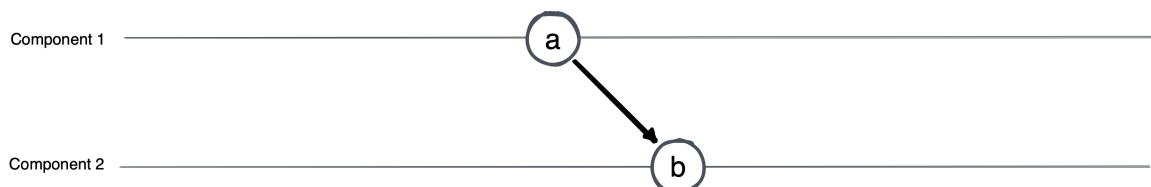


Figure 14. Happened-before, inter component

The happened-before relationship is a partial order, that is, for each pair of events a and b, either a happened before b, b happened before a, or a and b are concurrent.

a happened-before b	$a \rightarrow b$
b happened-before a	$b \rightarrow a$
a and b are concurrent	$a \parallel b = b \parallel a$

Transitivity

Unsurprisingly, the happened-before relationship is transitive, meaning that if event a happened before event b, and event b happened before event c, then event a also happened before event c.

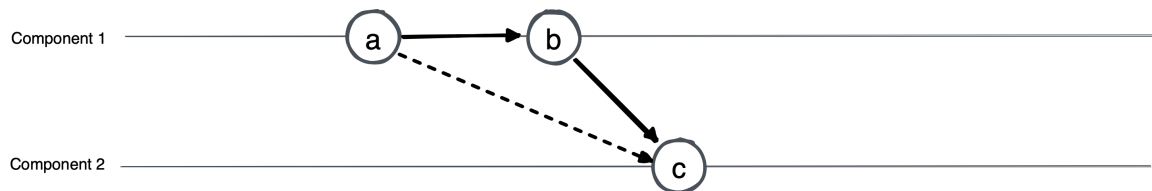


Figure 15. Happened-before, transitively

Causality

The happened-before relationship captures the causal relationship between events. Here, causal refers to the fact that an event a *potentially* influenced an event b, not that a *actually* influenced event b.

When you think about a distributed system, always think about the causality of events and carefully examine if there is a possibility of the system violating that causality.

AHA! Concurrency vs. Parallelism

We can rely on the happened-before relationship to accurately and concisely define concurrency and parallelism.

However, we first must address a mismatch between the entities involved in concurrency and parallelism. Concurrency is defined in terms of events, which are moments in time, while parallelism is defined in terms of operations, which are spans of time. To bridge this gap, we can think of an operation as a pair of start and end events.

By definition of the happened-before relationship, two events a and b are concurrent if neither a happened before b or b happened before a. By extension, two operations o1 and o2 are concurrent if neither the end event of o1 happened before the begin event of o2 or the end event of o2 happened before the begin event of o1.

Two operations are parallel if the operations overlap in time.

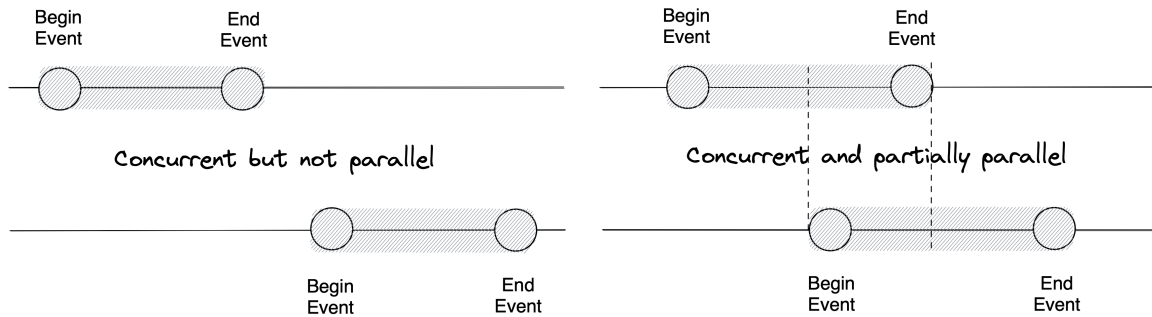


Figure 16 Concurrency vs. Parallelism

🕒 Concurrency is determined by a logical clock and logical time

🕒 Parallelism is determined by a physical clock and physical time

Which brings us straight to time and clocks.

Time & Clocks

Now that we understand that events are ordered and the importance of that order, we need to consider how the components of a distributed system can capture that order.

To do this, distributed systems rely on clocks as a source of timestamps. However, to ensure that clocks can be used to accurately order events, clocks must be able to guarantee *clock consistency*: Clock consistency states that if event a happened-before event b then the timestamp of a is less than the timestamp of b.

$$a \rightarrow b \Rightarrow C(a) < C(b)$$

If clocks are consistent, then they can be used to reason about the causal ordering of events in the system, even if those events occur on different components.

There are two different types of clocks, physical clocks and logical clocks. Both are used in distributed systems for ordering events, but they differ in their properties.

Physical Time & Physical Clocks

Physical clocks are physical devices and use physical time to timestamp events.

However, physical clocks in distributed systems face a major challenge: comparing timestamps from different clocks can be difficult due to clock skew and drift. These issues arise because physical clocks are neither perfect nor perfectly synchronized.

AHA! Wall Clock

As an all knowing observer with a global point of view, we can assume to have access to a wall clock, a hypothetical perfect physical clock. However, in reality, no component has access to this wall clock. Instead, if a component needs to measure time, it needs access to its own physical clock, which is an actual clock that is imperfect.

The wall clock is a hypothetical device that establishes a reference point for our discussions.

Clock Skew

Clock skew refers to the difference in time between two clocks in a distributed system.

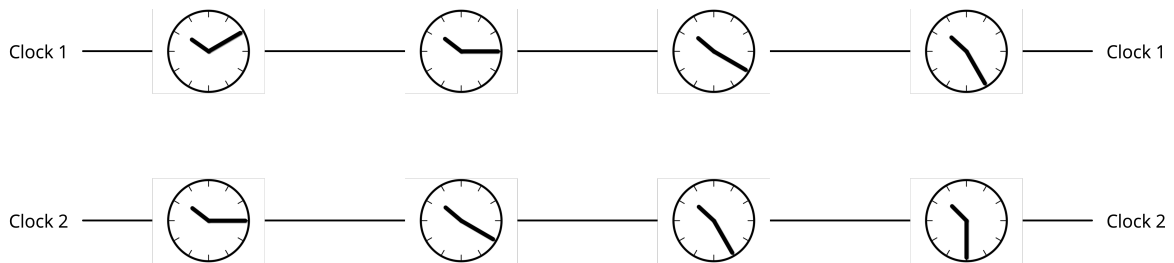


Figure 17. Clock Skew

Clock Drift

Clock drift refers to the difference in the frequency of ticks between two clocks in a distributed system.

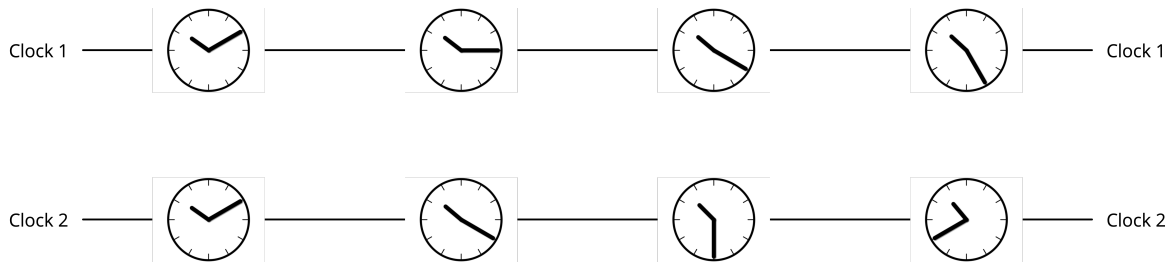


Figure 18. Clock Drift

Mitigation

Clock skew and clock drift complicate comparing timestamps and time intervals across different components in a distributed system. To mitigate this challenge, clock synchronization protocols like the widely used Network Time Protocol (NTP) are used.

NTP works by having one component, called the NTP server, broadcast its clock reading to other components in the system, known as NTP clients. The clients adjust their clocks to match the

reading of the server to mitigate the impact of clock skew and drift.

Time of Day Clock vs. Monotonic Clock

However, adjusting clocks can cause issues as well. If a clock needs to adjust backwards, time seems to move backwards.

To address this, hardware and software provide two types of clocks: time of day clocks and monotonic clocks.

- Time of day clocks provide a timestamp as close to wall clock time as possible, but may move backwards due to clock synchronization. Time of day clocks may violate clock consistency even when compared to itself.
- Monotonic clocks provide a timestamp independent of wall clock time, but guarantee that time will not move backwards. Monotonic clocks guarantee clock consistency when compared to itself.

Programming environments usually provide functions for both time of day clock as well as monotonic clock.

Time of day clock

```
var timestamp1 = System.TimeOfDay  
  
// Clock synchronization may happen here  
  
var timestamp2 = System.TimeOfDay  
  
// timestamp 2 may be less than timestamp 1!
```

Monotonic clock

```
var timestamp1 = System.Monotonic

// Clock synchronization may happen here,
// but monotonic clocks are unaffected

var timestamp2 = System.Monotonic

// timestamp 2 is always greater than timestamp 1
```

However, since monotonic clocks are independent of wall time, they can only be compared intra component, not inter component.

In conclusion, physical clocks are crucial in distributed systems, but they pose significant challenges, such as clock skew and drift. Being aware of these challenges and implementing appropriate measures to minimize their impact is essential.

Logical Time & Logical Clocks

In the same paper, Lamport proposed logical time and logical clocks to establish the order of events. Logical time is a concept that allows events to be ordered based on the happened-before relationship, their causality, rather than their physical time.

Logical clocks are logical devices and use logical time to timestamp events.

One of the most well-known logical clocks is the Lamport Clock, proposed by, you guessed it, Leslie Lamport. Each component in the distributed system maintains a Lamport Clock, a counter that is incremented with every internal or external step. Steps, or

rather the corresponding events, are tagged with the current value of the Lamport Clock. When a component sends a message to another component, the receiving component updates its own Lamport Clock value to be the maximum of its current timestamp and the received timestamp, plus one. This ensures that causally related events are assigned strictly, increasing logical time values.

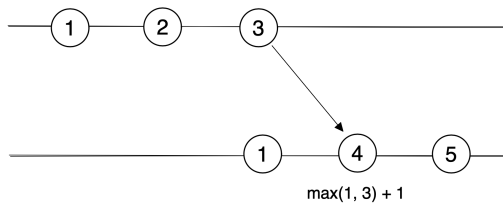


Figure 19. Lamport Clocks

Lamport Clocks and their extension, Vector Clocks, are widely known, widely discussed, but not necessarily widely implemented. Anecdotally, I have never used Lamport Clocks to capture the ordering of events.

However, Lamport Clocks and Vector Clocks are not the only manifestation of logical clocks. We can find logical clocks and logical timestamps in lots of places.

Let's take Apache Kafka as an example. Kafka is a messaging and streaming platform that organizes data into topics, which are further divided into partitions. Each message in a partition is assigned an offset that indicates its position within that partition. This offset allows consumers to track their progress through the partition.

Kafka ensures that messages are ordered within each partition. In other words, you can think of the partition as a logical clock and the message offset as a logical timestamp. If message a was enqueued before message b within the same partition (intra

partition), then the offset of message a is less than the offset of message b. However, message offsets cannot be compared across partitions (inter partition).

Another example of logical clocks can be found in Etcd, a distributed, linearizable key-value store. Etcd uses a sequence number for each key, and when a key is updated, the sequence number is incremented, allowing operations like compare-and-swap.

In Etcd, a key can be considered as a logical clock, and the sequence number as a logical timestamp. For each key, if the update operation a happened before the update operation b, the sequence number assigned to a is less than the sequence number assigned to b. However, just like Kafka, sequence numbers across keys in Etcd are not comparable.

So even if you do not see Lamport Clocks or Vector Clocks everywhere, if you squint just a little, you will see something that resembles logical clocks and logical timestamps A LOT.

Physical Time vs. Logical Time

Physical clocks and logical clocks are commonly used in tandem to create a sense of order and time in distributed systems.

Physical clocks are employed to measure the duration between events within a component, whereas logical clocks are employed to establish the order of events across components.

However, physical clocks can suffer from various issues in distributed systems, such as clock drift, skew, and synchronization problems, which may lead to inconsistencies in the ordering of events across the system. While logical clocks

are designed to provide a consistent ordering of events, even if physical clocks are out of sync.

Conclusion

In this chapter, we covered the topic of system models and examined common system models such as synchronous and asynchronous distributed systems. We explored the ideas of order, which is represented by the happened-before relationship, as well as the distinction between physical time and logical time.

Outlook

In the next chapter, we will discuss failure detection and failure mitigation. We will explore a formal definition of failures and their relationship to safety and liveness guarantees. In addition, we will explore the end-to-end argument in system design, application-level vs. platform-level failures, as well as transient, intermittent, and permanent failures.

References

1. Leslie Lamport, [Distributed systems, special relativity, and violating causality](#)

FAILURE TOLERANCE



In the previous chapter, we defined the notion of system models and discussed widely used system models like synchronous and asynchronous distributed systems, and explored the concepts of order, physical time, and logical time.

In this chapter, we will explore failure, failure tolerance, and failure handling. In short, this chapter will explore how to think about failure.

While reading the chapter, keep in mind that the primary objective of thinking about failure is to ensure *failure tolerance*, which refers to the guarantee that a distributed system functions in a well-defined manner even when failures occur.

Objectives:

- Understand failure and failure tolerance
- Understand failure detection and failure mitigation
- Understand application-level and platform-level failures
- Understand transient, intermittent, and permanent failures
- Outline for an ideal failure handling strategy

Terminology

The terms fault, error, and failure are subject to considerable ambiguity. What one author may refer to as a fault, another may label as an error or a failure. Although some authors attempt to distinguish between these terms, there is no universally accepted definition. This book will simply use the term *failure*. By using this term, we aim to reduce confusion and provide clarity in our discussion of these complex systems. As a result, we will also use the term *failure tolerance* instead of the more commonly used *fault tolerance*.

Introduction

The topic of failure, failure tolerance, and failure handling in distributed computing is broad, encompassing a significant body of theoretical and practical work. Therefore, this chapter is divided into two sections to provide a well-rounded perspective: the first section explores *thinking about failure in theoretical terms*, while the second section explores *thinking about failure in practical terms*.

The theoretical section draws heavily on the 1999 paper by F.C. Gärtner titled [Fundamentals of fault-tolerant distributed computing in asynchronous environments](#). This section explores the concepts of failure and failure tolerance in an abstract manner. The practical section draws heavily on the 1984 paper by J.H. Saltzer, D.P. Reed, and D.D. Clark titled [End-to-End Arguments in System Design](#). This section explores the concepts

of failure and failure tolerance in a concrete manner. We'll explore failure and failure tolerance in (micro)service-oriented distributed systems.

In Theory

Informally, a **failure** is an **unwanted but nevertheless possible** state transition of a system. On failure, the system transitions from a *good* state to a *bad* state. **Failure tolerance** is the ability of a system to behave in a well-defined manner when the system is in a bad state.

A formal approach to defining failure is based on the observation that a system can be in one of three types of states: legal states (the good state), illegal states (the bad states), and intolerable states (the 'everything is lost' states).

Additionally, a formal approach to defining failure is based on the observation that a system changes its state as a result of two types of transitions: normal transitions, and failure transitions.

These observations provide a comprehensive framework for understanding the concept of failure in distributed systems (Figure 1).

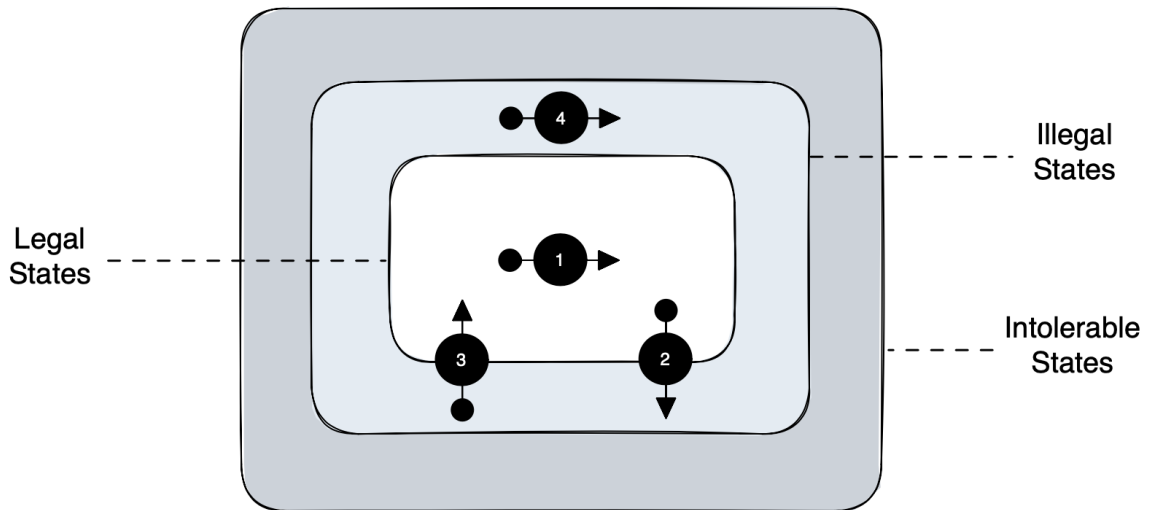


Figure 1. States & Transitions

1. State $\in$ Legal State, Transition $\in$ Normal Transition

If a system that is currently in a legal state executes a normal transition, then the system will transition to another legal state.

2. State $\in$ Legal State, Transition $\in$ Failure Transition

If a system that is currently in a legal state executes a failure transition, then the system will transition to an illegal state.

3. State $\in$ Illegal State, Transition $\in$ Normal Transition

If a system that is currently in an illegal state (repeatedly) executes a normal transition, then the system will transition to a legal state.

The sequence of transitions that lead from an illegal state to a legal state is also called failure recovery.

4. State $\in$ Illegal State, Transition $\in$ Failure Transition

If a system that is currently in an illegal state executes a failure transition, then the system will transition to another illegal state.

Where are Intolerable States & Transitions?

Fundamentals of fault-tolerant distributed computing in asynchronous environments excludes intolerable states and intolerable transitions from discussions as, by definition, we cannot tolerate and do not attempt to tolerate that situation.

The often cited and more than drastic earth-gets-hit-by-a-meteor example comes to mind - there are no contingencies. 🦖💥

Failure Tolerance

Legal states and illegal states are characterized by their safety and liveness properties. Recall from the first chapter:

- **Safety** guarantees that something bad will never happen.
- **Liveness** guarantees that something good will eventually happen.

We can define failure tolerance in terms of guaranteeing safety and liveness properties. In the absence of failure, a system always guarantees both its safety and liveness properties. However, in the presence of failure, we may have to compromise.

1. Masking Failure Tolerance

If a system guarantees both safety and liveness in the presence of failure, then the system provides masking failure tolerance.

Masking failure tolerance is the most desirable form of failure tolerance: Masking failure tolerance amounts to **failure transparency**. However, masking failure tolerance may either be too costly to achieve or simply impossible to achieve, forcing us to make choices.

Later in this book, we will discuss impossibility results, such as the CAP conjecture (Consistency or Availability under Partitioning), which prevent us from guaranteeing failure transparency in certain scenarios.

2. Non-Masking Failure Tolerance

If a system guarantees liveness but does not guarantee safety in the presence of failure, then the system provides non-masking failure tolerance.

Informally speaking, the system does not guarantee not to make any mistakes but the system does guarantee to make progress.

Consider the example of a queue that guarantees fully ordered (first in, first out) message delivery in the absence of failure. However, suppose the queue is non-masking failure tolerant. In that case, it will continue delivering messages in the presence of a failure, but messages may be delivered out of order for the duration of the failure.

3. Fail-Safe Failure Tolerance

If a system guarantees safety but does not guarantee liveness in the presence of failure, then the system guarantees Fail-Safe

Failure Tolerance.

Informally speaking, the system does guarantee not to make any mistakes but the system does not guarantee to make progress.

Continuing with the previous example, if the queue is fail-safe failure tolerant, it will stop delivering messages in the presence of a failure to prevent messages from being delivered out of order for the duration of the failure.

4. None of the above

If a system guarantee does not guarantee safety or liveness in the presence of failure then the system is simply not failure tolerant at all - Arguably in most cases, this is simply not acceptable.

Failure Tolerance		
	Safe	Not Safe
Live	Masking	Non-Masking
Not Live	Fail-Safe	😞

Example

Let's explore these concepts with the help of a simple example. The listing below illustrates a simple non-distributed system, consisting of one component, here called process P: The process has one variable x with the possible values 0, 1, 2, 3.

```
process P

  var  $x \in \{0, 1, 2, 3\}$  init 1

  transitions
    // normal transition
    ①  $x = 1 \rightarrow x := 2$ 
    ②  $x = 2 \rightarrow x := 1$ 
    // normal transition (repair)
    ③  $x = 0 \rightarrow x := 1$ 
    // failure transition
    ④  $x \neq 0 \rightarrow x := 0$ 
  end

end
```

As the creators of the system, we specify its correctness, that is, its desired safety and liveness properties:

- **Safety** Here, we define the system's safety property as the value of x is either 1 or 2.
- **Liveness** Similarly, we define the system's liveness property as there is a point in the future where x will be 1 and there is a point in the future where x will be 2.

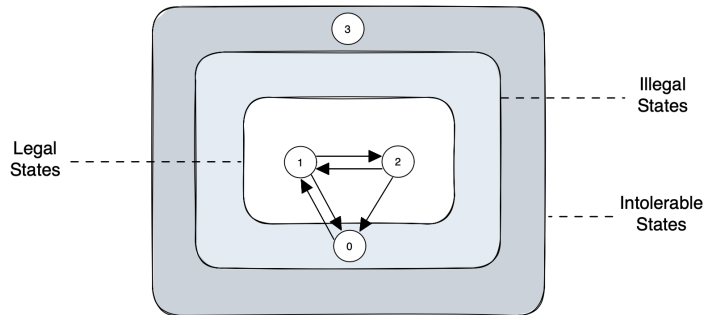


Figure 2. States & Transitions

So by definition, there are two legal states,

- $x = 1$
- $x = 2$

and one illegal state.

- $x = 0$

There are two normal transitions that are enabled when the system is in a legal state.

- ① $x = 1 \rightarrow x := 2$
- ② $x = 2 \rightarrow x := 1$

As well as one normal transition that is enabled when the system is in an illegal state, that is, a recovery transition.

- ③ $x = 0 \rightarrow x := 1$

However, there is also a failure transition, that is, an unwanted but nevertheless possible state transition.

- ④ $x \neq 0 \rightarrow x := 0$

The system can recover from failure with the help of transition 3.

This system is safe and live if $x = 1$ or $x = 2$. However, the system is not safe but is still live if $x = 0$. Therefore, this system is masking failure tolerant.

Note that the state $x = 3$ is intolerable. Should the system ever find itself in this state, there is no coming back.

In Practice

In the previous section, we established the theoretical foundations to think about failures. In this section, we will establish the practical foundation to think about failure. In addition, we will lay out an outline for a failure handling strategy in a (micro)service-oriented distributed system.

Note

Although retry mechanisms are a common approach to handling mitigation in (micro)service-oriented distributed systems, we will not cover them in depth in this chapter. We will explore retries in more detail in the next chapter, where we discuss message delivery and processing semantics.

System Model

For the remainder of this chapter, we will think in terms of service orchestration, which involves a service consumer component and one or more service provider components. These components interact through a request-response style message exchange.

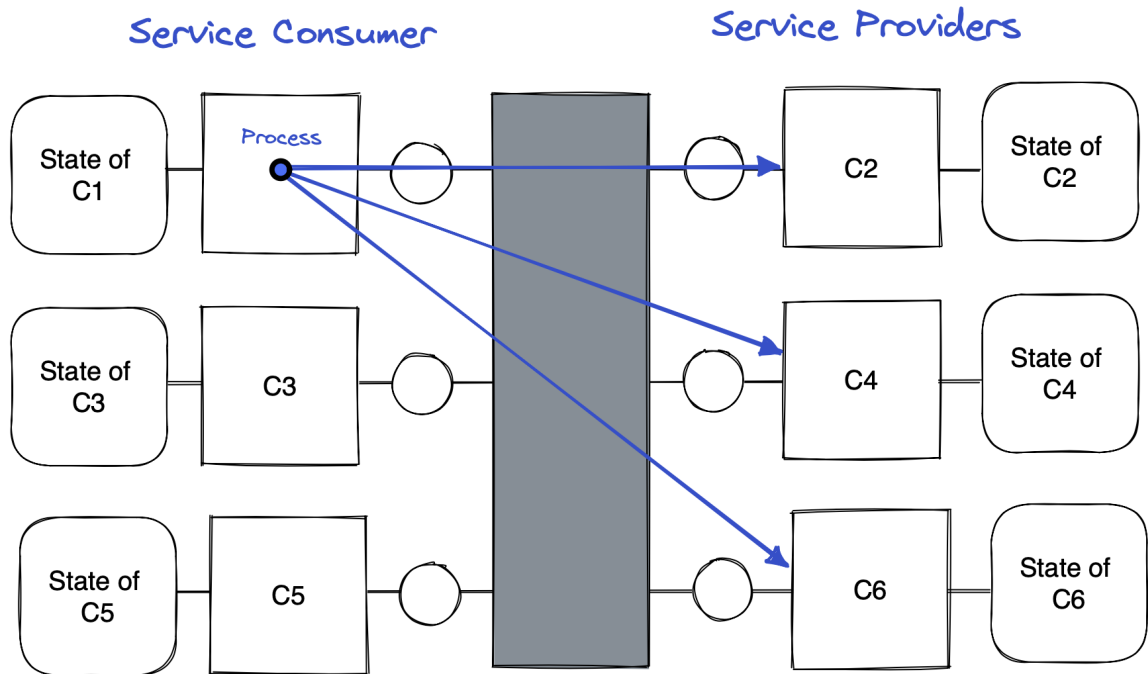


Figure 3. Service Orchestration

The service consumer executes a process, that is, a sequence of steps, where every step is a request to a service provider.

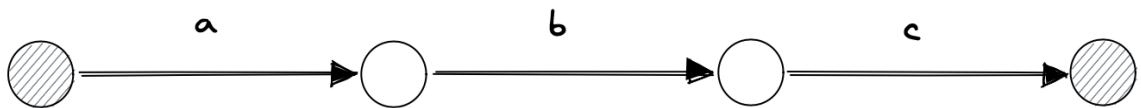


Figure 4. Process

Once a process is started, there are two possible scenarios.

- **Total Application:** In the absence of failures, the process successfully executes each step and transitions the system from a consistent state to another consistent state.
- **Partial Application:** In the presence of a failure, the process may only execute some steps, transitioning the system from a consistent state to a possibly inconsistent state.

In summary, a process is a sequence of steps where partial execution is undesirable. Therefore, in the event of a failure, we

need to ensure that the process executes in one of two ways:

- **Correct and desirable:** observably equivalent to a successful, total application (spoiler: forward recovery).
- **Correct but less desirable:** observably equivalent to no application (spoiler: backward recovery).

AHA! Moment • Completeness

Completeness is to distributed systems what ACID (Atomicity, Consistency, Isolation, and Durability) is to databases: a prerequisite for the proper execution of a sequence of steps. If it's incomplete, it's incorrect.

To illustrate our point, we will use the quasi-canonical example of an e-commerce checkout process. One step in this process involves charging the customer's credit card.

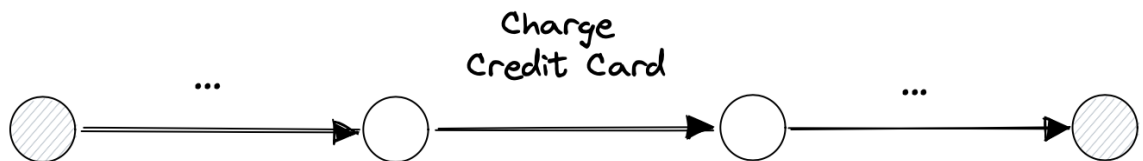


Figure 5. E-Commerce Process

It is imperative to ensure that the credit card is charged exactly once in the event of a successful checkout, and not charged at all if the checkout fails. Under no circumstances should the credit card be charged if goods are not shipped, or tickets are not issued.

Now let's handle some failures!

Failure Handling

Failure handling involves two main steps: failure detection and failure mitigation, that is, detecting that an unwanted event has occurred and then taking steps to mitigate its occurrence.

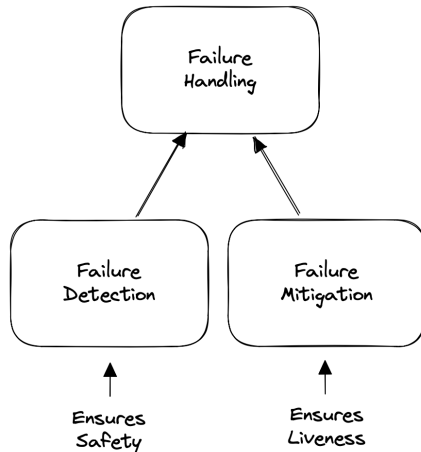


Figure 6. Failure Handling

AHA! Moment • Detection & Mitigation

The relationship between failure detection with safety and the relationship between failure mitigation with liveness did catch me by surprise.

In general, failure detection is the basis for guaranteeing safety: A commonly employed strategy is to detect a failure and subsequently inhibit “dangerous actions” to ensure the system maintains its safety.

Similarly, failure mitigation is the basis for guaranteeing liveness: A commonly employed strategy is to mitigate a failure to ensure the system resumes its liveness.

This relationship is demonstrated by consensus algorithms like Paxos, Raft, or Viewstamped Replication. Consensus algorithms rely on quorums to reach consensus, that is, to ensure their safety and liveness properties.

For example, when a node is unable to participate in the quorum (detection), it must refrain from responding to requests to ensure safety. Once the node rejoins the quorum (mitigation), it can resume responding to requests and resume liveness.

The relationship between these concepts is fascinating yet was not immediately obvious to me.

Failure Classification

To efficiently and effectively detect and mitigate failures, we must first understand their nature, that is, their classification.

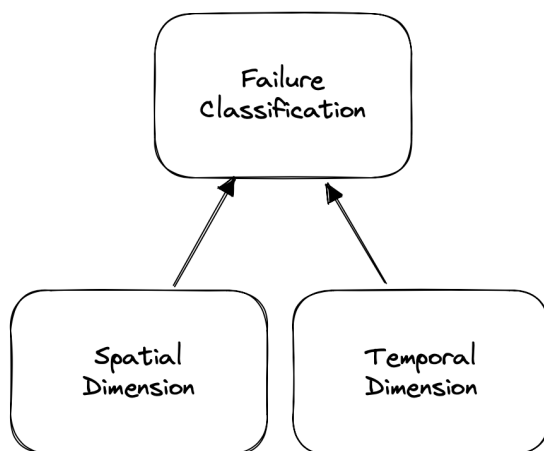


Figure 7. Failure Classification

Of course, failure can be classified in countless different ways, but here I want to focus on the classification illustrated in Figure 7 that guides my thinking. We will focus on two orthogonal dimensions, the spatial dimension and the temporal dimension.

Spatial Dimension

On the one hand, failures may be classified by *where* they occur, in other words, by location. However, we first need to add

another model to our repertoire: Thinking in **layers**.

Thinking in Layers

The mental model of layered architecture arranges components in layers. Components at a higher layer make a downcall to components at a lower layer, generally expecting a response. Less frequently, components at a lower layer make an upcall to components at a higher layer, generally via a previously registered callback.

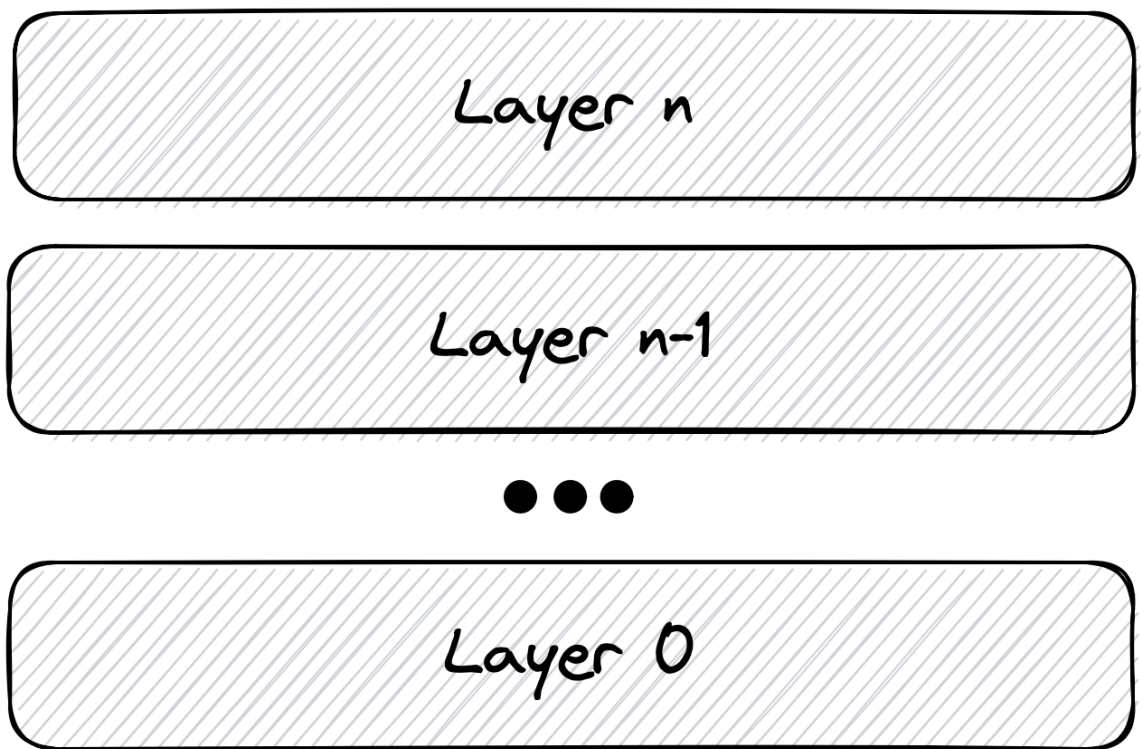


Figure 8. Thinking in Layers

AHA! Moment • Application Layer & Platform Layer

"Seek to characterize things by the role they play in context rather than by some intrinsic

characteristic”, Dr. Eugenia Cheng, The Joy Of Abstraction

We tend to consider the application layer as the top-level layer and the platform layer as the layers below. However, application layer and platform layer are not absolute concepts, instead, they are relative concepts.

From the point of view of any given layer, *that* layer is considered the *application layer* while *all lower layers* are considered the *platform layer*.

For instance, an operating system sees itself as the application layer and the hardware as the platform layer. Conversely, a database management system regards itself as the application layer and views the operating system and hardware as the platform layer.

Sometimes, we reason across three layers: the application layer, the platform layer, and the infrastructure layer. So from that point of view, the database management system is the application layer, the operating system is the platform layer, and the hardware is the infrastructure layer.

As discussed in Chapter One, when thinking in distributed systems, keep in mind that different authors may conceptualize the layers of a software system differently. Additionally, authors may associate the same components with different layers depending on the argument they are trying to make.

Classifying Failures as Application-Level or Platform-Level

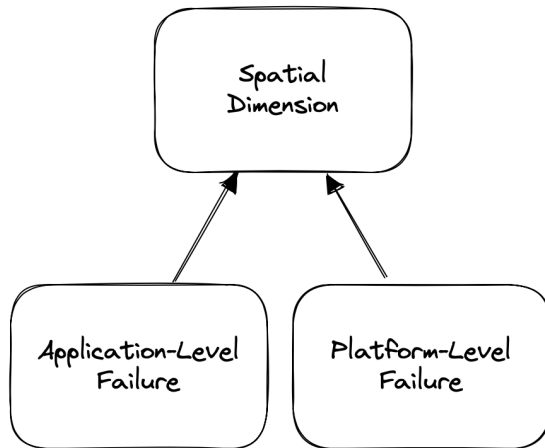


Figure 9. Spatial Classification

The End-to-End Argument states that, in a layered system, failure handling should be implemented in the lowest layer possible (viewed from the top down) that can **correctly and completely** handle failure detection and failure mitigation.

A failure can be classified as either an application-level failure or a platform-level failure, depending on the lowest layer that is able to detect and mitigate the failure.

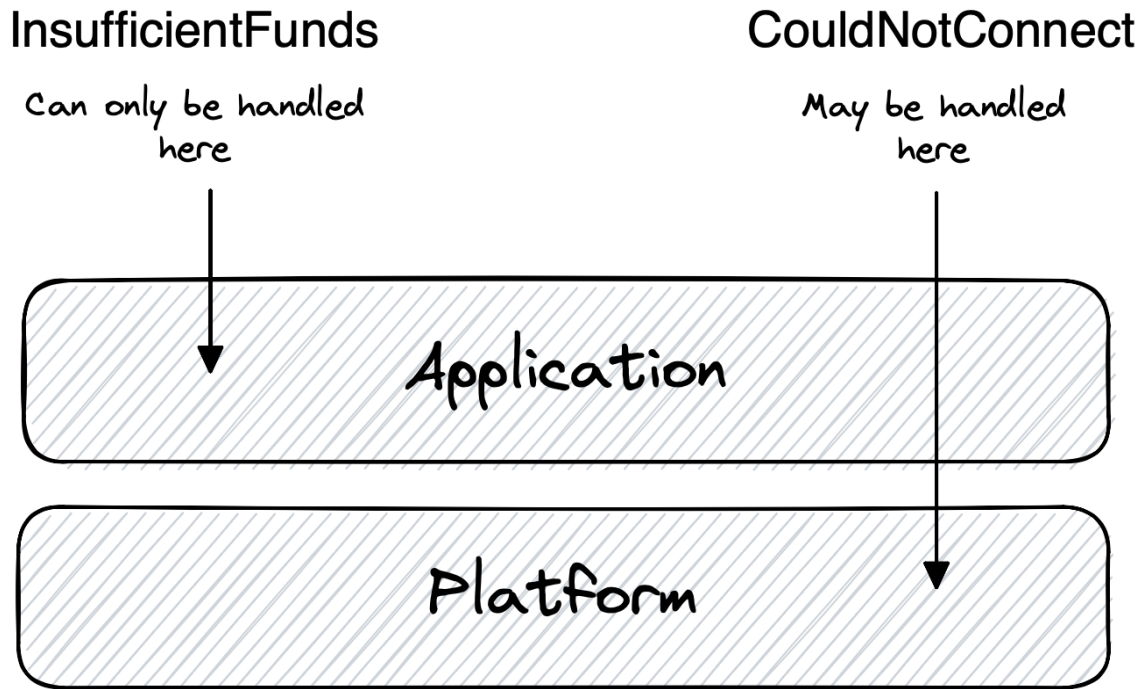


Figure 10. Application-Level vs Platform-Level Failure

For instance, an `InsufficientFunds` failure message indicates an application-level failure. The application-level is the lowest layer capable of correctly and completely resolving this failure; the failure is "meaningless" at the platform-level.

For instance, a `CouldNotConnect` failure message indicates a platform-level failure. Although the application could potentially mitigate the failure, the lowest layer capable of correctly and completely mitigating that failure is the platform-level, which can do so by retrying to establish the connection.

Temporal Dimension

On the other hand, failures may be classified by *when* they occur, in other words, by time.

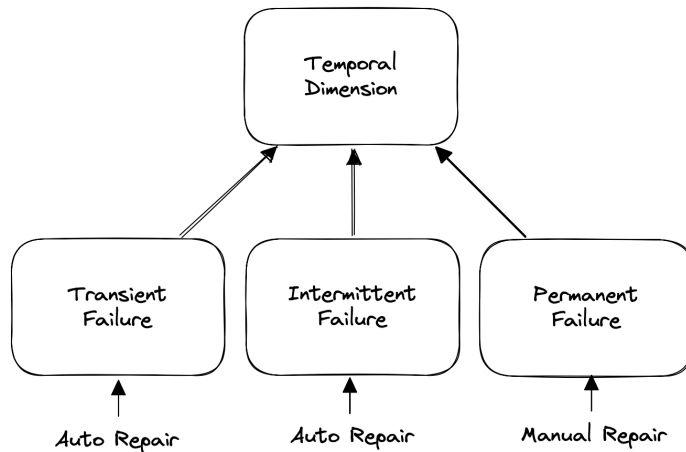


Figure 11. Temporal Dimension

Failures can be classified as transient, intermittent, or permanent. These classifications can be expressed as conditional probabilities, specifically the probability of a second failure occurring given that a first failure has already occurred.

In addition, transient, intermittent, and permanent failures can be distinguished by how they are repaired or resolved. Transient and intermittent failures often automatically repair, while permanent failures usually require manual repair.

Transient Failure

"Comes and goes"

If a failure is transient, we can reasonably assume that the probability of a second failure is not elevated.

Formally, a transient failure is defined by two characteristics. Firstly, the probability of a failure F_2 occurring after another failure F_1 is the same as the probability of F_2 occurring on its own. Secondly, transient failures are auto-repairing and resolve themselves without any intervention.

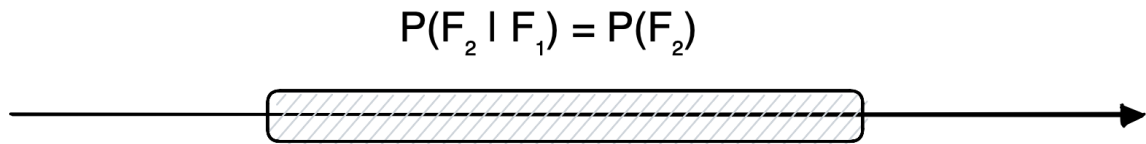


Figure 12. Past failures do not increase the probability of future failures

Example

In our example, if the cause of the failure is a router restart, then `CouldNotConnect` may be a transient failure. This type of failure auto-repairs quickly; once the router has restarted, the connection can be made.

Intermittent Failure

“Lingers”

If a failure is intermittent, we can reasonably assume that the probability of a second failure is elevated.

Formally, an intermittent failure is defined by two characteristics. Firstly, the probability of a failure F_2 occurring after another failure F_1 is higher than the probability of F_2 occurring on its own. Secondly, intermittent failures are also auto-repairing and resolve themselves without any intervention.

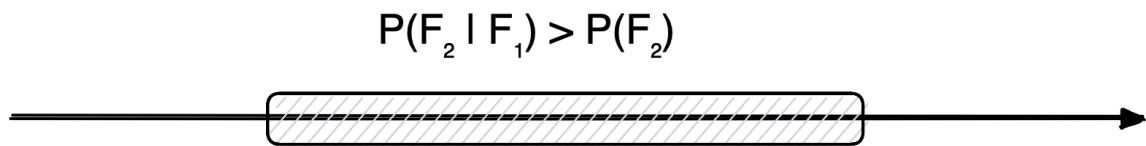


Figure 13. Past failures increase the probability of future failures

Example

In our example, if the cause of the failure is an outdated routing table, then `CouldNotConnect` may be classified as an intermittent

failure. This type of failure auto-repairs, but with some delay; once the router updates its routing table, the connection can be made.

Permanent Failure

“Here to stay”

If a failure is permanent, we can reasonably assume that a second failure is certain.

Formally, a permanent failure is defined by two characteristics. Firstly, the probability of a failure F_2 occurring after another failure F_1 is 1. Secondly, permanent failures require manual intervention to resolve.

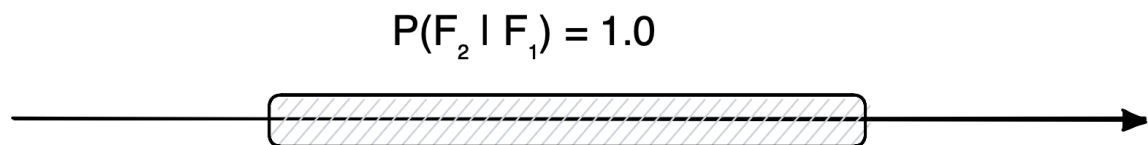


Figure 14. Past failures result in future failures

Example

In our example, if the cause of the failure is an expired certificate when attempting the connection, then `CouldNotConnect` may be classified as a permanent failure. This type of failure does not auto-repair and requires manual intervention; an operator must provide a new, valid certificate.

Failure Detection

The first component of failure handling is failure detection, which refers to a mechanism that detects if a failure has occurred.

However, I struggle with the notion of failure detectors in distributed systems. Most authors focus on detecting component failures such as crashes (Crash Stop, Omission, and Crash Recovery Failures).

In the paper titled Unreliable Failure Detectors for Reliable Distributed Systems the author describes failure detectors as follows: “We consider distributed failure detectors: each process has access to a local failure detector module. Each local module monitors a subset of the processes in the system, and maintains a list of those it currently suspects to have crashed.”

That is how the majority of blog posts, papers, or books on distributed systems that I am aware of present failure detectors.

However, when I think about failure detection and failure detectors, I prefer to cast a wider net. Let's recall the informal definition of failure: a failure is an unwanted, yet possible state transition that leaves the system in an illegal state.

In my perspective, a failure detector is **a witness**, or predicate, that confirms the occurrence of a failure - that is, an unwanted yet possible state transition that leaves the system in an illegal state. Often, these witnesses are integrated into the underlying algorithm or protocol itself.

To detect an application-level failure, it may be as simple as to examine the return code of a step. For instance, when attempting to charge a credit card, an `InsufficientFunds` response indicates a failure on the application-level.

Similarly, detecting a platform-level failure can also be as simple as examining the return code of a step. For instance, when attempting to charge a credit card, a `CouldNotConnect` response indicates a failure on the platform-level.

Nonetheless, we still need to detect complete unavailability. In this case, failure detection pertains to the widely-discussed mechanisms mentioned previously to identify whether a particular component is experiencing a component failure.

The failure detector has various implementations, but they all function by exchanging messages at set intervals between the observer and the observed component.

- In a pull scenario, the observer sends a request to the observed component at a set frequency and anticipates a response within a designated time interval.
- In a push scenario, the observed component sends messages, typically called "heartbeats," to the observer at a set frequency.

In this context, the predicate or witness of a crash is a timeout. If the observing component fails to receive a message from the observed component within a specified timeframe, it presumes that the observed component has crashed.

However, this assumption may be incorrect. Consider the following scenarios:

- In a synchronous system that communicates over a reliable network, a missing message does indeed indicate a failure with certainty.
- However, in a partially synchronous or asynchronous system communicating over an unreliable network, a missing message does not necessarily indicate a failure. The message may have been delayed (in the case of a partially

synchronous or asynchronous system) or lost (in the case of an unreliable network).

Failure detection is **complete** if the failure detector eventually identifies every actual failure and **accurate** if every identified failure is an actual failure. In other words, completeness implies no misses (false negatives), and accuracy implies no mistakes (false positives).

In a partially synchronous or asynchronous system communicating over an unreliable network, it is impossible to achieve a complete *and* accurate failure detector due to the inherent uncertainty of whether a message was delayed or lost.

AHA! Moment • Who Supervises The Supervisor?

When one component assumes the role of the observer and monitors another component acting as the observed, it raises the question of who monitors the observer.

In practice, there are many different observation, or supervision strategies, with Erlang arguably being most famous for its elaborate supervision trees.

However, there comes a point where we reach the end of the road and enter intolerable territory.

Failure Mitigation

The second component of failure handling is failure mitigation, which refers to a mechanism that resolves a (suspected) failure.

Broadly speaking, there are two failure management techniques: backward and forward recovery.

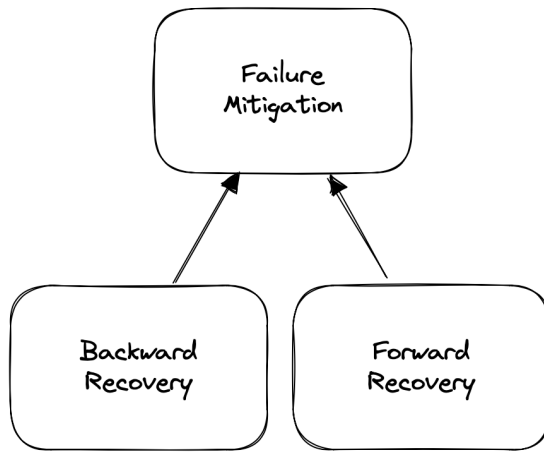


Figure 15. Failure Mitigation

Backward Recovery

Backward failure recovery refers to failure mitigation strategies that transition the system from the intermediary, illegal state to a state that is equivalent to the initial, legal state (move the process backward). As a rule of thumb, backward failure recovery does not require repairing the underlying cause of the failure, since we do not try to push past it.

Backward failure recovery is a common application-level failure mitigation strategy in the form of compensation.

Forward Recovery

Forward failure recovery refers to failure mitigation strategies that transition the system from the intermediary, illegal state to a state that is equivalent to the final, legal state (move the process forward). As a rule of thumb, forward failure recovery

requires repairing the underlying cause of the failure, since we do try to push past it.

Forward failure recovery is a common platform-level failure mitigation strategy in the form of retries.

Putting Everything Together

In this section, we provide an outline of an ideal failure handling strategy with the highest likelihood of successfully completing a process execution in the event of a failure.

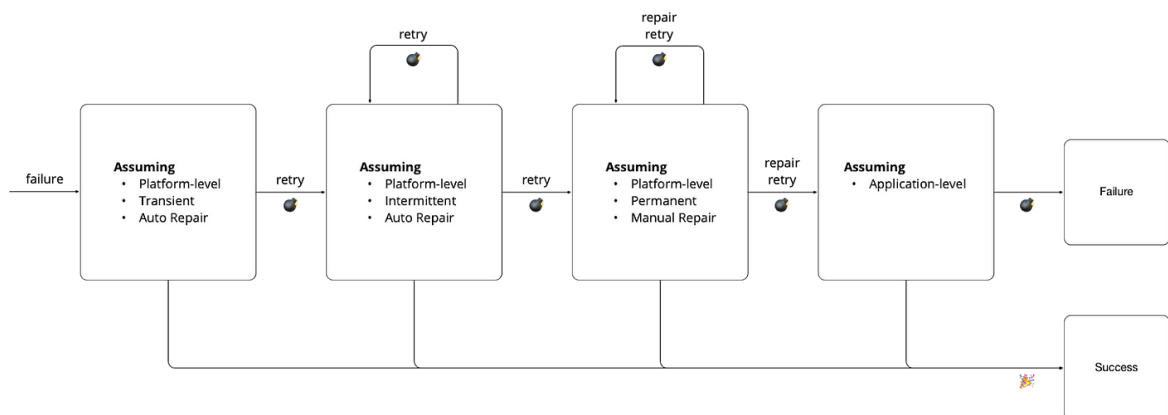


Figure 16. Outline of failure handling strategy in an orchestration scenario

Application-level Failure Mitigation

When a failure may unambiguously be classified as an application-level failure, the application layer is responsible to mitigate the failure.

For instance, the `InsufficientFundsException` in our example is an application-level failure that can be tackled with backward

recovery by compensating for the steps that have already occurred.

Platform-level Failure Mitigation

If a failure can not be unambiguously classified as an application-level failure or the failure can be unambiguously classified as a platform-level failure, the platform layer is responsible to mitigate the failure.

First, we assume a failure is a platform-level, transient, auto-repair failure; to mitigate the failure, we will issue an immediate retry.

Next, we assume a failure is a platform-level, intermittent, auto-repair failure; to mitigate the failure, we will schedule multiple retries with a backoff strategy.

Next, we assume a failure is a platform-level, permanent, manual-repair failure; to allow for mitigation, we will suspend the process, repair the underlying condition of the failure, and resume the process to retry.

If these mitigation efforts are ultimately unsuccessful, we have to elevate the failure to an application-level failure, presenting the failure to the application.

Be Vigilant

However, what happens if backward recovery encounters a failure? For example, let's say that in the course of backward

recovery, we need to refund a credit card charge, but the refund fails. What can we do?

We must elevate the failure from the application layer to the human layer. The operators of the software system are responsible for detecting and mitigating this failure. Worst case, someone must write and mail a check to refund the customer.

Conclusion

In this chapter, we covered failure, failure handling, that is, failure detection and failure mitigation, as well as failure tolerance. Failure in distributed computing is a broad area with an extensive body of theoretical and practical work - with one single objective: Guarantee that a distributed system functions in a well-defined manner even in the presence of failure.

Minimizing the impact of failures on the system and ensuring its continued functioning are the primary objectives of failure handling in distributed computing. Achieving this goal is a challenging yet essential task for building robust distributed systems.

Outlook

We will apply the concepts discussed in Chapters Two and Three to examine message delivery and processing in the upcoming chapter.

We will discuss at most once, at least once, and the much coveted exactly once delivery and processing guarantees. We will explore that *exactly* is impossible to achieve but that *effectively* once is an adequate substitute. Additionally, we will explore the conceptual and practical foundations necessary to achieve effectively once delivery and processing guarantees.

MESSAGE DELIVERY & PROCESSING



In Chapter 2, we defined the notion of system models and discussed widely used system models like synchronous and asynchronous distributed systems, and explored the concepts of order, physical time, and logical time. In Chapter 3, we explored failure, failure tolerance, and failure handling.

In this chapter, we will build upon the knowledge gained in Chapters 2 and 3 to explore message delivery and processing.

Objectives:

- Understand message delivery and message processing
- Understand at-least once, at-most once, and exactly once
- Understand equivalence and idempotence



Paul Cantrell
@inthehands@hachyderm.io

There are two race conditions hard
problems in distributed computing:
and guaranteed deli

Introduction

When talking about message delivery and processing, you may come across numerous misconceptions and instances of borderline deception. These can be detrimental to gaining accurate and concise mental models and can cloud our understanding.

For example, many authors use the term *exactly once processing*, but what they really mean is exactly once processing *semantics*.

For another example, marketing material may claim that a software product guarantees exactly once processing semantics to make the product more appealing without disclosing the necessary boundary conditions or volunteering that information.

If you have already established an accurate and concise mental model, misconceptions or misinformation will not affect you. However, if you are still developing that mental model, misconceptions or misinformation can hamper your progress.

In this chapter, we will be rigorously accurate and concise in order to develop a mental model that is also rigorously accurate and concise.

Overview

As defined in Chapter 1, a distributed system is a set of concurrent, communicating components that communicate by

sending and receiving messages over a network. Each component has exclusive access to its own local state. Changing the local state is known as a **(side) effect**.

As defined in Chapter 2, we assume a partially synchronous system model consisting of unreliable components communicating over an unreliable network that acts synchronously most of the time and asynchronously sometimes. Components may fail due to crashes, but they may also recover, forgetting some state (volatile) but remembering other state (durable). The network may reorder, drop or duplicate messages.

We do not allow Byzantine failures, that is, we do not allow components to behave in an arbitrary manner, which includes actions of deception like intentionally deviating from its intended algorithm.

For the remainder of this chapter, we will think in terms of two components having a dialogue: the sender and receiver interact through a request-response style message exchange.

Although both components send and receive messages, we refer to the component submitting the *request* as the *sender* and the component submitting the *response* as the *receiver*.

In this scenario, the sender sends a request to the receiver, who then processes the message, potentially causing a side effect, before sending a response to the sender.

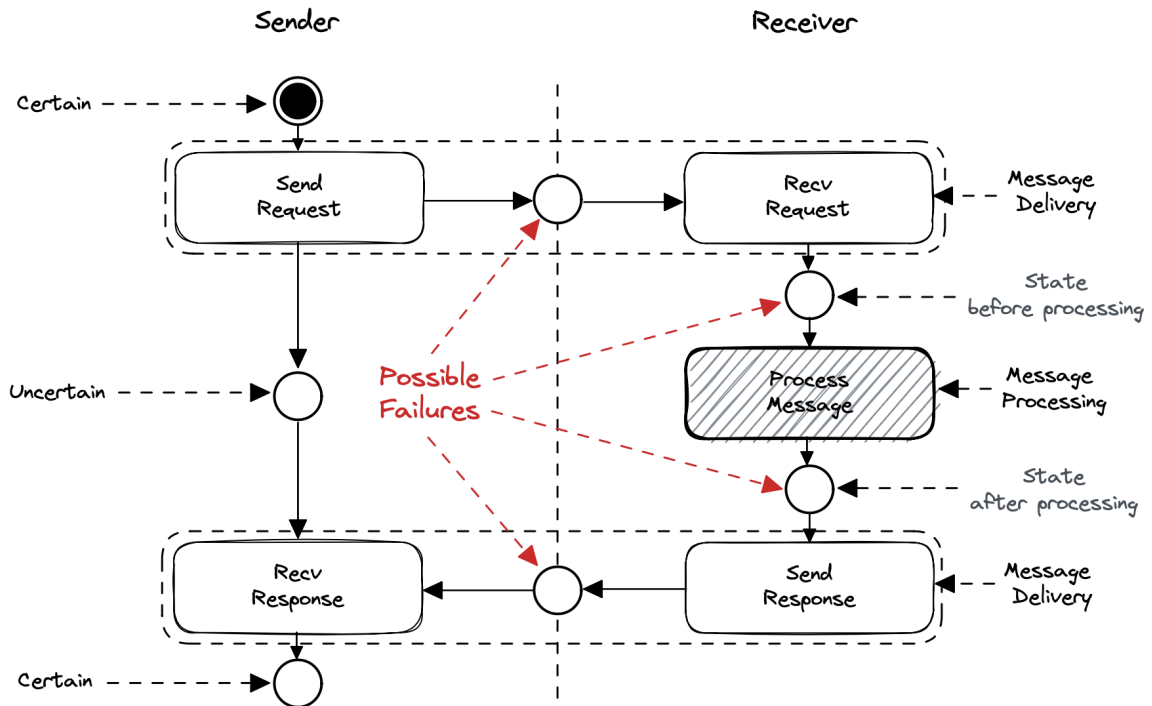


Figure 1. Request-Response Dialog

Figure 1. depicts a Petri net capturing one request-response dialog. Petri Nets model the dynamic aspects of concurrent systems: Places (circles) represent the control state of the system and transitions (rectangles) represent the events of the system. A transition is enabled if each input place contains a token and each output place does not contain a token. When a transition fires, the transition consumes the tokens from its input places and produces tokens in its output places.

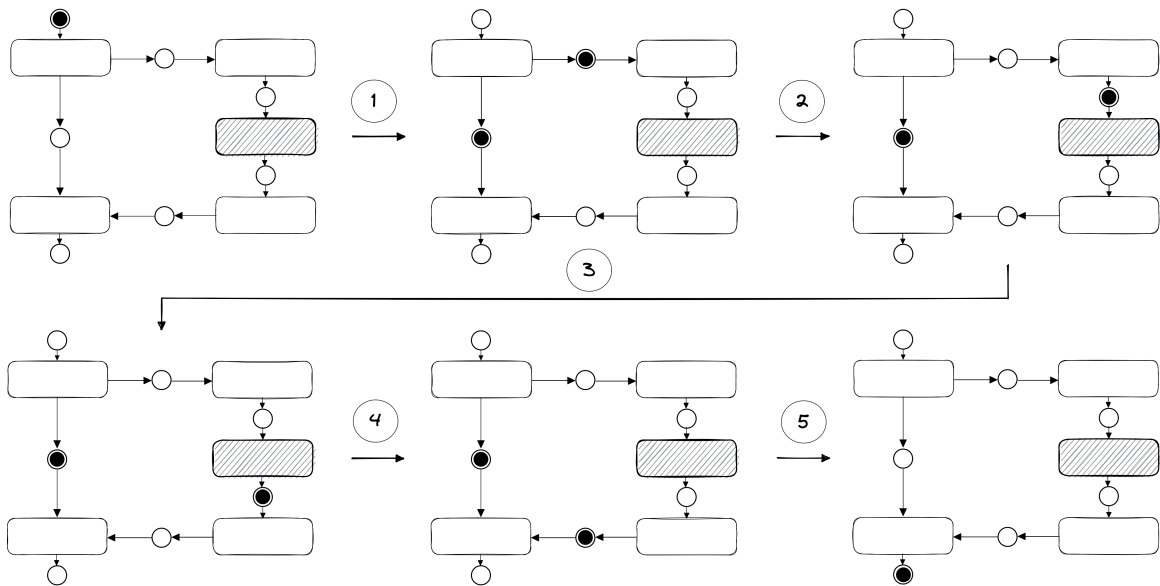


Figure 2. Step-by-step Progression

Figure 2. depicts the step-by-step progression of the request-response dialog, illustrating how the dialog unfolds:

1. The sender submits a request to the network.
2. The network delivers the request to the receiver.
3. The receiver processes the request, potentially causing a side effect
4. The receiver submits a response to the network.
5. The network delivers the response to the sender.

We can think of message exchange as a two-step process:
Message delivery and message processing.

Message delivery refers to the transfer of a message within a network, as well as the presentation of the message from the network to the component.

Message processing refers to both the generation of a response and the causing of a side effect.

Generally, related to at-most once, at-least once, and exactly once, our focus is on message processing rather than message

delivery. Message processing embodies the application logic, which includes any side effects.

This chapter explores message processing that occurs at-most once, at-least once, and exactly once.

Message Delivery

Message Delivery	On The Receiver Side
At-most once	The network delivers the message zero or one time
At-least once	The network delivers the message one or more times
Exactly once	The network delivers the message once

Message Processing

Message Processing	On The Receiver Side
At-most once	The receiver processes the message zero or one time
At-least once	The receiver processes the message one or more times
Exactly once processing	The receiver processes the message once

The Uncertainty Principle of Message Delivery & Processing

As discussed in Chapter 1 on Global View vs. Local View, a component can only observe its own state and channel to the network. This provides the component with a limited, local view (Figure 3.).

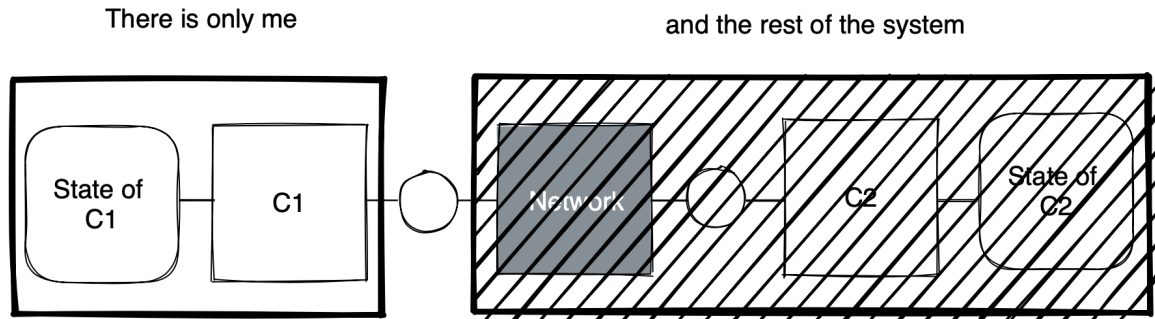


Figure 3. C₁'s point of view, there is only C₁ and the rest of the system

This limited view causes the sender to transition from a state of certainty to uncertainty, with the goal of ideally returning to a state of certainty (Figure 1., sender side).

Before sending the request

At this point, the sender can be certain that no work has been done and no side effects have occurred.

After sending the request, before receiving a response

At this point, the sender cannot be certain whether the work has been done, is about to be done, or will never be done. The sender cannot distinguish between the cases where

- the request was lost in the network,
- the receiver crashed before processing the message,
- the receiver crashed after processing the message,
- the response was lost in the network, or
- the network or the receiver is just slow.

After receiving a response

At this point, the sender can be certain again, either the response indicates a success, so the work has been done, or the

response indicates a failure, so the work has not been done (remember, no Byzantine failures, components are not allowed to deceive).

AHA! Moment • Distributed Systems and TCP

Q: Doesn't TCP prevent message loss in the network?!

A: Although TCP is considered a reliable protocol, TCP will not save you!

TCP is a reliable delivery protocol that guarantees accurate and complete transmission of data between the sender and receiver. This is achieved through the use of acknowledgments and retransmissions.

TCP is implemented in the operating system kernel and made available to applications via the socket interface. On both the sender and receiver sides, TCP writes data to a local buffer and then acknowledges receipt of the data.

If something goes awry, TCP retransmits data that has already been transmitted and deduplicates data that has already been received.

So are we safe? Unfortunately, we are not! The kernel acknowledges receipt before attempting to deliver data to the application, transitioning from at-least once delivery semantics to at-most once delivery semantics.

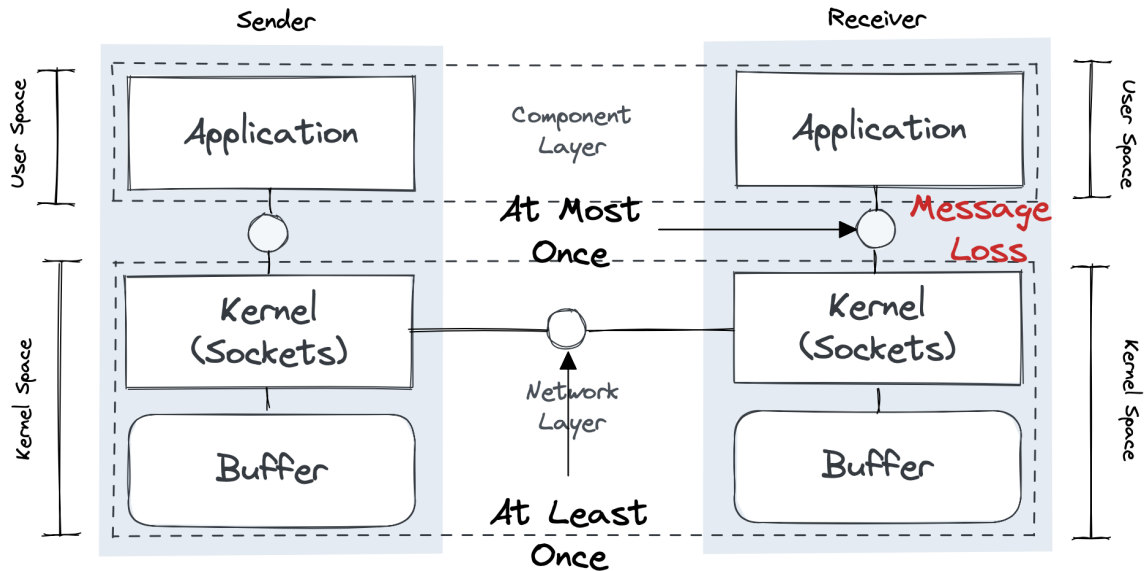


Figure 4. TCP Message Exchange

If we trip after acknowledging receipt but before delivering the message, the message will be lost. In other words, TCP cannot guarantee to prevent message loss.

Silence

After sending a request but before receiving a response, we are in a state of uncertainty that we ultimately need to resolve.

As discussed in Chapter 3 on Failure Detection, a common witness for the occurrence of a failure is a timeout.

When sending a request, the sender sets a time limit for expecting a response. If the sender does not receive a response within the set time limit, that is, a timeout occurs, the sender suspects that either the request or the response was lost or the receiver crashed.

However, since our system model is only partially synchronous, the system provides limited timing guarantees:

- If the system currently operates synchronously, it provides timing guarantees, and a timeout is an accurate witness (true positive).
- If the system currently operates asynchronously, it cannot provide any timing guarantees, and a timeout may not be an accurate witness (true positives or false positives).

In the absence of a response, the sender truly does not know and has no way of knowing whether a side effect occurred or will occur sometime in the future.

Chatter

However, the sender not receiving a response from the receiver is not our only challenge. Another challenge is the receiver receiving duplicate requests from the sender. Our partially synchronous system model does not only allow for message loss failures but also message duplication failures!

The canonical, real-life example of a protocol that allows for both message loss and message duplication in the network is the UDP protocol.

In short, be prepared for anything.

Mitigating Silence & Chatter

So, at this point, we begin to understand that *guaranteeing* exactly once delivery or processing is impossible!

In the absence of a failure or a delay, the sender sends a request and will receive a response. In the presence of a failure or a delay, the sender sends a request but will not receive a response.

The sender has exactly two choices, move on or try again. If the sender moves on, the sender *intends* to deliver and process a message *at-most once*, that is, zero or one time. However, messages may get duplicated in the network. If the sender tries again, the sender *intends* to deliver and process a message *at-least once*, that is, one or more times — However, messages may *continuously* get lost in the network.

However, understanding that *guaranteeing* exactly once processing is impossible, can we *guarantee* exactly once processing *semantics*?

AHA! Moment • Two Failed Attempts

In order to solidify our understanding of the fundamental impossibility of exactly once processing, let's look at two implementations that come up short.

On the receiver side, we will keep track of the requests that have previously been delivered or processed.

Listing 1. illustrates an algorithm that tracks a request before processing the request. If a duplicate gets delivered, the if statement will short the message handler.


```
on message request do

  if m in received then
    return
  end

  received = received + m

  process(m)

  return

end
```

Does this guarantee exactly once processing? No! If the message handler crashes untimely *after tracking* the request but *before processing* the request, when a retry gets delivered, the if statement shorts the message handler. *The message **will not be processed**, amounting to at-most once processing.*

Let's try the other way around. Listing 2. illustrates an algorithm that processes a request before tracking the request. If a duplicate gets delivered, the if statement will short-circuit the message handler.

```
on message request do

  if m in received then
    return response
  end

  process(m)

  received = received + m

  return response

end
```

```
end
```

Does this guarantee exactly once processing? No! If the message handler crashes untimely *after processing* the request but *before tracking* the request, when a retry gets delivered, the if statement will short-circuit the message handler. *The message **will be processed again**, amounting to at-least once processing.*

Exactly Once Processing Semantics

Exactly once processing semantics do not focus on the processing itself but on the processing outcome: Exactly once processing semantics guarantee that the state of the system processing a message once is equivalent to the state of the system processing the message twice or more times.

$$P(m) \equiv P(m) \cdot \dots P(m)$$

Note that we are not talking in terms of *equality* but in terms of *equivalence*: In general, we cannot guarantee that the state of the system processing a message once is equal to the state of the system executing the message twice or more times. The simplest example is logging: A system may log that it processed a message. In case of two or more processings, there are two or more log entries. However, we generally regard the duplication

of log entries as inconsequential and the state of the system is not equal but equivalent.

AHA! Moment • Application Specific

Like correctness guarantees, equivalence is specific to the application. As a software engineer, you have the ability to define equivalence. In other words, what you may consider equivalent, I may consider to be different. Equivalence is in the eye of the beholder.

Guaranteeing Exactly Once Processing Semantics

Exactly once processing semantics are an end-to-end guarantee, both the sender and the receiver play an important role in ensuring exactly once guarantees.

Self evidently, guaranteeing exactly once processing semantics requires guaranteeing at-least once delivery and processing: The sender must not stop submitting requests until it receives a response and the receiver must not short-circuit processing.

However, now we run into the situation that message processing may happen multiple times. How can we guarantee the state of the system is equivalent to processing happening exactly once? The answer is idempotence!

Idempotence

Idempotence is a mathematical term referring to the property of an operation that applying the operation multiple times will yield the same value as applying the operation once.

$$f(f(x)) = f(x)$$

Similarly, in the context of distributed systems, idempotence refers to the property of an operation that executing the operation multiple times will have an equivalent effect as executing the operation once.

If message processing consists of an idempotent operation, at-least once message processing guarantees have exactly once processing semantics guarantees.

However, the bad news is that guaranteeing idempotence can be *extremely* challenging.

Guaranteeing Idempotence

Just like in the case of exactly once processing semantics, idempotence is specific to the application.

And just like in the case of exactly once processing semantics, idempotence is an end-to-end guarantee, both the sender and the receiver play an important role in ensuring idempotence.

Some operations on *some* data structures are inherently idempotent. For example, consider a write-once register: The register starts with an initial value of `NULL`. The first request to set the value “locks in” its value forever. Any subsequent

requests will not alter the value. The operation is inherently idempotent: If a request gets duplicated but the request has already been processed, the register will be in the exact same state.

However, *most* operations on *most* data structures are not inherently idempotent. For example, consider charging a credit card: The operation is inherently not idempotent: If a request gets duplicated but the request has already been processed, the credit card gets charged twice.

A common pattern in implementing idempotence is the use of an idempotence key: The sender generates a uuid and attaches the uuid to the request and every retry of the request. The receiver promises to deduplicate the request and its retries based on the uuid.

A common pattern in implementing receiver-side deduplication is to transactionally process and record the request. For example, an application may use a separate database table to track idempotence keys and only commits if and only if a new entry for the idempotence key can be made without violating a uniqueness constraint alongside the actual side effect.

Case Study • Charging a Credit Card

To illustrate exactly once processing semantics and idempotence, we will use the example of booking an airline ticket. One step in this process involves charging the customer's credit card.

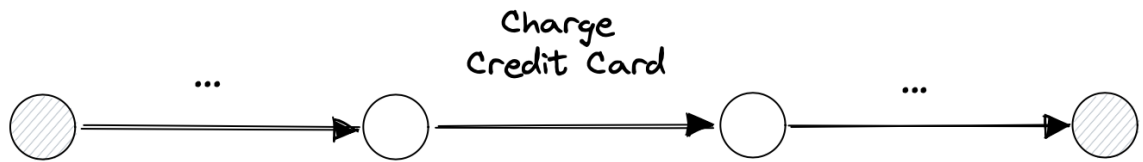


Figure 5. Booking an airline ticket

We want to guarantee that the credit card is charged exactly once per purchase, not zero, two, or more times.

In the absence of a failure or a delay, the sender sends a request and will receive a response. In the presence of a failure or a delay, the sender sends a request but will not receive a response.

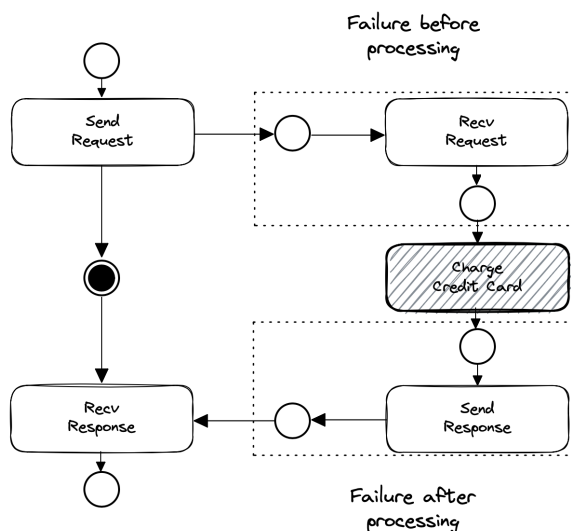


Figure 6. Request-Response Dialog Charging a Credit Card

In the absence of a response, the web app or mobile app may retry a bounded number of times ultimately informing the user if all retries failed to yield a response. In that case, the user may even have to contact support to determine if a ticket was issued — although most likely a confirmation email or a refresh of the My Trips tab of the app will bring some certainty.

How could the app ensure exactly once processing semantics and idempotence? As exactly once processing semantics and idempotence are in the eye of the beholder, there are many possibilities, with different user experiences.

For example, some payment processors, such as Stripe, support idempotence keys in their APIs. From [Stripe's website](#):

Stripe's idempotency works by saving the resulting status code and body of the first request made for any given idempotency key, regardless of whether it succeeded or failed. Subsequent requests with the same key return the same result, including 500 errors.

Assuming the reservation number is itself a unique identifier, the sender may use the reservation number as an idempotence key.

However, other options do exist. The following is a recount of a personal, particularly frustrating experience.

While using an airline's mobile app to book a ticket, the app repeatedly crashed on me upon checkout. So I restarted the app multiple times to resume the checkout process, eventually giving up after four or five times.

Upon checking my credit card activity, to my dismay, there were four or five pending charges. I contacted the airline's support who assured me that within 24h only one pending charge will take effect.

The airline's software engineers considered this to be exactly once processing semantics. In their eyes, the state of the system processing a request exactly once was equivalent to the state of the system processing a request twice or more times. Personally, I disagree. I was certainly in a very different state of mind.

Seeing four or five charges on my credit card activity made me uneasy.

So, as a cautionary tale, when implementing exactly once processing semantics, do consider the resulting experience for your users.

Conclusion

Exactly once = At-least once + Idempotence

In the absence of failure, exactly once message delivery and processing is trivial. In the presence of failure, exactly once message delivery and processing is impossible!

However, we are able to guarantee exactly once processing semantics if we are able to guarantee at-least once delivery and processing and an idempotent processing step.

However, at that point, you are merely given the chance to make things right. Guaranteeing idempotence is one of the most complex aspects of distributed systems and a one size fits all solution does not exist.

Guaranteeing exactly once processing semantics and idempotence will require all your skills and all your wits. Why? Because your mission is to guarantee correctness without being offered any guarantees to build upon. Message reordering, message duplication, message loss, crashes, or simply delays, anything is fair game.

Outlook

In the next chapter, we will explore the concept of transactions. While transactions are an important concept in the context of non-distributed systems, transactions are an equally important concept in the context of distributed systems. Therefore, before we have a look at *distributed* transactions and related concepts like partitioning and replication, we'll have a principled look at transactions themselves.

TRANSACTIONS



In the previous chapter, we explored message delivery and message processing. We focused our attention on guaranteeing exactly-once processing semantics in an environment that allows for message reordering, message duplication, message loss, component crashes, and arbitrary delays.

In this chapter, we will build upon the knowledge gained in Chapters 2, 3, and 4 and explore one of the most popular, most powerful abstractions in software engineering: Transactions.

Objectives:

- Understand the concept of abstractions
- Understand the concept of transactions
- Understand the significance of transactions
- Understand the implementation of transactions

*Transactions turn the world upside down,
transactions provide certainty in an uncertain
world.*

Introduction

Transactions are rooted in the world of database systems not in the world of distributed systems. So why are we covering transactions in this book? Transactions were swiftly and broadly adopted in the context of distributed systems and are recognized as the *benchmark for an exceptional developer experience*.

Transactions allow you to pretend that concurrency or failure do not even exist!

Today, transactions stand as one of the most ubiquitous abstractions in the field of software engineering. Almost every software engineer knows of transactions and their fundamental principles encapsulated by *ACID*, the acronym for Atomicity, Consistency, Isolation, and Durability.

Fun Fact • ACID

According to James "Jim" Gray, Turing Award Winner of 1998 for his seminal contributions to database and transaction processing, his colleague Andreas Reuter coined the acronym ACID because his wife Christiane Reuter "hates sweet things and loves vinegar."

Most authors introduce and explain transactions in terms of ACID guarantees. However, ACID presents the properties of transactions as separate, discrete concepts, that is, from a reductionist point of view. In Chapter 1, we emphasized the value of exploring different mental models and adopting alternative thinking approaches to gain a holistic understanding of a topic. Building upon that notion, we will introduce and explain

transactions from a different perspective, not through the lens of ACID guarantees, but rather through the lens of correctness and completeness guarantees.

Abstractions

Before we discuss the details of transactions, we will spend some time discussing the details of abstractions.

In arguably the most fundamental work in computer science, *Structure and Interpretation of Computer Programs* by Abelson and Sussman, the term abstraction appears on page one, chapter one, headline one.

Abelson and Sussman identify Abstractions as one of the three fundamental building blocks of computer programs: Primitive expressions, means of combinations, and means of abstractions.

1. **Primitive expressions** which represent the simplest entities of the language
2. **Means of combination** by which compound elements are built from simpler elements
3. **Means of abstraction** by which compound elements are named and manipulated as a unit

While this definition accurately captures the nature of abstractions, it does not stress their essence: Abstractions have the power to transform entire domains of discourse, effectively reshaping entire worlds.

In the book *Modern Operating Systems*, Tanenbaum and Bos created a variation of Figure 1 to illustrate the transformative nature of abstractions.



Figure 1. Transformative nature of abstractions

The lower level presents a set of entities, the ugly entities, that the higher level consumes and transforms into a different set of entities, the beautiful entities.

For instance, from the perspective of an operating system, the hardware components such as the CPU, RAM, and disks represent the ugly interface. The operating system takes in these entities and presents a beautiful interface, processes, memory, and files.

However, beauty lies in the eye of the beholder. For example, from the point of view of database systems, the operating system presents its ugly interface, now processes, memory, and files are undesirable. Now the database system consumes these entities and presents its beautiful interface, tables consisting of rows and columns, and transactions.

This example of databases and operating systems also emphasizes the recursive relationships that exist between abstractions. Higher-level abstractions are built upon lower-level abstractions until we reach the fundamental primitives, which represent the simplest entities in the system.

Each time we transition across these boundaries, moving from a database system to an operating system and from an operating system to hardware, we enter an entirely distinct world. This entails encountering different entities, relationships among those entities, and constraints on those relationships. Each transition presents an entirely different semantic.

In his paper “Beyond Distributed Transactions,” Helland uses a variation of Figure 2 to highlight the transformative nature of abstractions.

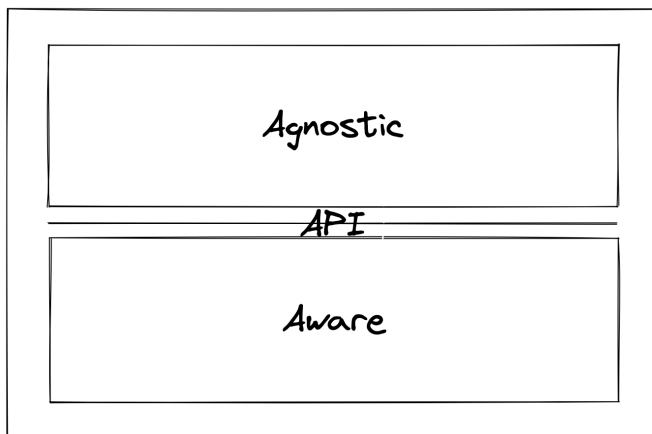


Figure 2. Transformative nature of abstractions

Instead of the terms beautiful and ugly, Helland uses the terms agnostic and aware. More importantly, Helland emphasizes the Application Programming Interface that maps the higher-level, agnostic concepts onto the lower-level, aware concepts. The Application Programming Interface translates between worlds.

Personally, I prefer Figure 3 to highlight higher-level and lower-level abstractions and to reason about their relationships.

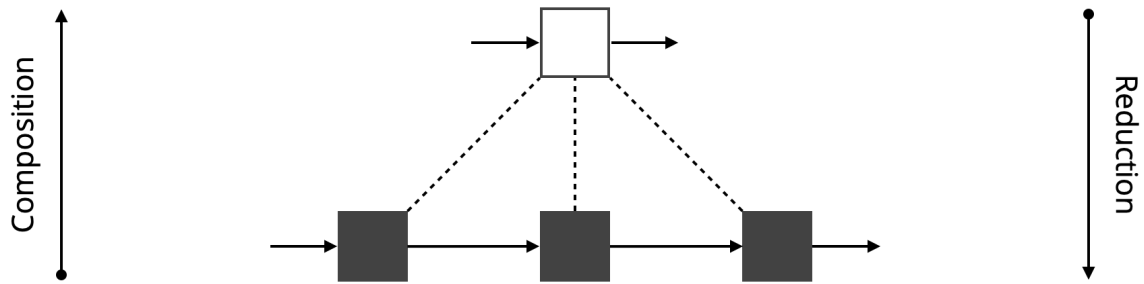


Figure 3. Equivalence between higher and lower level

From a top-down perspective, we observe a reduction: An entity using the abstractions of the higher level is reduced, transformed, or compiled into entities using the abstractions of the lower level.

Conversely, from a bottom-up perspective, we see a composition: Entities using the abstractions of the lower level are composed into an entity using the abstractions of the higher level and the entity on the higher level emerges.

Regardless if you look at this system top-down or bottom-up, there is a notion of **equivalence**: Whether we reason about the system in terms of higher-level abstractions or in terms of lower-level abstractions, we shall come to the same conclusion. Simply said, whether we reason in terms of the higher level or lower level the system shall return the same result.

In the remainder of this chapter, we will explore transactions as abstractions that transform concurrency-agnostic and failure-agnostic definitions into concurrency-aware and failure-aware and failure-tolerant executions. In other words, we will explore transactions as abstractions that enable you to pretend that concurrency and failure were non-existent.

The Magic of Transactions

Web applications are distributed applications. However, when I started developing web applications I did not *realize* that web applications are distributed applications. I didn't connect the dots. How could I have missed that crucial connection? The answer is *transactions*.

The short version: Transactions guarantee correctness and completeness. If you build your application on top of a transactional database system, transactional guarantees extend from the database to your entire system. Transactions obfuscate the fact that you are building a concurrent, partially synchronous distributed system, consisting of components subject to failure and communicating over an unreliable network.

```
app.post('/transfer', async (req, res) => {
  const { source, target, amount } = req.body;

  const query = `
    BEGIN;
    UPDATE accounts SET balance = balance - $1 WHERE id = $2;
    UPDATE accounts SET balance = balance + $1 WHERE id = $3;
    COMMIT;`;

  await db.none(query, [amount, source, target]);

  res.status(200).end();
});
```

Listing 1. illustrates a simplified Money Transfer Web API: The `/handler` endpoint accepts a post request containing three parameters, a source identifier, a target identifier, and an amount. The endpoint transfers money from the source account to the target account, for this example regardless of overdraft.

Do you see what is *not* part of this handler? There are no visible guardrails. First, there is no visible detection or mitigation of concurrency. Second, there is no visible detection or mitigation of failure.

Concurrency

Recall from Chapter 2, a system contains a race condition if the system has multiple *possible* executions, some of which are considered correct and some considered incorrect.

An update to a single account balance is inherently vulnerable to race conditions: The process of updating an account balance consists of a sequence of a read operation followed by a write operation. The results may differ based on the interleaving of these read and write operations for concurrent transfers.

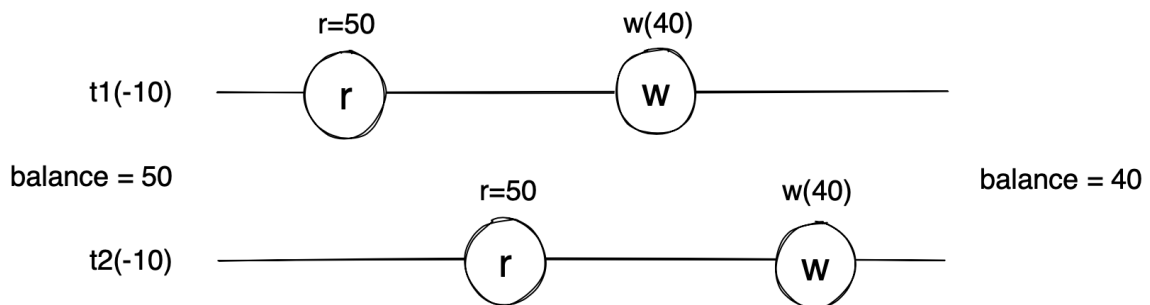


Figure 4. Possible Effects of Concurrency

Consider the concurrent transfers from the same source account depicted in Figure 4. There are two concurrent transfers of 10 units each from the source account that initially holds 50 units. The interleaving of read and write operations from transfer t_1 and transfer t_2 results in a final balance of 40 units, which differs from our expected result of 30 units.

Transactions to the rescue 🛡️

However, transactions *guarantee correctness*. Although the query does not display any visible attempt at detection, mitigation, or prevention of **race conditions**, transactions execute as if **concurrency** does not exist.

Failure

Recall from Chapters 2, 3, and 4, a system is subject to component and network failures.

An update to two account balances is inherently vulnerable to failures. The process of updating two account balances consists of a sequence of two write operations (For this example, we may ignore the reads). If the execution fails after the first write but before the second write the transfer is only partially applied.

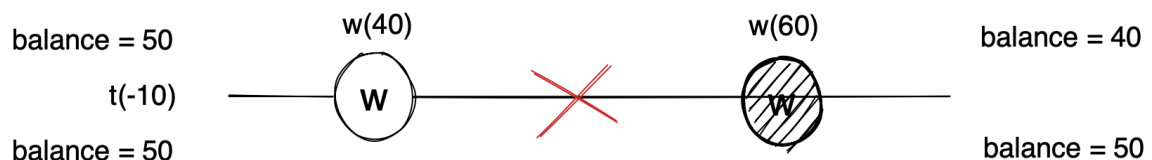


Figure 5. Possible Effects of Failure

Consider the transfers from the source account and target account depicted in Figure 5. There is a transfer of 10 units from the source account to the target account, both initially holding 50 units. If the execution fails after the first write but before the second write, the transfer results in a final balance of 40 units and 50 units, which differs from our expected result of 40 units and 60 units.

Transactions to the rescue 🛡️

However, transactions *guarantee* **completeness**. Although the query does not display any visible attempt at detection, mitigation, or prevention of **failures**, transactions execute as if **failure** does not exist.

In summary, although there are no visible concurrency or failure detection or mitigation steps, transactions guarantee correctness and completeness even in the presence of concurrency and failure: A concurrency and failure-agnostic definition guarantees a concurrency and failure-aware and tolerant execution.

A remarkable result and a delightful developer experience.

The Model of Transactions

This section heavily relies on the 1980 research report by Jim Gray titled [A Transaction Model](#). Keep in mind that this is not an in-depth exploration of database systems and we won't explore nuances such as different isolation levels or details such as performance optimizations.

A transaction is a sequence of operations on a set of objects that ends with exactly one commit or exactly one abort. A transaction guarantees both correctness and completeness.

These guarantees of correctness and completeness can be attributed to the combination of four fundamental principles famously referred to as the ACID guarantees.

ACID Guarantees

ACID, which stands for Atomicity, Consistency, Isolation, and Durability, characterizes transactions as follows:

- **Atomicity** guarantees that a transaction executes observably equivalent to either completely or not at all.
- **Consistency** guarantees that a transaction transitions the database from one consistent state to another consistent state.
- **Isolation** guarantees that a transaction executes as if it is the only transaction executing, preventing interference.
- **Durability** guarantees that once a transaction commits, its effects are permanent, preventing “going back on its word”.

AHA! Moment • Consistency

Consistency sets itself apart from atomicity, isolation, and durability as it is an application-level guarantee, whereas the latter three are platform-level guarantees.

To illustrate this distinction, let's consider two SQL snippets that define the structure of an accounts table.

In the first snippet, the balance is defined as an integer without any additional constraints. As a result, the balance can hold both positive and negative integer values.

In contrast, the second snippet defines the balance as an integer with an additional constraint in the form of a `CHECK` clause. This constraint ensures that the balance can only hold positive integer values.

```
1. Application that does allow overdraft
```

```
CREATE TABLE accounts (  
    ...
```

```
    balance INT
    ...
);

2. Application that does not allow overdraft

CREATE TABLE accounts(
    ...
    balance INT CHECK(balance >= 0)
    ...
);
```

Consistency, being an application-level guarantee, can differ based on the requirements of the application. Developers possess the flexibility to establish and enforce consistency rules that aligns with their specific needs.

In the first case, the database system will commit a transaction if the balance of the account falls below zero. In the second case, the database system will abort a transaction if the balance of the account falls below zero.

In contrast, atomicity, isolation, and durability are platform-level guarantees that are not affected by application-level semantics.

Note • Isolation

Originally, isolation guaranteed that a transaction executes as if no other transactions were executing, a concept referred to as serializability (explained further below). However, modern databases now provide many isolation levels that extend beyond the scope of this book.

Formal Discussion

In general, we think of a database as a collection of tables with tables consisting of rows and columns. However, for our purposes, let's adopt a more abstract perspective. We will think of a database as a collection of objects, where each object is represented by a name-value pair.

Defintion

$P = [<A1, N1>, <A2, N2>, <A3, N2>]$

----- ↙

$t = [<a1, o1>, <a2, o2>, <a3, o2>]$

Execution

Figure 6. Definition & Execution

A program is a sequence of operations performed on a set of names. When executed, a program results in a transaction. A transaction, in turn, is a sequence of actions executed on a set of objects, concluding with either exactly one commit or exactly one abort. Essentially, a program is the definition of a transaction and a transaction is the execution of a program.

Design Time / Definition	Runtime / Execution
Program P	Transaction t
Operation A	Action a
Name N	Object o

We model a transaction as a trace, which consists of a sequence of triples in the form of $\langle t_x, a_i, o_i \rangle$, where:

- t_x represents the transaction
- a_i represents the operation
- o_i represents the object modified by the operation

The trace of a transaction t_x can be expressed as $t_x = [\langle t_x, a_i, o_i \rangle \mid i = 1 \dots n]$. The inclusion of the transaction t_x in the triple allows us to analyze the interleaving of multiple transactions.

The database may interleave the execution of two or more transactions. This execution of a set of transactions is referred to as a history. A history is denoted by the sequence:

```
history = [⟨tj, ai, oi⟩ | j = 1 .. m, i = 1 .. n]
```

The database system translates the failure-agnostic definition into a failure-aware and failure-tolerant execution to guarantee correctness and completeness.



Correctness

This section explores how a database systems ensures correctness, that is, how a database system translates a concurrency-agnostic definition into a concurrency-aware and concurrency-tolerant execution. In other words, the detection and mitigation of concurrency anomalies is handled solely by the database system without burdening the developer.

A transaction transitions the database from a consistent state to a consistent state. We can express that idea as a Hoare triple that states the pre-condition, transaction, and post-condition.

```
{ C } tx { C }
```

However, it's important to note that consistency may be temporarily violated during the execution of a transaction. For instance, in the case of a fund transfer transaction between

source and target accounts, the temporary inconsistency arises when the money is debited from the source account before being credited to the target account.

In a non-concurrent setting, each transaction begins with a consistent state and produces a consistent state. However, when transactions are executed concurrently, one transaction may observe inconsistencies introduced by another transaction.

This observation leads to a straightforward conclusion: If transaction t_1 transitions the database from a consistent state to a consistent state, and transaction t_2 also transitions the database from a consistent state to a consistent state, then the sequential composition of t_1 and t_2 will also ensure the transition from a consistent state to a consistent state.

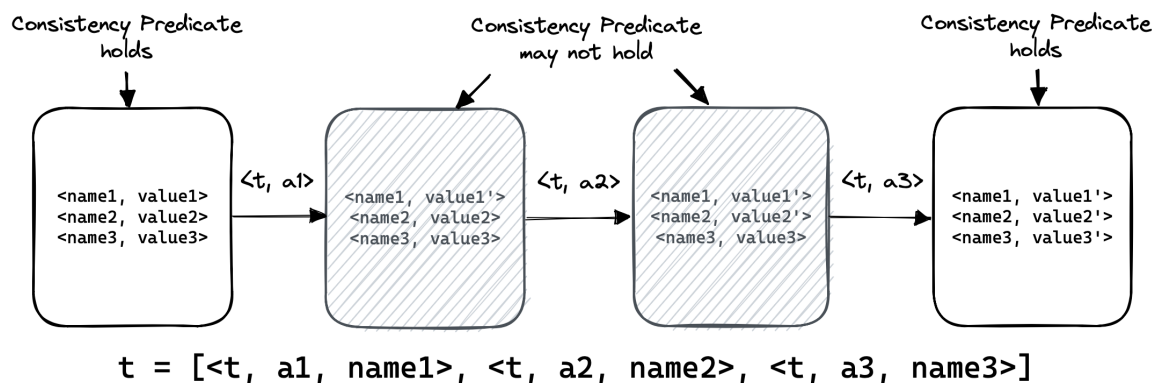


Figure 8. Temporary Consistency Violation

In other words: a history without concurrency, known as a serial history, does not exhibit concurrency anomalies.

$$\{ C \} t_1 \{ C \} \wedge \{ C \} t_2 \{ C \} \Rightarrow \{ C \} t_1; t_2 \{ C \}$$

As a result, two concurrent transactions t_1 and t_2 execute correctly if the combined effects of their concurrent execution are equivalent to the effects of executing t_1 before t_2 or t_2 before t_1 .

```
{ C } t1 || t2 { C } => { C } t1; t2 { C } v { C } t2; t1 { C }
```

This guarantee of correctness is referred to as serializability, where concurrency is allowed only if it does not introduce inconsistencies in the database.

Serializability

Serializability is a **consistency model**. A consistency model is a predicate on execution histories and groups execution histories into good, legal, or valid histories and bad, illegal, or invalid histories.

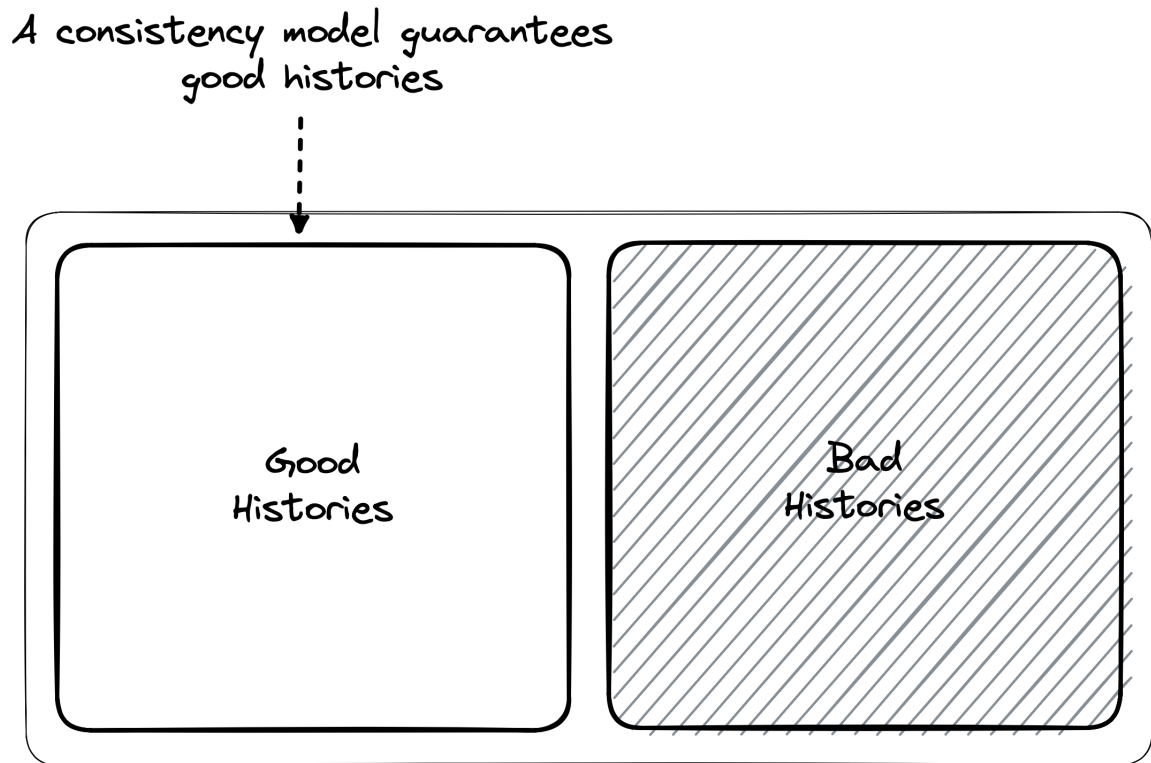


Figure 9. Good & Bad Histories

Consistency models represent correctness guarantees, and as a software engineer, you have the authority to define the consistency model for your system, as discussed in Chapter 1.

Introduced by Edgar Codd, serializability acts as a consistency model for the concurrent execution of a set of transactions. Serializability is based on the concept of equivalence of a concurrent execution to a sequential execution.

When considering the concurrent execution of two transactions t_1 and t_2 , the execution is serializable if the tuple of (transaction result, database state) is equivalent to either

- the sequential execution of $t_1 \cdot t_2$ or
- the sequential execution of $t_2 \cdot t_1$

In Figure 10, if the interleaved execution of actions `a • c • b • d` is equivalent to

- `(a • b) • (c • d)` or
- `(c • d) • (a • b)`

then the history is serializable.

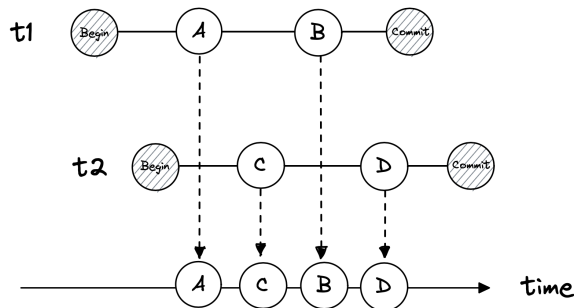


Figure 10. Serializability

Note that serializability does not necessitate equivalence to a specific sequential execution, but rather to *some* sequential execution.

AHA! Moment • Implementing Serializability

Implementing serializability is trivial. Implementing serializability *efficiently* is hard. Since this is not a book about databases, we'll just outline the general idea.

To implement serializability we could guard access to all objects with one lock. Only the transaction holding the lock is allowed to proceed. Here, we are forcing a sequential schedule. We can now start using more fine granular locks to guard the access to subsets of all objects. Here, we are allowing a concurrent schedule that still exhibits sequential semantics.

Our goal must be to refine our implementation to allow the highest degree of concurrency while still guaranteeing sequential semantics.

Completeness

This section explores how a database systems ensures completeness, that is, how a database system translates a failure-agnostic definition into a failure-aware and failure-tolerant execution. In other words, the detection and mitigation of failure is handled solely by the database system without burdening the developer.

Database systems employ various strategies for failure detection and mitigation. As discussed in Chapter 3, there are broadly two techniques: backward recovery and forward recovery. In the context of database systems, backward recovery is often referred to as Undo, while forward recovery is referred to as Redo. While actual databases often combine both Undo and Redo approaches, we will focus solely on Undo.

The concept behind Undo is straightforward: every operation is accompanied by an Undo operation. The database manages a data structure called the Transaction Undo Log, represented by an object `ou`. Before executing an operation on an object, the system records the corresponding Undo operation in the Transaction Undo Log.

Thus, instead of directly executing an operation `<t, ai, oi>`, the system performs two operations: `<t, w, ou + <-ai, oi>>` (representing the recording of the Undo operation) and `<t, ai, oi>` (representing the original operation).

Now there are two valid traces for a transaction:

- **Commit trace** is the execution that unfolds in the absence of failure, executing all regular operations.

```

$$t_x = [\langle t_x, a_i, o_i \rangle \mid i = 1..n]$$

```

- **Abort trace** is the execution that unfolds in the presence of failure, executing some regular operations and their undo operations, effectively restoring the system to its state prior to the transaction.

```

$$t_x = [\langle t_x, a_i, o_i \rangle \mid i = 1..k] [\langle t_x, \neg a_i, o_i \rangle \mid i = k..1]$$

```

There are two scenarios to consider: application-level abort and platform-level abort. Application-level abort is straightforward. However, platform-level abort introduces certain challenges that demand careful deliberation and consideration.

Application-Level Abort

An application-level abort occurs when a transaction is explicitly or implicitly aborted. An explicit abort is triggered by issuing an abort command explicitly, indicating the termination of the transaction. An implicit abort is triggered by conditions, such as a consistency violation, that result in the automatic termination of the transaction.

In the case of an application-level abort, the database system simply performs the undo operations associated with the transaction.

Platform-Level Abort

A platform-level abort, on the other hand, occurs when a crash failure and subsequent restart occur at an arbitrary point during the execution of a transaction.

In this case, when the database system restarts, the database system has to perform recovery, that is, the database system has to examine its Transaction Undo Log for any transaction that has not been committed and execute its undo operations.

When a platform-level abort occurs and the database system restarts, the database system has to perform recovery. The database system examines its Transaction Undo Log to identify any transactions that were not yet committed at the time of the crash failure. Then the database system performs the undo operations of those transactions.

However, there is one problem: recording an undo operation and executing the operation are two separate steps. If the system crashes and restarts after the undo operation is recorded but before the operation is performed then the undo operation will be applied to the old value, and not the new value of the object. Additionally, if the system crashes and restarts during undo operations, undo operations will be performed multiple times.

In order to solve this problem, undo operations must not just be idempotent but must be restartable (noop + idempotent):

- **Noop (No Operation)** This means that applying an operation and its corresponding undo operation on an object should be equivalent to only applying the undo operation on the object.

$$\langle t_x, a_i, o_i \rangle \langle t_x, \neg a_i, o_i \rangle = \langle t_x, \neg a_i, o_i \rangle$$

- **Idempotent** This means that the undo that applying an operation and its undo operation multiple times should have the same effect as applying them once.

$$\langle t_x, a_i, o_i \rangle \langle t_x, \neg a_i, o_i \rangle = \langle t_x, a_i, o_i \rangle \langle t_x, \neg a_i, o_i \rangle \dots \langle t_x, \neg a_i, o_i \rangle$$

Conclusion

Transactions are an abstraction that allows an application to pretend that concurrency and failure do not exist. Application developer define transactions in a failure-agnostic manner while the database system guarantees failure-aware and failure-tolerant execution.

At their core, transactions make a compelling promise. They guarantee correctness and completeness, ensuring that in the worst case, a transaction executes equivalently to not executing at all. This promise alleviates concerns about potential consistency anomalies or partial execution, providing a sense of certainty.

Transactions are commonly introduced and discussed in the context of ACID guarantees: Atomicity, Consistency, Isolation, and Durability. While this characterization accurately captures important aspects of transactions, solely focusing on individual guarantees can lead to a limited or reductionist perspective.

By embracing a holistic viewpoint, we recognize that transactions create an encompassing world where the challenges of concurrency and failure become virtually non-existent.

The magic of transactions.

Outlook

In the next chapter, we will explore distributed transactions and most importantly, the Two-Phase Commit Protocol, one of the most famous, most well-known protocols in distributed systems.

DISTRIBUTED TRANSACTIONS & TWO- PHASE COMMIT PROTOCOL



In the previous chapter, we explored one of the most popular, most powerful abstractions in software engineering: Transactions.

In this chapter, we will explore *distributed* transactions and the Two-Phase Commit (2PC) protocol.

Objectives:

- Understand the concept of distributed transactions
- Understand the 2PC in the absence of failure
- Understand the 2PC in the presence of failure
- Understand possible improvements of 2PC

From Transactions to Distributed Transactions

In the previous chapter, we explored the concept of transactions, one of the most popular, most powerful abstractions in software engineering.

Transactions are an abstraction that allows an application developer to pretend that concurrency and failure do not exist. Application developers can define transactions in a failure-agnostic manner, while the database system guarantees failure-aware and failure-tolerant execution.

So far, we have only explored transactions on a *single* database system. Now, we will expand our exploration to include transactions that span *multiple* database systems, also known as distributed transactions.

Transactions are commonly introduced and discussed in the context of ACID guarantees: Atomicity, Consistency, Isolation, and Durability. However, we will focus our discussion of distributed transactions on atomicity and will not explicitly discuss consistency, isolation, and durability.

A Note on Terminology

In the context of distributed transactions, the participants in the distributed transaction are often referred to as *resource managers*. The term resource manager encompasses not only database systems, but also other systems such as message queuing systems that can participate in a distributed transaction. From here on out we will use the

terms “resource manager” and “database system” interchangeably.

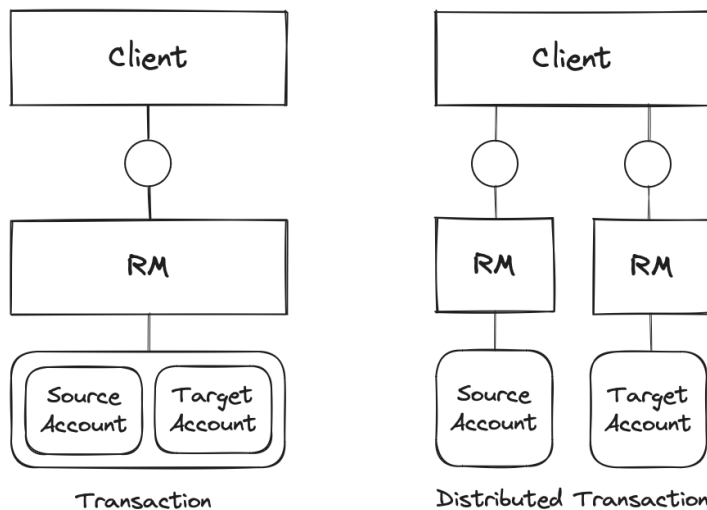


Figure 1. From a single RM to multiple RMs

Let's revisit the example of a money transfer discussed in Chapter 5. Listing 1. illustrates a transaction that executes on a single resource manager. It transfers money from the source account to the target account, regardless of overdraft.

```
BEGIN
  UPDATE accounts SET balance = balance - $1 WHERE id = $2;
  UPDATE accounts SET balance = balance + $1 WHERE id = $3;
COMMIT
```

However, if the source and target account are hosted on two separate resource managers, how do we guarantee completeness?

Listing 2. Illustrates two transactions that execute on two resource managers, RM1 And RM2, to transfer money from the source account hosted on RM1 to the target account hosted on RM2.

```
RM1:
BEGIN
  UPDATE accounts SET balance = balance - $1 WHERE id = $2
COMMIT

RM2:
BEGIN
  UPDATE accounts SET balance = balance + $1 WHERE id = $3
COMMIT
```

How can we prevent disagreement and guarantee that both transactions either commit or abort? In other words, how can we guarantee that the composition of two (sub-)transactions is a transaction in itself?

The answer is *atomic commit protocols*.

Atomic Commitment

The remainder of this chapter explores atomic commitment. Recall that a transaction is a sequence of operations on a set of objects that ends with exactly one commit or exactly one abort.

A transaction is atomic, meaning that if it ends in a commit, it executes observably equivalent to exactly once. Conversely, if it ends in an abort, it executes observably equivalent to not at all.

From a single RM to multiple RMs

Recall that a distributed system is a collection of concurrent, communicating components that communicate by sending and receiving messages over a network. Each component has exclusive access to its own local state, which is not accessible by any other components.

We may think about the distributed system as if it proceeds in discrete steps, with either a component or the network taking a step.

On a single resource manager, atomic commit is guaranteed via writes to the local state. On multiple resource managers, atomic commit is guaranteed via atomic commit protocols.

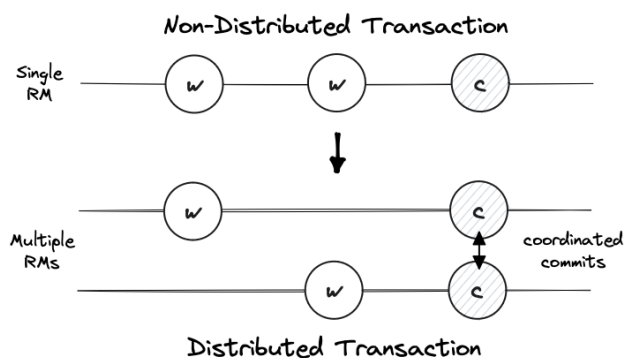


Figure 2. How do we **coordinate & guarantee** multiple commits?

Transaction on a single RM

For non-distributed transactions that execute on a single resource manager, atomicity is guaranteed via *one atomic* write to its local state: When the resource manager commits or aborts a transaction, the resource manager takes a single step, that is, the resource manager writes a commit or abort entry to its local state.

(If the resource manager fails before writing a commit or abort entry, on recovery, the resource manager proceeds as if there exists an abort entry)

Transaction on a multiple RMs

For distributed transactions that execute on multiple resource managers, atomicity is guaranteed via an atomic commit protocol. Atomic commit ensures that a distributed transaction either commits or aborts, that is, *all* sub transactions unanimously commit or abort.

The correctness of an atomic commit protocol is specified via its safety and liveness guarantees:

- The safety guarantee asserts that no two participants in a transaction arrive at a conflicting decision,

```
S:  $\nexists$   $rm_1, rm_2$ :  $state[rm_1] = \text{commit} \ \& \ state[rm_2] = \text{abort}$ 
```

In words: There is no pair of resource managers such that one is in the commit state and the other is in the abort state.

- The liveness guarantee asserts that every participant will eventually arrive at a decision.

```
L:  $\forall$  rm:  $\langle \rangle [] \text{ state[rm]} = \text{commit} \mid \langle \rangle [] \text{ state[rm]} = \text{abort}$ 
```

In words: For all resource manager, the state of a resource manager is eventually always committed eventually always aborted.

Blocking and Non-Blocking

There are two types of atomic commit protocols: blocking and non-blocking. Blocking protocols guarantee safety, but not liveness, in the presence of participant failure. Non-blocking protocols guarantee both safety and liveness in the presence of *one* participant fails.

In other words, for a commit protocol to be non-blocking, a single participant's failure does not prevent other participants from deciding whether the transaction is committed or aborted.

FORMAL DISCUSSION



Reasoning about atomic commit protocols, including their safety and liveness guarantees, is a challenging task. This is especially true in the presence of failures, and requires an accurate and concise mental model of distributed transactions.

A distributed transaction, also referred to as global transaction, consists of two or more non-distributed transactions, also referred to local transactions.

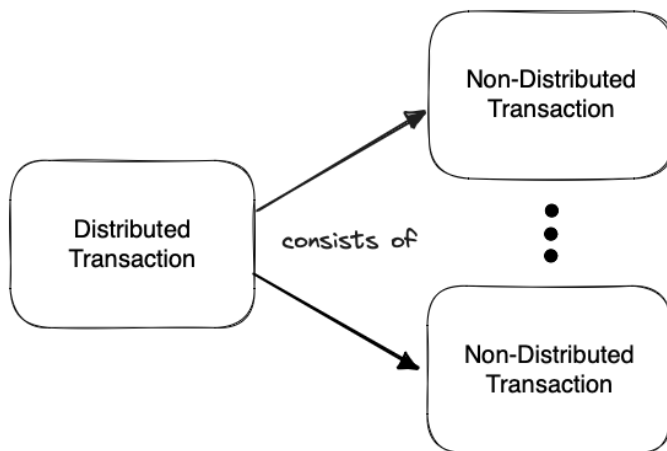


Figure 3. A distributed transaction consists of two or more non-distributed transactions

We can think of a non-distributed transaction as being in one of four states: working, prepared, committed, or aborted.

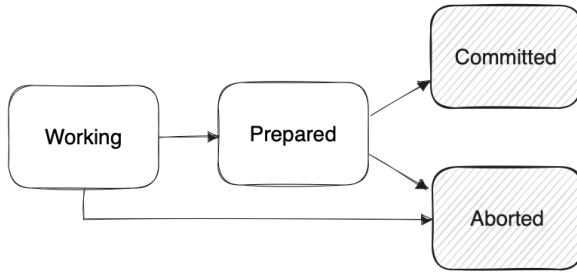


Figure 4. State Machine of non-distributed transactions

- **Working** The transaction is executing operations. From here, it may decide to transition to aborted (for example, if a consistency constraint is violated) or to transition to prepared and wait for the request to commit or abort.
- **Prepared** The transaction is not executing operations but did not transition to committed or aborted yet. From here, the transaction may transition to committed on receiving a commit request or transition to aborted on receiving an abort request.
- **Committed** or **Aborted** The transaction has reached its final, irreversible state either by being committed or aborted.

The state of a distributed transaction consists of a state vector that contains the states of non-distributed transactions, as well as the outstanding messages in the network.



Figure 5. Global transaction (Outstanding messages not illustrated)

TWO PHASE COMMIT PROTOCOL



The Two-Phase Commit (2PC) protocol is the most well-known and the most well-studied atomic commit protocol.

The 2PC protocol guarantees safety and liveness in the absence of participant failure. However, in the presence of even one (specific) participant failure, the protocol only guarantees safety but not necessarily liveness. Therefore, the protocol is considered a blocking atomic commit protocol.

In the absence of failure

The 2PC protocol partitions the system into exactly one transaction coordinator and two or more resource managers. The protocol executes in two phases: the Prepare Phase and the Commit Phase.

A client initiates a transaction with each participating resource manager and executes standard read and write operations. However, instead of directly asking the resource managers to commit or abort the transactions, the client asks the transaction

coordinator to commit or abort the transactions, marking the start of the 2PC protocol execution.

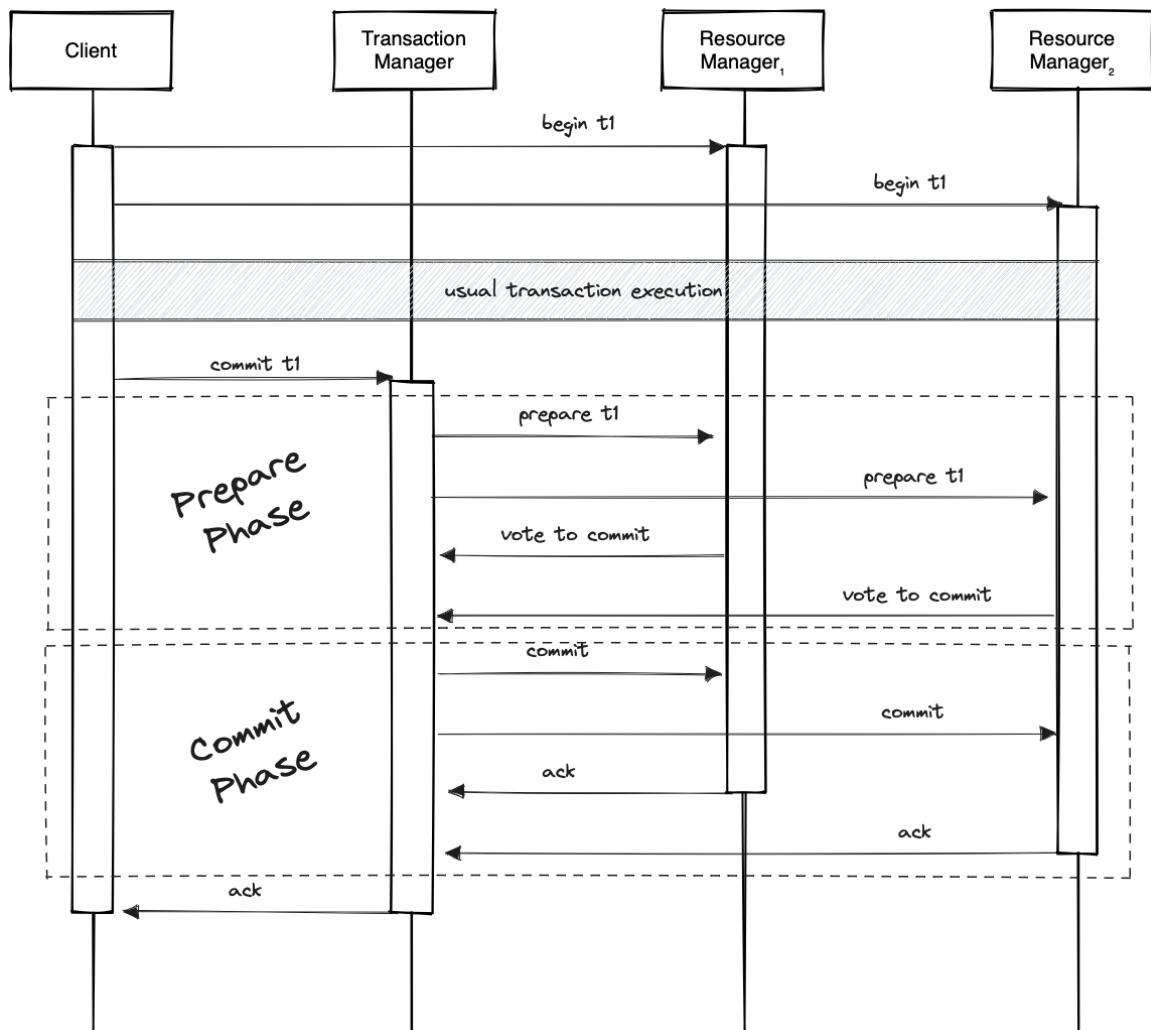


Figure 6. Two Phase Commit protocol

During its first phase, the transaction coordinator asks all participating resource managers if they vote to commit or if they will abort their local transactions. In its second phase, the transaction coordinator instructs all participating resource managers to commit or to abort.

Phase 1 • Prepare Phase

1. The transaction coordinator TC persistently records a $\langle \text{Prepare}, \text{Timeout} \rangle$ entry to its log and sends a $\langle \text{Prepare} \rangle$ request to all participating resource managers RM_i
2. Each participating resource managers RM_i receives the $\langle \text{Prepare} \rangle$ request and decides whether to commit or abort its local transaction t_i :
 - If RM_i decides to commit, RM_i persistently records a $\langle \text{Vote-to-Commit} \rangle$ entry to its log and sends $\langle \text{Vote-to-Commit} \rangle$ to the transaction coordinator TC.
 - If RM_i decides to abort, RM_i persistently records an $\langle \text{Abort} \rangle$ entry to its log, sends $\langle \text{Abort} \rangle$ to the transaction coordinator TC, and aborts the transaction t_i .

AHA! Moment • Vote-to-Commit vs Abort

To this day, I still occasionally find myself perplexed by the asymmetry inherent in the Preparation Phase of the protocol. The Preparation Phase does **not** consist of a $\langle \text{Vote-to-Commit} \rangle$ and $\langle \text{Vote-to-Abort} \rangle$ pair nor a $\langle \text{Commit} \rangle$ and $\langle \text{Abort} \rangle$ pair. Instead, the protocol consists of a $\langle \text{Vote-to-Commit} \rangle$ and $\langle \text{Abort} \rangle$ pair. In other words, during this phase, a resource manager cannot unilaterally commit, but they can unilaterally abort.

Phase 2 • Commit Phase

1. If the transaction coordinator TC receives a $\langle \text{Vote-to-Commit} \rangle$ response from all participating resource managers RM_i , TC persistently records a $\langle \text{Commit} \rangle$ entry to its log and sends a $\langle \text{Commit} \rangle$ request to all participating resource managers RM_i .
2. If the transaction coordinator TC receives an $\langle \text{Abort} \rangle$ response from at least one participating resource manager RM_i or a timeout, TC persistently records an $\langle \text{Abort} \rangle$ entry to its log and sends an $\langle \text{Abort} \rangle$ request to all participating resource managers RM_i .

3. Each participating resource managers RM_i receives the $\langle \text{Commit} \rangle$ or $\langle \text{Abort} \rangle$ request, persistently records a $\langle \text{Commit} \rangle$ or $\langle \text{Abort} \rangle$ entry to its log, and commits or aborts its local transaction t_i .

In the presence of failure

In the absence of failure, the 2 Phase Commit protocol guarantees both safety and liveness. However, in the presence of failure, the 2 Phase Commit protocol only guarantees safety but does not *guarantee* liveness.

As defined in Chapter 2, we assume a partially synchronous system model consisting of unreliable components communicating over an unreliable network that acts synchronously most of the time and asynchronously sometimes.

Resource Manager Failure

The 2 Phase Commit protocol guarantees both safety and liveness even in case one or more *resource managers* fail. In other words, a failure of one or more resource managers does not block the protocol.

If a resource manager RM_i fails before persistently recording $\langle \text{Vote-to-Commit} \rangle$ or $\langle \text{Abort} \rangle$ to its log, on recovery, the RM_i persistently records an $\langle \text{Abort} \rangle$ and sends an $\langle \text{Abort} \rangle$ to the Transaction Coordinator TC.

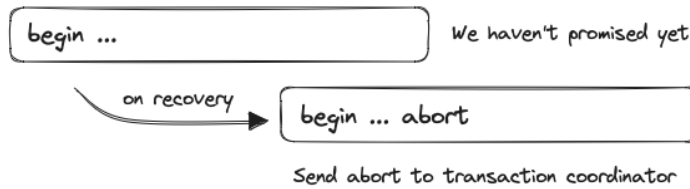


Figure 7.1.

If a resource manager RM_i fails after persistently recording (Vote-to-Commit) to its log, on recovery, the RM_i must inquiry with the TC whether to commit or abort.

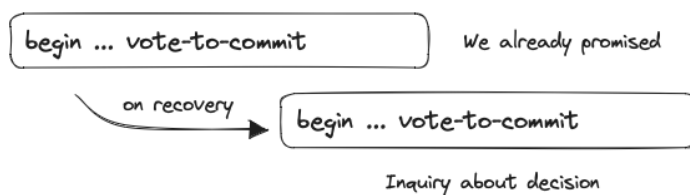


Figure 7.2.

If a resource manager RM_i fails after persistently recording (Commit) to its log, on recovery, the RM_i must perform a REDO.

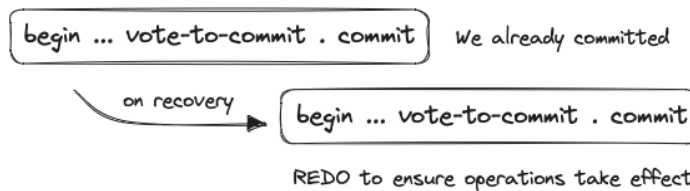


Figure 7.3.

If a resource manager RM_i fails after persistently recording (Abort) to its log, on recovery, the RM_i must perform an UNDO.

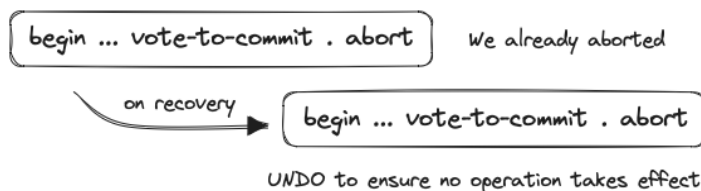


Figure 7.4.

Transaction Coordinator

The 2 Phase Commit protocol guarantees safety but does not *guarantee* liveness in case the *transaction coordinator* fails. In other words, a failure of the transaction coordinator may block the protocol.

If the transaction coordinator fails after a participating resource managers RM_i persistently recorded (Vote-to-Commit) the resource manager RM_i is stuck! In other words, if the transaction coordinator fails after a participating resource manager persistently recorded (Vote-to-Commit), that resource manager is blocked until the transaction coordinator is repaired.

Example

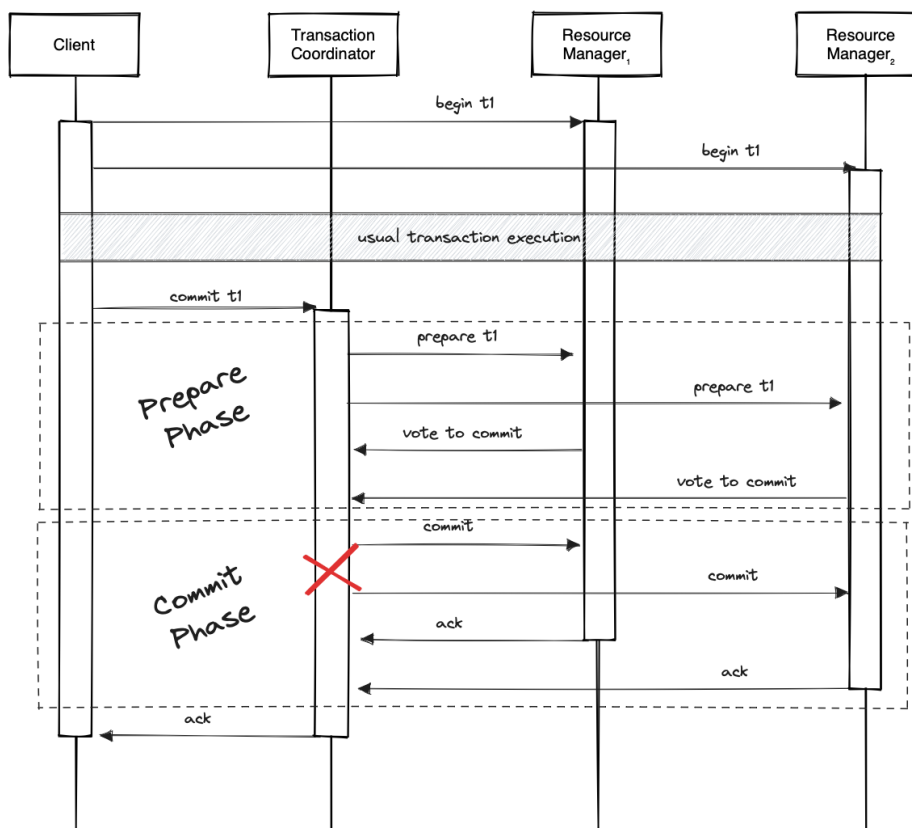


Figure 8. Failure of transaction coordinator after the first commit

Figure 8 illustrates a scenario where all participating resource managers voted to commit in the Prepare Phase, but the transaction coordinator fails in the Commit Phase. As a result, resource manager RM₂ can neither commit nor abort the transaction because it does not know if any other resource managers have already committed or aborted.

Improvement

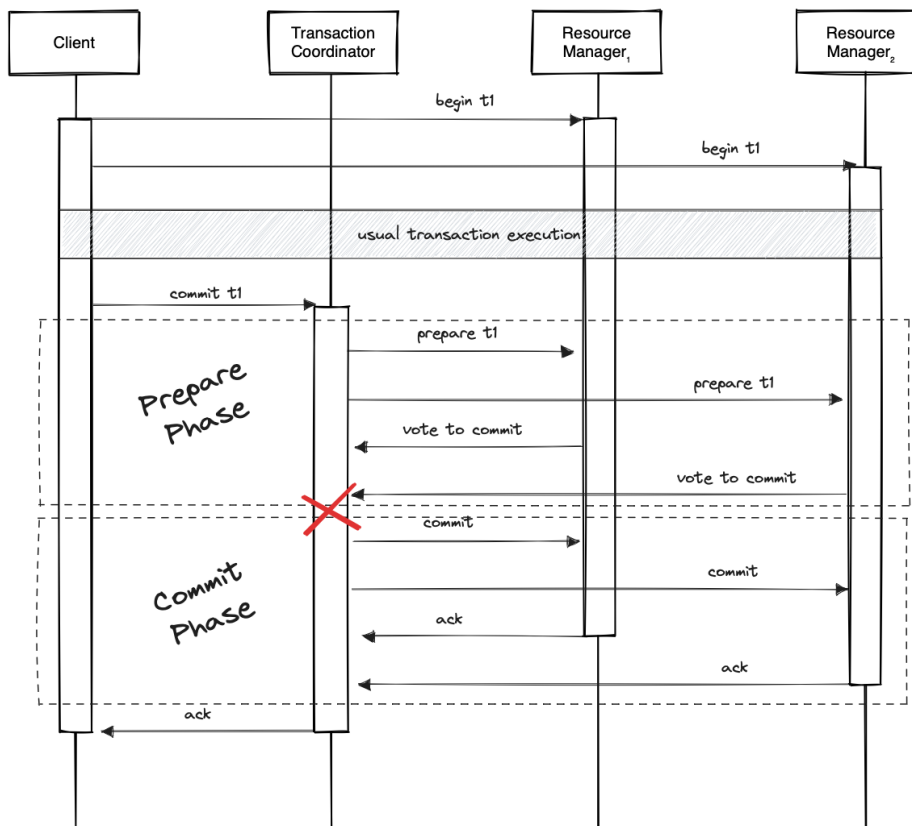


Figure 9. Failure of transaction coordinator before the first commit

There are many variations of the 2 Phase Commit protocol with the goal of improving its liveness guarantees. These variants have *fewer* blocking executions, but they still have *some* blocking executions.

Let's revisit the scenario where the transaction coordinator fails: If the transaction coordinator fails after a participating resource manager RM_i persistently recorded $\langle \text{Vote-to-Commit} \rangle$ the resource manager RM_i is stuck.

One way to improve the protocol is to allow the resource manager RM_i to communicate with other resource managers RM_{ii} : If RM_{ii} already committed or aborted, RM_i may safely commit or abort as well.

However, what happens if all participating resource managers vote to commit but none have yet received a $\langle \text{Commit} \rangle$ or $\langle \text{Abort} \rangle$ request? One might think, "No problem, in that case we can simply timeout and abort." Yet, that approach may lead to a safety violation. Since clocks are not perfectly synchronized, if the transaction coordinator recovers and sends a $\langle \text{Commit} \rangle$, some resource managers may believe they received the $\langle \text{Commit} \rangle$ within their timeout window, while others may believe they received the $\langle \text{Commit} \rangle$ outside their time window and have already aborted.

This is a great reminder how subtle and intricate distributed protocols can be!

Conclusion

For transactions that execute on a *single* resource manager, atomicity is guaranteed by performing *one* write to its local state. For transactions that execute on multiple resource managers, atomicity must be guaranteed by an atomic commit protocol. Atomic commit ensures that a distributed transaction

either commits or aborts, that is, *all* sub transactions unanimously commit or abort.

We differentiate between blocking and non-blocking atomic commit protocols. Blocking protocols guarantee safety but not liveness whereas non-blocking protocols guarantee both safety and liveness.

Outlook

In the following chapters, we will explore some of the most advanced topics in distributed systems, partitioning and replication

PARTITIONING



In Chapter 1, we explored the question of why we need distributed systems. We came to the conclusion that distributed systems are required to guarantee scalability and reliability.

In the next chapters we will explore two essential techniques in guaranteeing the scalability and reliability of a distributed system: partitioning and replication. Partitioning seeks to overcome scalability limitations of a single component while replication seeks to overcome reliability limitations of a single component.

First up: Partitioning

Objectives:

- Understand static and dynamic partitioning
- Understand vertical and horizontal partitioning
- Understand item-based lookup vs directory-based lookup
- Understand common strategies

Encyclopedias & Volumes

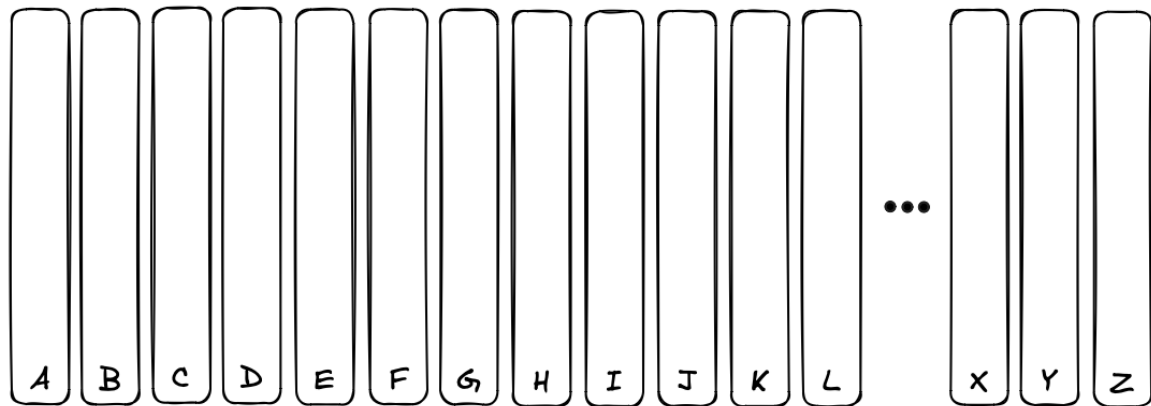


Figure 1. Encyclopedias & Volumes

When I was a teenager, one of my most cherished possessions was a small encyclopedia. An encyclopedia is a compilation of entries that are sorted alphabetically. However, my encyclopedia was too extensive to fit into a single book. Instead, my encyclopedia was divided into multiple volumes.

The encyclopedia represents a *logical object* while the volumes represent *physical objects*. *Logically*, the encyclopedia contains all entries, but *physically* the volumes contain disjoint subsets of entries.

The authors could have placed any item in a randomly selected volume. However, doing so would have made the encyclopedia completely useless. If I needed to find an item, I would be forced to scan through the entire encyclopedia to locate the item or come to the realization that the item doesn't exist.

Therefore, the authors, the producer, of the encyclopedia and I, the consumer, agreed on a straightforward system. Each volume is assigned a letter of the alphabet and contains entries starting with that letter, sorted alphabetically. If the producer wants to add or *place* an entry, they will place it in the corresponding volume. Likewise, if I, the consumer, want to find or *fetch* an

entry, I will search for and fetch the entry from the corresponding volume.

While the practice of dividing an encyclopedia into multiple volumes alphabetically is a tried and true approach, this example highlights various challenges that may arise.

- **Uneven distribution** Not all letters in the English language have an equal number of corresponding entries. For example, the letter 'E' may have a bulky volume of entries, while the letter 'X' may have a slim volume.
- **Uneven demand** For this reason, we may end up reaching for the volume with the letter 'E' more often than the one with the letter 'X'.
- **Cross-references** Some entries may refer to other entries in order to provide a comprehensive picture. For example, a detailed entry about "Partitioning," found under "P," may reference "Replication," found under "R," necessitating access to two volumes.

Just as an encyclopedia may be too extensive to fit in a single book, a software system may be too extensive to fit in a single component, called a node in this context. In such cases, the system can be divided into disjoint subsets or partitions.

Partitioning a software system can present challenges similar to partitioning an encyclopedia, such as uneven distribution, uneven demand, or cross-referencing between partitions. However, optimizing your partition strategy depends entirely on your requirements.

The remainder of this chapter explores partitioning in greater depth to provide a strong foundation to building scalable distributed systems.

Thinking in Partitions

The term *partitioning* refers to representing a single *logical* object by multiple, disjoint *physical* objects.

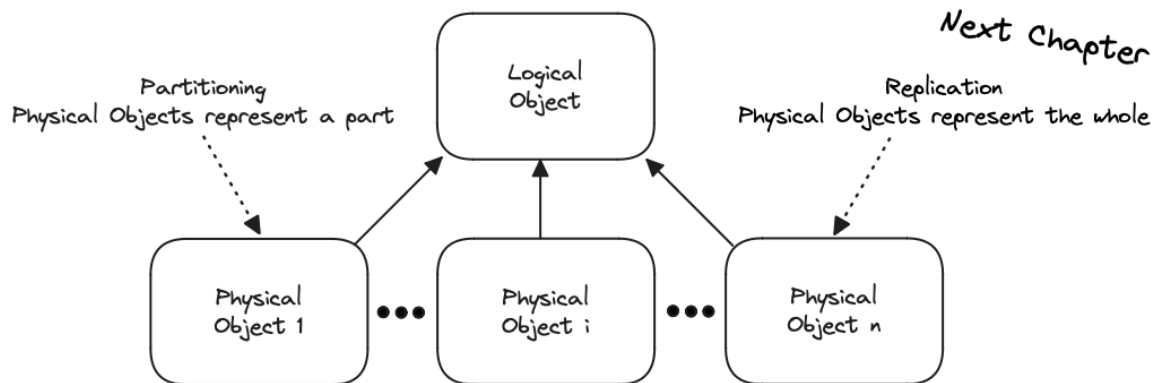


Figure 2. Thinking about Partitioning

Partitioning is used to improve the *scalability* of a distributed system. By dividing a logical object into multiple physical objects, called *partitions*, and distributing them across multiple components, the *nodes*, we can distribute *demand* across multiple nodes and avoid hitting the scalability limits of a single node.

AHA! Moment • Distributed Systems & Partitions

As defined in Chapter 1, a *distributed system* is a set of *concurrent, communicating components* [...]. Each component has exclusive access to its own local state, which is not accessible by any other components.

Although I don't usually conceptualize distributed systems this way, a distributed system is inherently and by definition

partitioned: each component and its local state can be understood as a partition.

Most often, we think about partitioning as a dataset that is *too large to store* on one node. Therefore we split that dataset into multiple smaller datasets and distribute them across multiple nodes.

However, partitioning is not only about growing beyond data volume. Partitioning is about growing beyond *any limitation* imposed by a *single resource* by using *multiple resources*. A common example is data volume, an intrinsic characteristic. Another common example is request volume, an extrinsic property.

AHA! Moment • Big Data is relative

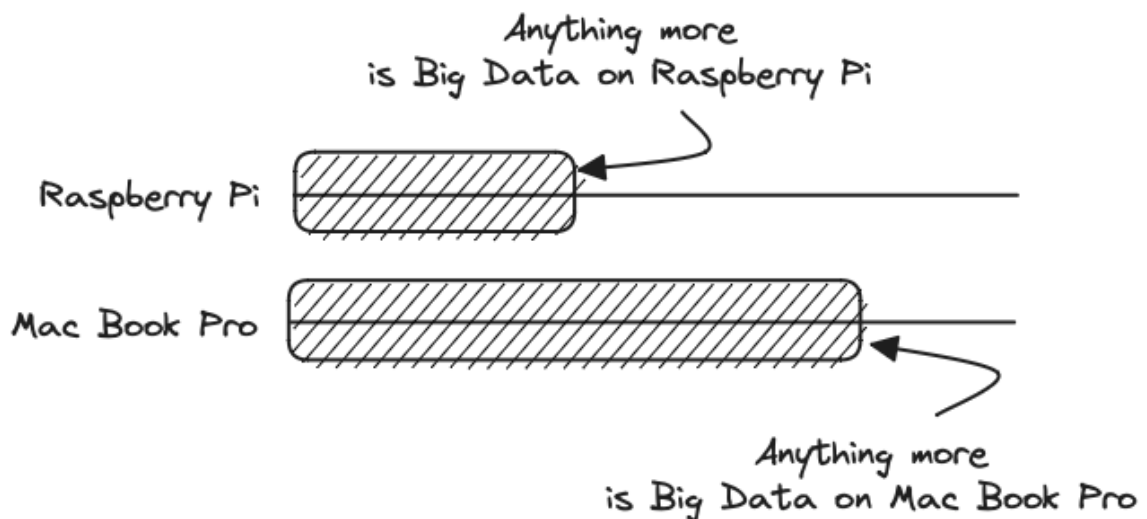


Figure 3. The relativity of Big Data

A pragmatic definition of Big Data is *anything that doesn't fit on one node*, in other words, Big Data refers to data storage and processing that requires partitioning. If, as shown in Figure X, a

node is a single key value register, then a dataset with two or more key value pairs is considered Big Data.

Mechanics of Partitioning & Balancing

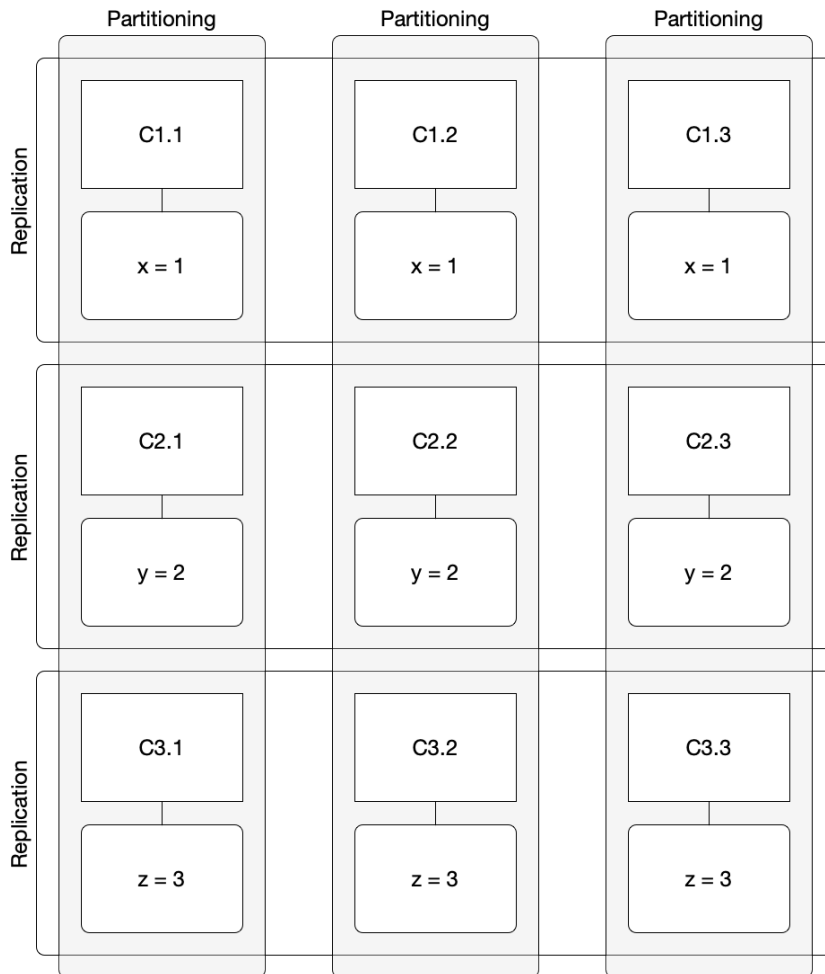


Figure 4. Partitioning a key-value store

For the remainder of this chapter, we will think in terms of a key-value store. Our dataset consists of a collection of key-value items. The dataset is split into partitions, and each partition is hosted on a node.

For example, in Figure 4., the dataset is { x=1, y=2, z=3 }, the partitions are P1, P2, and P3, and the nodes are N1, N2, and N3. Figure X lists the assignment of data items to partitions and the assignment of partitions to nodes.

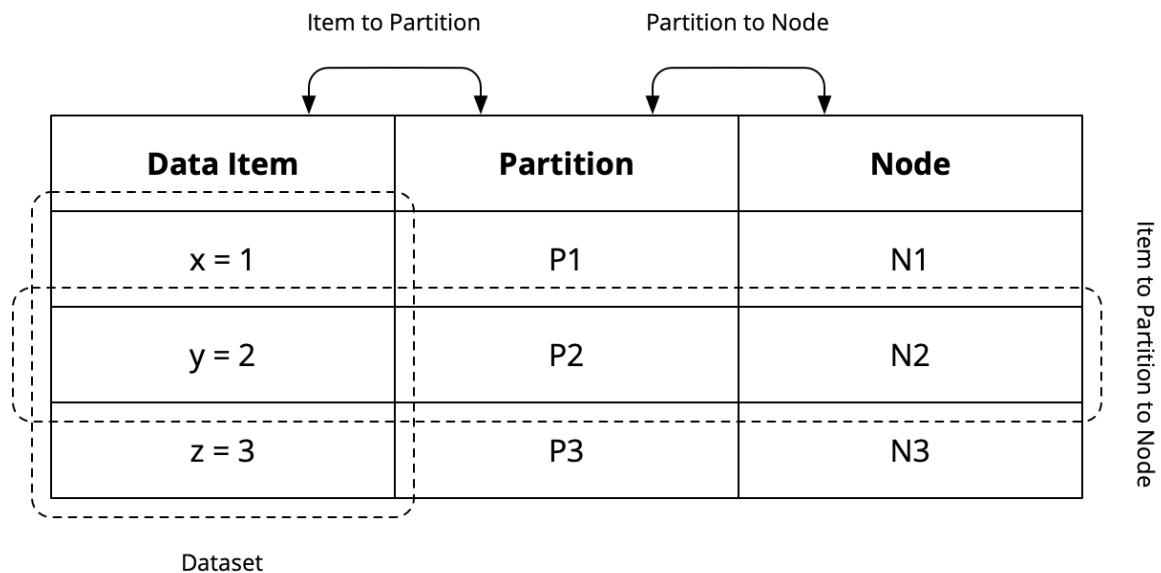


Figure 5. Assignment of data items to partitions and the assignment of partitions to nodes

In order to process more data than a single node can handle, the system splits the dataset into partitions and distributes them across multiple nodes.

When thinking about partitioning, two related yet different relationships are of interest: The assignment of a data item to a partition, and the assignment of a partition to a node.

Keep in mind

Partitioning may seem straightforward in theory, but partitioning is quite complex in practice. Partitioning requires carefully calculated tradeoffs and balancing between often competing

requirements. The design of your partitioning strategy will depend entirely on the unique characteristics of your system.

For example, let's consider a social media application like Twitter. In this case, users post tweets, while other users read tweets or post replies. One option is to partition your data by user. While this may result in well-balanced reads and writes for the average user, this strategy may become skewed for celebrities or influencers.

Another example to consider is an Internet of Things (IoT) application, where sensors periodically post measurements. In this case, you could partition your data by date, which ensures perfect data distribution for a fixed set of sensors. Every day contains the same amount of data. However, writes become skewed to only one partition.

There are other options to partition your data, not only by date, but by sensor type, measurement type, or region, each option with its own virtues and limitations.

Ultimately, designing an adequate partitioning strategy depends entirely on the unique characteristics and unique requirements of your system.

Frequently, partitioning is not a one-and-done process. You begin with an initial set of anticipated requirements and conditions, but as these requirements and conditions change, your partitioning strategy will need to adapt. This may require a nontrivial and nonobvious transition.

(Re)Partitioning



Figure 6. Partitioning

Partitioning refers to the assignment of data items to partitions. Repartitioning refers to the reassignment of previously assigned data items to different partitions, for example, when there is a change in the number of partitions.

Types of Partitioning

Partitioning can be categorized along different dimensions. Here, we will look at static versus dynamic partitioning, as well as horizontal versus vertical partitioning.

Static & Dynamic Partitioning

Static partitioning refers to a strategy where the number of partitions is fixed, while dynamic partitioning refers to a strategy where the number of partitions is variable. Note that static and dynamic partitioning only refers to the number of partitions, not the number of nodes.

- **Static Partitioning** With static partitioning, the number of partitions is fixed and cannot be changed *online*. Any changes to the number of partitions must be done offline, as it is considered an administrative operation of the system.
- **Dynamic Partitioning** With dynamic partitioning, the number of partitions is variable and can be changed *online*, that is, changing the numbers of partitions is a normal operations of the system. In other words, the system is elastic.

Static partitioning cannot adapt automatically to changing demand, i.e. the system is not elastic. Dynamic partitioning can adapt automatically to changing demand, i.e. the system is elastic. However, dynamic partitioning adds *a lot of* complexity (see Repartitioning).

Strategy	Static Partitioning	Dynamic Partitioning
Elasticity	✗	✓
Complexity	Low	High

AHA! Moment • No Change vs Slow Change

The term "static" does not only refer to "*no change*" but also "*slow change*" or "*infrequent change*". If changing the number of partitions is not part of the normal operations but part of the administration of a system the number of partitions can be considered static. Administrative actions are actions that require planning, the preparation of migrations and rollbacks, and can potentially cause downtime.

Horizontal & Vertical Partitioning

When thinking about horizontal and vertical partitioning, the most intuitive and tangible mental model is that of a relational database tables. A relational database table consists of rows and columns.

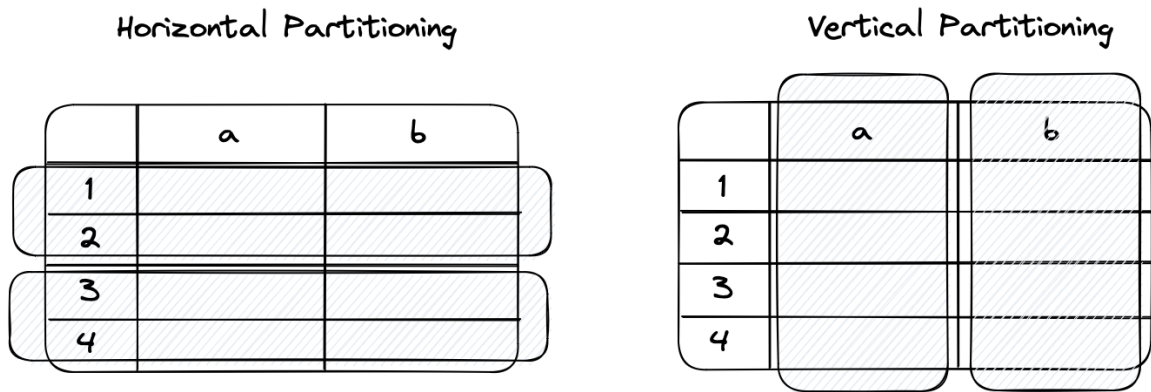


Figure 7. Horizontal & Vertical Partitioning

- **Horizontal partitioning** also known as sharding, refers to partitioning data based on rows. For example, in Figure 7., left, one partition contains rows 1 and 2, while the other partition contains rows 3 and 4.
- **Vertical partitioning** refers to partitioning data based on columns rather than rows. For example, in Figure 7., right, one partition contains the column a and one partition contains the column b.

Horizontal and vertical partitioning can be used in combination. Let's consider an application that manages user information. This information includes text-based data like a user's first name, last name, etc., and binary large objects (BLOBs) like a profile picture (Figure 8).

First, we can apply vertical partitioning. In this case, we can separate data by type, that is, we can separate the text-based profile data from the profile pictures. This is practical because these data types are fundamentally different. The text-based data, such as names, can be efficiently stored and searched in a

relational database, while the image files (profile pictures) are better suited to a file storage system designed to manage large data files.

Next, we can apply horizontal partitioning. In this case, we can distribute text-based profile data across multiple relational databases, one per horizontal partition. In addition, we can distribute profile pictures across multiple file storages, again one per horizontal partition.

In other words, we divide user information into two parts. First, we split data by type, text versus images, using vertical partitioning. Then, we distribute text and images using horizontal partitioning.

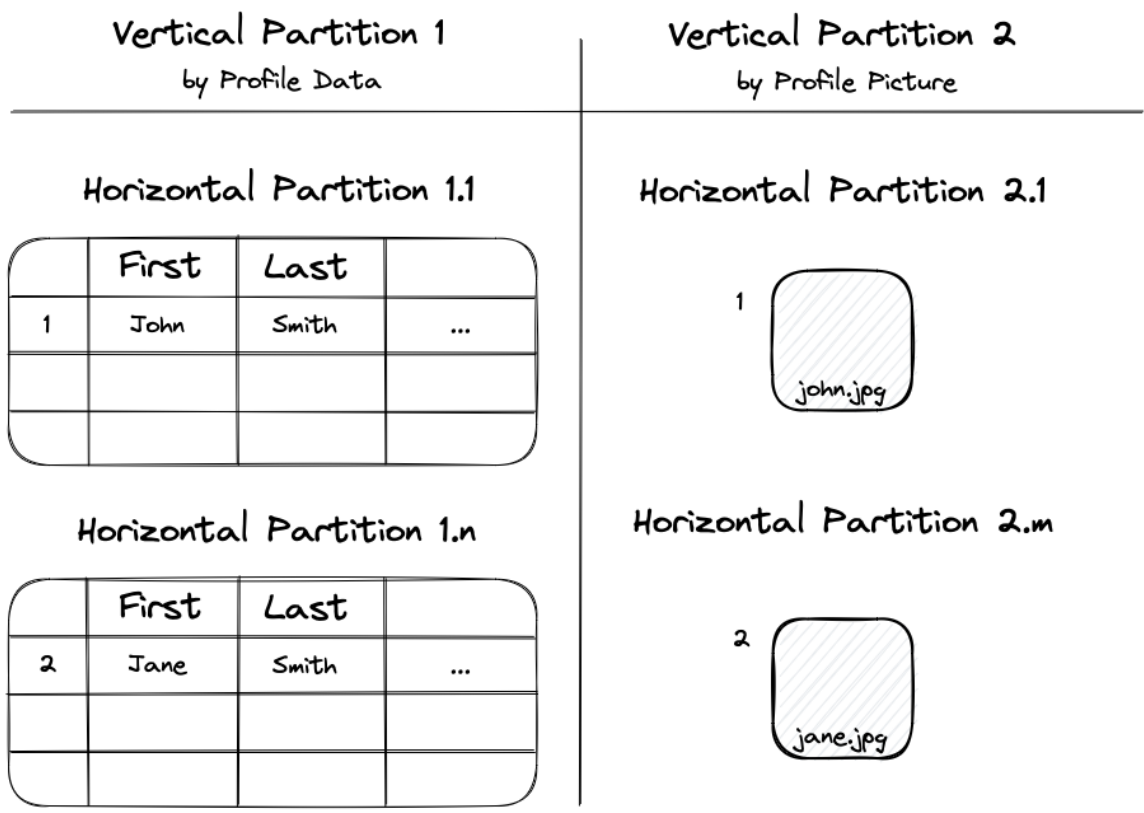


Figure 8. Partitioning user information

This partitioning strategy helps us to scale different aspects of our application independently of each other and efficiently meet demand.

Data Item to Partition Assignment Strategies

When data is partitioned, first, we need to determine which partition an item belongs to. Item-based assignment and directory-based assignment are two widely used approaches.

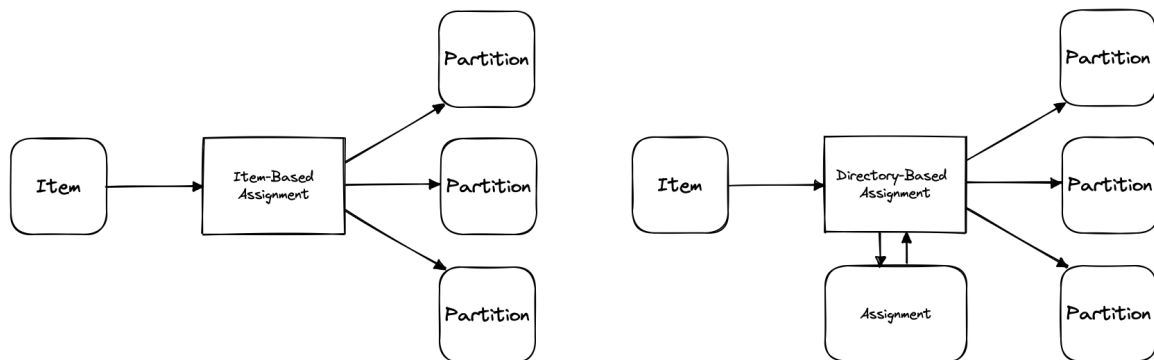


Figure 9. Item-based & Directory-based Assignment

The fitness of an assignment strategy is often measured in *variance* and *relocation*. Here, variance refers to the degree to which items are uniformly distributed across partitions. Relocation corresponds to the number of items requiring relocation when the number of partition changes.

Item-based Assignment Strategy

Item-based assignment is a partitioning strategy that assigns each data item to a partition based solely on its own characteristics, for example, in case of key value pairs, its key or

a hash of its key. Item-based assignment is stateless and therefore simple to develop and operate.

Listing 1.

```
partitions = {i: {} for i in range(5)}

def placement(key):
    # Simple placement function that works with integers.
    return key % 5

def place(self, (key, val)):
    # Calculate the partition ID based on our hash function.
    partition = placement(key)
    # Add the item to the appropriate partition.
    partitions[partition][key] = value

def get(self, key):
    # Calculate the partition ID based on our hash function.
    partition = placement(key)
    # Retrieve the item from the appropriate partition.
    return partitions[partition].get(key)
```

Directory-based Assignment Strategy

In contrast to an item-based assignment strategy, a directory-based assignment strategy is a function of the item and a separate component called a directory or lookup table. In other words, directory-based assignment is stateful and therefore more complex to develop and operate.

This strategy uses a directory or mapping system that provides the partition assignment based on both the item and the directory state. Unlike item-based strategy, directory-based strategy has knowledge of the system's state.

Listing 2.

```
partitions = {i: {} for i in range(5)}
assignment = {}

def place(key, value):
    # Instead of calculating the partition, randomly choose and
    # update the directory.
    partition = random.randint(0, 4)
    # Add the key to the directory
    assignment[key] = partition
    # Add the item to the appropriate partition.
    partitions[partition][key] = value

def get(key):
    # Retrieve the partition ID from the directory.
    partition = assignment.get(key)
    # Retrieve the item from the appropriate partition.
    if partition:
        return partitions[partition].get(key)
    else:
        return None
```

However, since the directory is a stateful component, the directory itself may be a bottleneck, that is, the limiting factor regarding scalability and reliability of the system. In other words, the directory itself may be in conflict with the goal of partitioning!

In summary, due to their simplicity, item-based assignment strategies are beneficial for systems where coarse grained control is sufficient. On the other hand, despite their complexity, directory-based assignment strategies are beneficial for systems where fine grained control is necessary.

AHA! Moment • Coarse Grained vs Fine Grained

Item-based assignment strategies work on a coarse grained level: Item-based assignment strategies are stateless, based on range or a hash function. While this assignment strategy is able to balance items across partitions, it is not able to place items on specific partitions. Now there is the risk of collisions, where two or more high-demand ("hot") items may be assigned to the same partition. There's no simple workaround because the assignment of items is determined entirely by the assignment function and not by manual intervention.

Directory-based assignment strategies work on a fine grained level: Directory-based assignment strategies are stateful, based on a directory or lookup table. This assignment strategy is able to place items on a specific partition.

Common Item-based Assignment Strategies

We need to find an assignment function that ensures we are able to *find* an item and *balance* items.

Let's run some experiments: Unix and Unix-like operating systems provide a file `words`, typically located at `/usr/share/dict/words`, with a list of English words. In this experiment we will compare the fitness of different assignment functions by assigning each word in the `words` file to one of five partitions.

We will use the code in Listing 3. to count how many items are assigned to each partition. At the time of writing, the `words` file

on my machine contains 235,976 words. Therefore, ideally, each partition should contain just shy of 42,000 items.

Listing 3.

```
partitions = {0: 0, 1: 0, 2: 0, 3: 0, 4: 0}

def placement(word):
    ...

with open('/usr/share/dict/words') as f:

    words = [word.lower() for word in f.read().split()]

    for word in words:

        partitions[placement(word)] += 1
```

Range Partitioning

One possible assignment strategy is based on key range, where each partition is responsible to host items with keys in a specific range of keys. The following code demonstrates an example assignment function that assigns keys to partitions based on the first letter of the key.

Listing 4.

```
def placement(key):
    if key[0] in ['a', 'b', 'c', 'd', 'e']:
        return 0
    if key[0] in ['f', 'g', 'h', 'i', 'j']:
        return 1
    if key[0] in ['k', 'l', 'm', 'n', 'o']:
        return 2
```

```
if key[0] in ['p', 'q', 'r', 's', 't']:
    return 3
if key[0] in ['u', 'v', 'w', 'x', 'y', 'z']:
    return 4
```

On my machine, running the script in Listing 3. with the placement function of Listing 4 yields the following distribution:

```
0: 67730
1: 33203
2: 35829
3: 73432
4: 25782
```

This distribution indicates that the partitions are not well balanced. Partition #3 and partition #0 are obvious hot spots revealing an uneven distribution of items across partitions.

Hash Partitioning

Another possible assignment strategy is based on hash value. The following code demonstrates an example assignment function that calculates a hash value for each key and assigns the key to one of five partitions based on the result.

Listing 5.

```
def placement(key):
    return hash(key) % 5
```

On my machine, running the script in Listing 3. with the placement function of Listing 5 yields the following distribution:

```
0: 47219
1: 47187
2: 47362
3: 47175
4: 47033
```

This distribution indicates that the partitions are well balanced. There are no hot spots in terms of uneven distribution of items across partitions.

Repartitioning

Over time, things change and the number of partitions may need to decrease or increase. As we will see, this is particularly challenging for item-based assignment strategies as the number of partitions has an outsized effect on placement.

We will use the code in Listing X to count the number of items remaining in their partition and the number of items that must be relocated to a different partition when the number of total partitions changes from 5 to 6.

Listing 6.

```
same = 0
diff = 0

def placement_5(word):
```

```

...

def placement_6(word):
    ...

with open('/usr/share/dict/words') as f:

    words = [word.lower() for word in f.read().split()]

    for word in words:

        if placement_5(word) == placement_6(word):
            same += 1
        else:
            diff += 1

```

Range Partitioning

Listing 7.

```

def placement_5(word):
    if word[0] in ['a', 'b', 'c', 'd', 'e']:
        return 0
    if word[0] in ['f', 'g', 'h', 'i', 'j']:
        return 1
    if word[0] in ['k', 'l', 'm', 'n', 'o']:
        return 2
    if word[0] in ['p', 'q', 'r', 's', 't']:
        return 3
    if word[0] in ['u', 'v', 'w', 'x', 'y', 'z']:
        return 4

def placement_6(word):
    if word[0] in ['a', 'b', 'c', 'd']:
        return 0
    if word[0] in ['e', 'f', 'g', 'h']:
        return 1
    if word[0] in ['i', 'j', 'k', 'l']:

```

```
        return 2
    if word[0] in ['m', 'n', 'o', 'p']:
        return 3
    if word[0] in ['q', 'r', 's', 't']:
        return 4
    if word[0] in ['u', 'v', 'w', 'x', 'y', 'z']:
        return 5
```

On my machine, running the script in Listing 6. with the placement function of Listing 7. yields an almost even split. Specifically, 48.6% of items remain in their current partition and 51.4% of items must relocate to a different partition.

```
Same: 114790
Diff: 121186
```

Hash Partitioning

Listing 8.

```
def placement_5(word):
    return hash(word) % 5

def placement_6(word):
    return hash(word) % 6
```

On my machine, running the script in Listing 6. with the placement function of Listing 8. yields a significantly skewed split. Specifically, 16.7% of items remain in their current

partition, while 83.3% of items must relocate to a different partition.

```
Same: 39443  
Diff: 196533
```

Strategy	Key Range	Hashing
Minimal variance	✗	✓
Minimal relocation	✗	✗

Consistent Hashing

David Karger and his colleagues at the Massachusetts Institute of Technology (MIT) introduced *consistent hashing* to achieve both minimal variance and minimal relocation.

A consistent hashing algorithm is designed such that, given n items and m partitions, only n/m items need to be relocated to a different partition on average.

Listing 9.

```
from uhashring import HashRing

def placement_5(word):
    return HashRing(nodes=[0, 1, 2, 3, 4]).get_node(word)

def placement_6(word):
    return HashRing(nodes=[0, 1, 2, 3, 4, 5]).get_node(word)
```

On my machine, running the scripts in Listing 3. and Listing 6. with the placement functions of Listing 9. yields great results. Specifically, 81.5% of items remain in their current partition, while 18.5% of items must relocate to a different partition.

```
0: 49084
1: 47753
2: 48172
3: 47144
4: 43823
```

```
Same: 192243
Diff: 43733
```

AHA! Moment • Thinking about Algorithms

Personally, I enjoy thinking in systems, but not in algorithms. However, many reading materials, such as blog posts, detail algorithms from the bottom up, leaving me to derive my own

mental model and understanding — I have to “reverse engineer” the big picture.

I enjoy reading materials that detail the specification of an algorithm from the top down, rather than the details of the implementation. In other words, when thinking about algorithms, I prefer to *Think in Signatures* or *Think in Interfaces* .

In that spirit, when thinking on a system level, I prefer to think about the effect of consistent hashing, not how consistent hashing is implemented.

(Re)Balancing



Figure 10. Balancing

Balancing refers to the assignment of partitions to nodes. Rebalancing refers to the reassignment of previously assigned partitions to different nodes, for example, as a response to a change in demand.

Over Partitioning

A twist on partitioning is over partitioning, where an excess of partitions is created compared to the number of nodes. This subsequently allocates multiple partitions to each individual node.

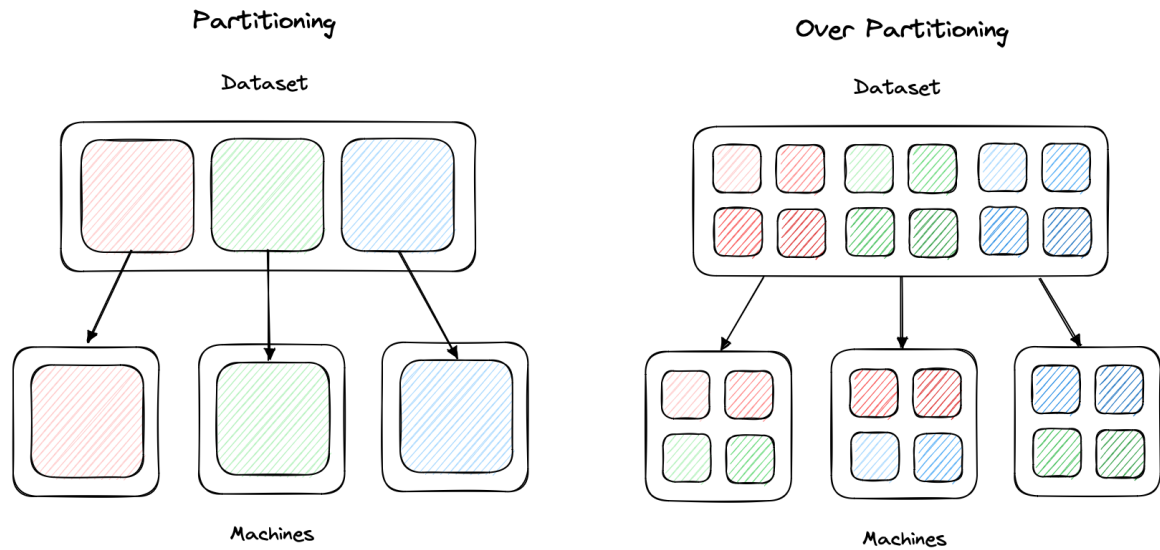


Figure 11. Partitioning & Over Partitioning

Let's consider a situation where we have the same number of partitions and nodes in a system. In this setup, each node gets exactly one partition, as shown in Figure 11. (left). Here, the system has no flexibility to accommodate fluctuating demand. Since the nodes and partitions are locked in a one-to-one relationship, we cannot make adjustments to accommodate increasing or decreasing demand.

Now let's consider a situation where we start with more partitions than nodes. In this setup, each node gets multiple partitions, as shown in Figure 11. (right). Here, the system has some flexibility. If demand drops, we can decommission some nodes and reassign orphaned partitions to remaining nodes. If demand rises, we can commission more nodes and spread out partitions among nodes.

However, there's a limit: the number of partitions determines the maximum number of nodes. Once all partitions are distributed, no additional nodes can be added. So, while this approach is more flexible than the first, this approach is still constrained by the number of partitions.

Conclusion

Partitioning and replication are two techniques to guarantee the scalability and reliability of distributed systems. Partitioning aims to improve the scalability of a system, growing beyond the scalability limits of a single resource.

Designing an adequate partitioning strategy depends entirely on the unique characteristics and unique requirements of your system. Additionally, things change. You begin with an initial set of anticipated requirements, but as these requirements change, your partitioning strategy will need to change as well.

Outlook

In the next chapter, we will explore replication.

REPLICATION & CONSISTENCY



In Chapter 1, we explored the question of why we need distributed systems. We came to the conclusion that distributed systems are required to guarantee scalability and reliability.

In the last chapter we explored partitioning, that is, how to overcome the scalability limits of a single component. In this chapter we will explore replication, that is, how to overcome the reliability limits of a single component.

Next up: Replication

Objectives:

- Understand Redundancy
- Understand Replication
- Understand Consistency

For me, the most complex aspects of distributed systems to reason about involve redundancy — especially in combination with failure.

Introduction

In Chapter 5, we explored the four fundamental principles of database transactions, which are famously referred to as ACID guarantees. ACID stands for Atomicity, Consistency, Isolation, and Durability. Let's recall the definition of durability:

- **Durability** guarantees that once a transaction is committed, its effects are permanent, preventing the possibility of "going back on its word."

The requirement of not being able to "go back on its word" or to "backtrack" is essential for many business processes. When we receive a positive acknowledgment for a promise, we rely on the durability of the promise to move forward.

For example, in the case of an e-commerce application, we rely on the durability of a payment processor's promise to collect payment to ship goods or render services. If the payment processor backtracks, we will be unpleasantly surprised when we realize we shipped the goods but actually never got the payment.

In a theoretical system model, where components are assumed to be completely reliable and not subject to any kind of failure, such as Crash-Stop or Crash-Recovery failures, backtracking is not possible. All promises made by the system are guaranteed to be fulfilled since components cannot fail. (Recall that we are not considering Byzantine failures, so the system must not lie.)

However, in a practical system model where components are subject to failures, backtracking is possible. For example, if a

component responsible for processing transactions fails, the system may not be able to fulfill promises made to users.

To prevent this situation, that is, to prevent a single point of failure, we add redundancy to the system.

AHA! Moment • Redundancy & Scalability

Redundancy improves the *reliability* of a distributed system. Nevertheless, redundancy *aids* the *scalability* of a distributed system: Redundancy distributes data across multiple nodes. Subsequently, load partitioning (e.g. round robin load balancing) distributes the load across replicas. However, as we will see in subsequent chapters, sometimes redundancy can have the opposite effect and decrease the scalability of a system.

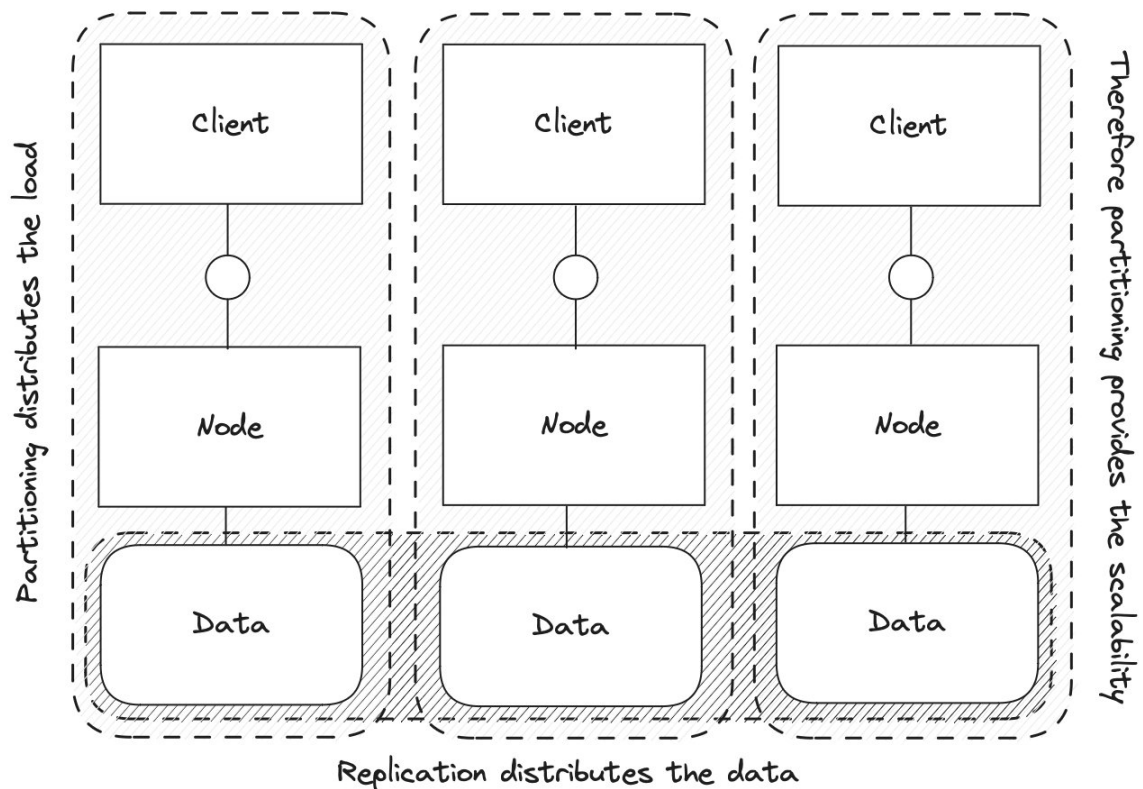


Figure 1. Replication distributing data, partitioning distributing load

In conclusion, the relationship between redundancy and scalability is not straightforward.

Redundancy

Redundancy refers to the duplication and coordination of subsystems, so that an increase in the duplication factor results in increased reliability and/or scalability.

There are two types of redundancy: Static Redundancy and Dynamic Redundancy.

- **Static Redundancy** refers to redundancy where the set of components and the set of interactions between the components do not change during the lifetime of a system. This type of redundancy is predominantly present in hardware systems.
- **Dynamic Redundancy** refers to redundancy where the set of components and the set of interactions between the components do change during the lifetime of a system. This type of redundancy is predominantly present in software systems.

AHA! Moment • Duplication & Coordination

In my experience, our mental models often involve duplicating components, but rarely involve coordinating those duplicated components. However, a collection of components is not a system; rather, a collection of components must be composed into a coherent system, meaning that the components must be coordinated. This coordination can be as simple as a round-robin load balancing strategy or as complex as a consensus protocol,

but it must exist. Therefore, I emphasize the duality of duplication and coordination in my mental models.

Example

Figure 2 illustrates the “Hello World” of redundant (hardware) systems: a redundant logic gate. A logic gate takes n inputs and produces 1 output, computing a Boolean function f . For this exploration, the actual function is irrelevant.

$$y = f(x_1, x_2, \dots, x_n)$$

We assume that if the logic gate fails, the system fails. In other words, the logic gate is a single point of failure. So, how can we make the system more reliable? One solution is to add redundancy in the form of duplication and coordination to the system.

We can replicate the logic gate and use three logic gates instead of one. To coordinate the replicas, we need a majority gate. This way, if one logic gate fails, the other logic gates can compensate the failure. To avoid the majority gate becoming the single point of failure, we also replicate the majority gate.

The majority gate combines the outputs of the replicas and selects the majority vote of the replicas.

$$R = (a \wedge b) \vee (b \wedge c) \vee (a \wedge c)$$

To summarize, we replaced a single logic gate with a group of logic gates and majority gates. We ensure that the group of components behaves like the original single component but can tolerate the failure of one component.

The magic of redundancy!

A common implementation of duplication is replication. Replication means having multiple instances of "the same thing". The remainder of this chapter will focus on replication, while we will revisit general duplication in later chapters.

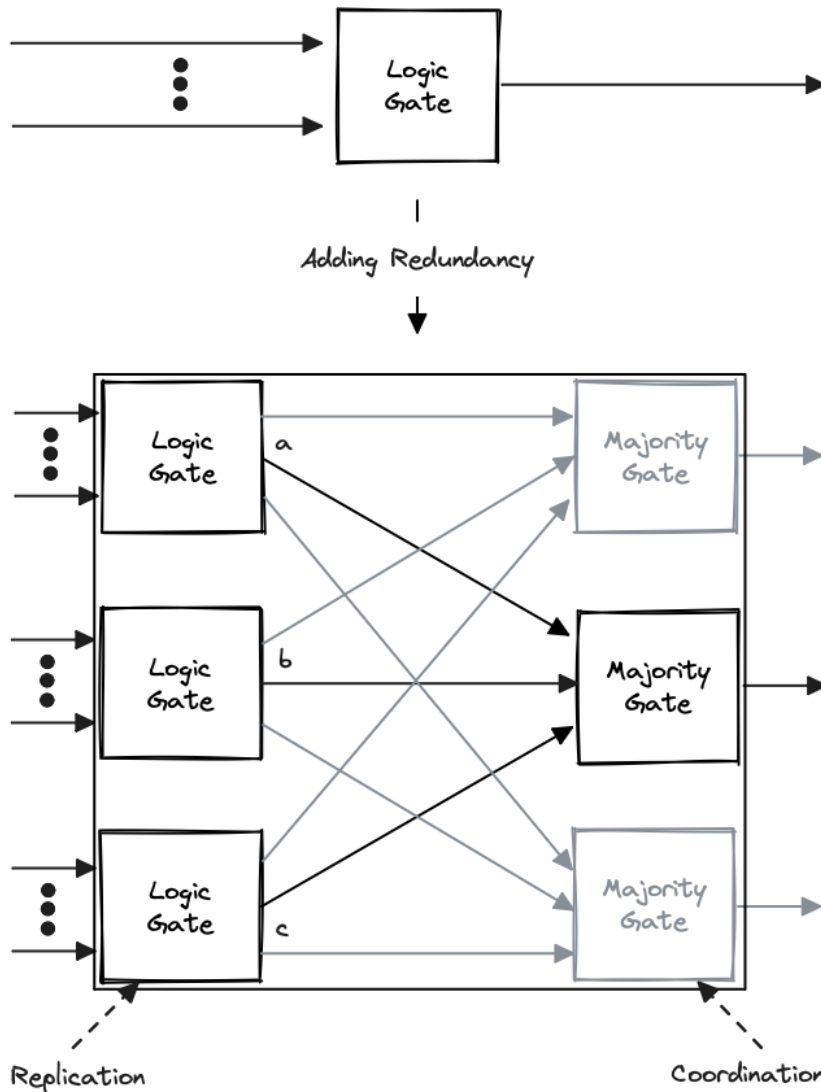


Figure 2. Redundancy as Duplication and Coordination

Thinking about Replication & Consistency

What is “one thing”? That appears at first to be a trivial, irrelevant, irreverent, absurd question. It’s not. The question illustrates how deeply

ambiguity and misunderstanding are ingrained in the way we think and talk.

— *Data and Reality, William Kent*

Before we explore the formal mental models of replication and consistency in software systems, let's first understand the nuances behind what we think of as, “one thing.”

Consider the example of a book, an author, and a library. An author writes a *book*¹ titled *Structure and Interpretation of Computer Programs* (SICP). A library is filled with a vast collection of *books*², including 10 copies of SICP.

What do we mean by the term ‘book’? Does the term ‘book’ refer to the work of the author¹ (the logical item) or to the copies in the library² (the physical item)?

Now consider editions: The author writes SICP 1st edition in 1984, SICP 2nd edition in 1996, and SICP js edition in 2022.

Editions of *Structure and Interpretation of Computer Programs*

Name	Edition	Year
SICP	1st	1984

Name	Edition	Year
SICP	2nd	1996
SICP	Javascript edition	2022

Are SICP 1st, 2nd, and js editions the same or different books?
The examples in the SICP 1st and 2nd editions are authored in the Scheme programming language, while the examples in SICP js edition are authored in the Javascript programming language. So, are the 1st and 2nd editions sufficiently similar to each other to be considered the same, while the js edition is deemed different?

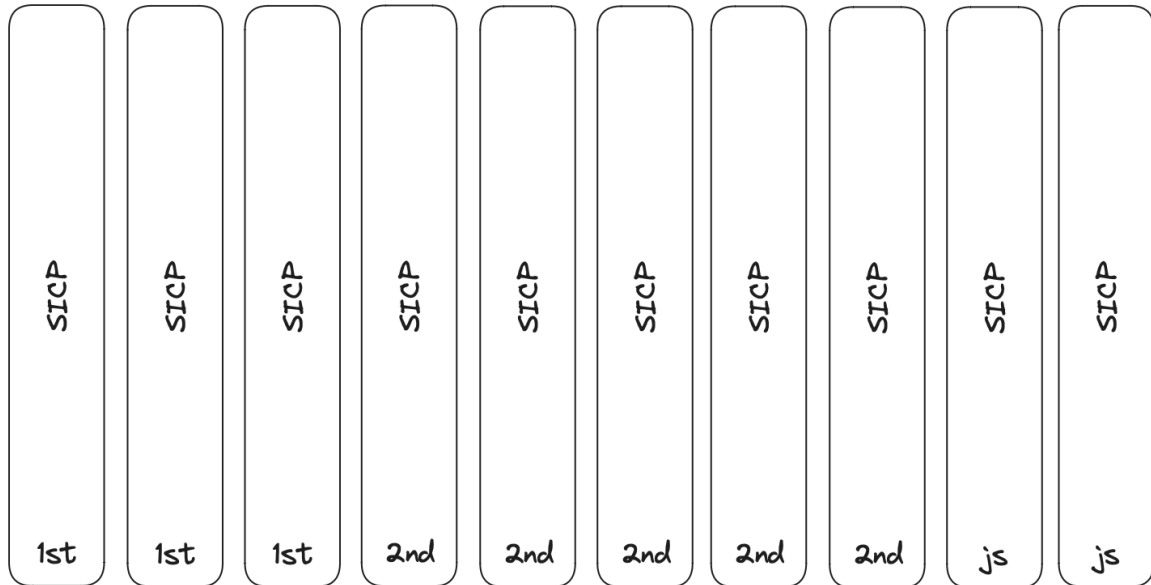


Figure 3. Library inventory of Structure and Interpretation of Computer Programs

Figure 3 illustrates the library's inventory. How many copies of SICP are available?

This question leads us to the core of replication and consistency. Replication and consistency is not only about the number of physical items that exist, but also about whether these physical entities faithfully represent the logical item.

The answer to this question depends entirely on one's perspective:

- Some readers may argue that any edition covers the relevant concepts, and therefore claim that there are 10 books in stock.
- Other readers may argue that the relevant concepts can only be faithfully captured in the classic Scheme programming language, and therefore claim that there are 8 books in stock.
- Some readers may argue that the relevant concepts can only be tangibly captured in the modern Javascript programming

language, and therefore claim that there are 2 books in stock.

The exploration of “one thing” forces us to scrutinize the underlying principles of identity and equivalence, *Oneness* and *Sameness*. Understanding these principles is foundational to replication and consistency, whether in the context of books or distributed software systems.

The complexity and ambiguity of these principles are not just challenges to overcome; they are intrinsic to the reality of distributed software systems themselves.

The Epic Struggle

The CAP theorem is too simplistic and too widely misunderstood to be of much use for characterizing systems. Therefore I ask that we retire all references to the CAP theorem, stop talking about the CAP theorem, and put the poor thing to rest. Instead, we should use more precise terminology to reason about our trade-offs.

— Please stop calling databases CP or AP, Martin Kleppmann

Before we continue to explore replication and consistency, I want to address the elephant in the room: The CAP Theorem. The CAP Theorem states that any replicated store can provide only

Consistency or **A**vailability under **P**artitioning. So the CAP theorem divides the world into **CP** and **AP** systems.

Informally, consistency refers to the guarantee that every read request receives a response reflecting the value of the most recent write request, while availability refers to the guarantee that every request receives a response.

The CAP Theorem has had both positive and negative effects on the distributed system community. On the positive side, CAP has taught us to think about the inherent trade-offs in distributed systems. However, on the negative side, CAP is sometimes misused as a definitive argument to shut down a conversation, even if CAP may not be applicable to the particular situation at hand.

In the next chapter, we will discuss and explore the CAP theorem in great detail, after establishing the fundamentals of replication and consistency.

Replication

The term *replication* refers to representing a single *logical* object by multiple, identical *physical* objects.

Replication is a technique used to improve the *reliability* of a distributed system. By replicating a logical object into multiple physical objects, called *replicas*, we can avoid reaching the reliability limits of a single object.

Central to the discussion of replication is *replication transparency*: Replication transparency refers to the system's

ability to hide the existence of multiple objects providing the illusion of a single object. In essence, the discussion of replication transparency is a discussion how the system manages to balance between concealing and revealing the details of replication.

This concept challenges software engineers to balance various factors such as consistency, availability, and latency in the system.

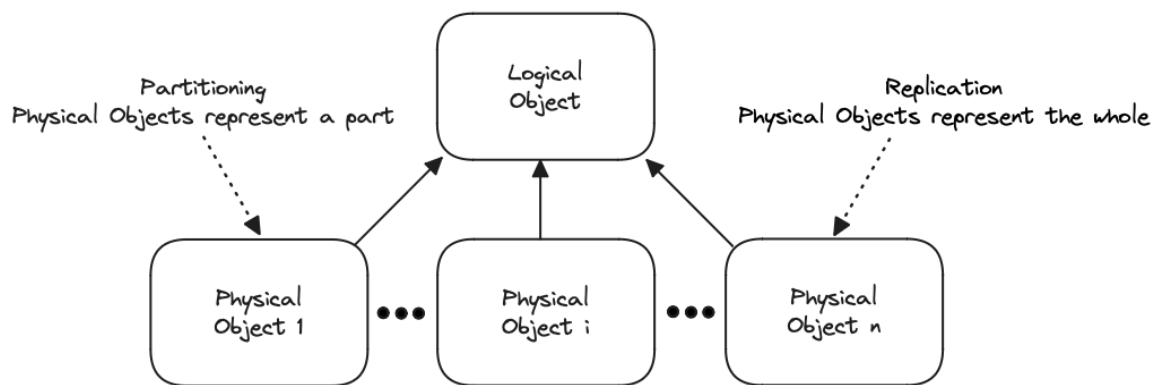


Figure 4. Replication

Mechanics of Replication

Replication is not only applicable to stateful components, which manage state in their local storage, but it is also relevant for stateless components.

For example, logic gates deployed redundantly are stateless components. Their outputs depend solely on their inputs. In the context of software engineering, we often redundantly deploy stateless services and load balance between them. This form of replication is so straightforward that we often do not even recognize it as replication.

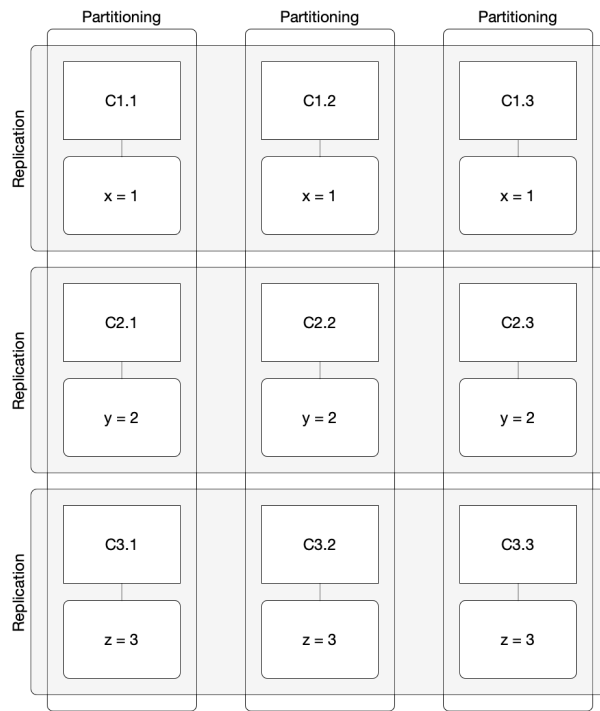


Figure 5. A replicated key-value store

In this chapter, we focus on stateful components as they harbor most of the complexity related to this topic.

Let's consider a key-value store, which comprises a collection of key-value items. However, unlike in Chapter 7, we ignore partitioning and assume that the entire data set can fit on a single node.

In a distributed system, complexity in replication arises from *change*. When an object remains static, replicating the object is straightforward. Yet, when the object changes, we must ensure that these *changes are properly propagated*.

AHA! Moment • Not All Change Is Created Equally

Not all changes in a system have the same consequences on (our knowledge of) the state of the system. Some changes can

potentially invalidate our current knowledge. While others preserve our current knowledge.

Consider a counter where we only care if its value exceeds 10:

- If the counter supports both `increment` and `reset` operations, any modification could invalidate our current knowledge. Even if we know the value crossed 10, a subsequent `reset` operation takes us back to 0.
- However, for a counter that only supports an `increment` operation, once its value crosses 10, further increments will not invalidate our knowledge that the counter is above the threshold.

This property, where a new change don't contradict current knowledge, is known as monotonicity.

System Model

As highlighted in Chapter 1, a distributed system consists of concurrent, communicating components that communicate by sending and receiving messages over a network. Importantly, within this framework, exactly one component or the network takes a single step at a time.

As highlighted in Chapter 2, a distributed system is partially synchronous consisting of components subject to Crash-Stop failure, Omission failure, and Crash-Recovery failure communicating over an unreliable network.

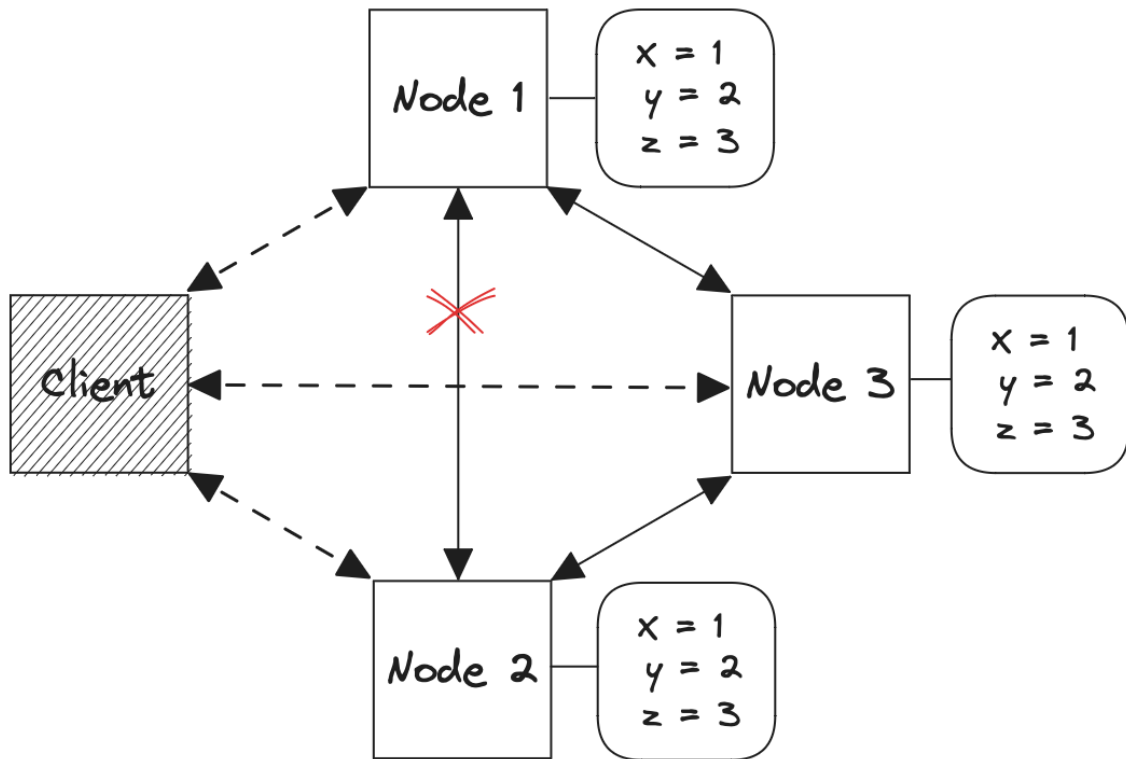


Figure 6. The network as point-to-point communication links between components

Recall Chapter 1, Figure 1. where we modeled the network as a central component every component was connected to. In this chapter, we will model the network as point-to-point communication links between components. This model enables us to concisely think about—and visualize—network partitions.

In the context of replication, we attribute message loss to a network partition, that is, to a temporary, intermediate, or permanent failure of one or more communication links.

For example, Figure 6 illustrates one client, three replicas, and a network partition between replica 1 and replica 2.

Replication Lag

Based on this system model, instantaneous propagation of changes is impossible, resulting in an inherent replication lag. Since exactly one component or the network can take a single step at a time, we cannot update all replicas simultaneously. Thus, we must update them sequentially, one replica after another (Figure 7).

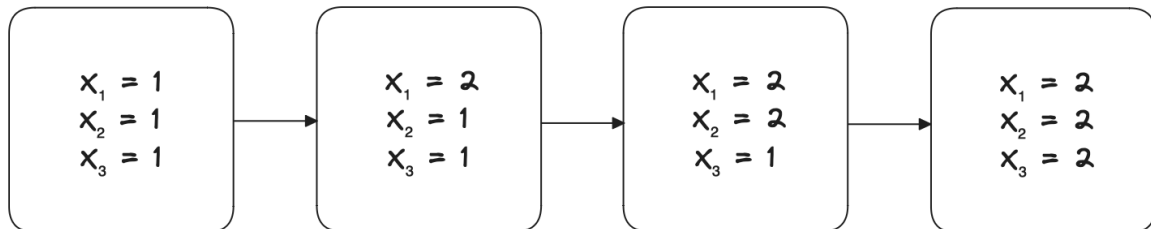


Figure 7. Replication Lag

Replication lag is not merely a temporal delay; it is an inherent aspect of replicated, distributed systems. This lag is the source of many challenges, such as the potential loss of replication transparency.

We can differentiate between *inherent* replication lag, which is a result of the nature of distributed systems, and *imposed* replication lag, which is a result of network partitions and component failures. This dichotomy captures the distinction between unavoidable replication lag and additional replication lag within a distributed system.

Synchronous vs Asynchronous Replication

In synchronous replication, an operation is not considered complete until the changes have been replicated and acknowledgments have been received from all other nodes. In other words, the completion of an operation is tightly coupled

with the replication process. Replication takes place in the foreground.

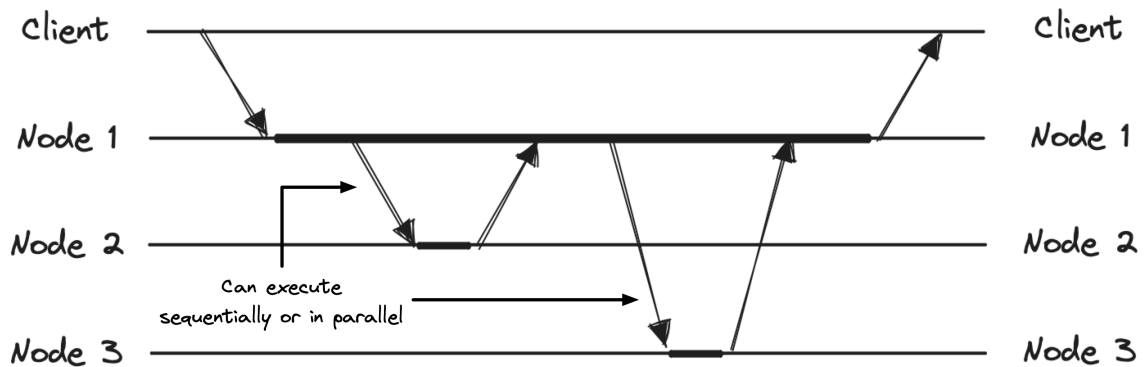


Figure 8. Synchronous Replication

In asynchronous replication, the operation is considered complete as soon as it's processed on the initial node, without waiting for the changes to be replicated and acknowledgments to be received from the other nodes. In other words, the completion of an operation is loosely coupled with the replication process. Replication takes place in the background.

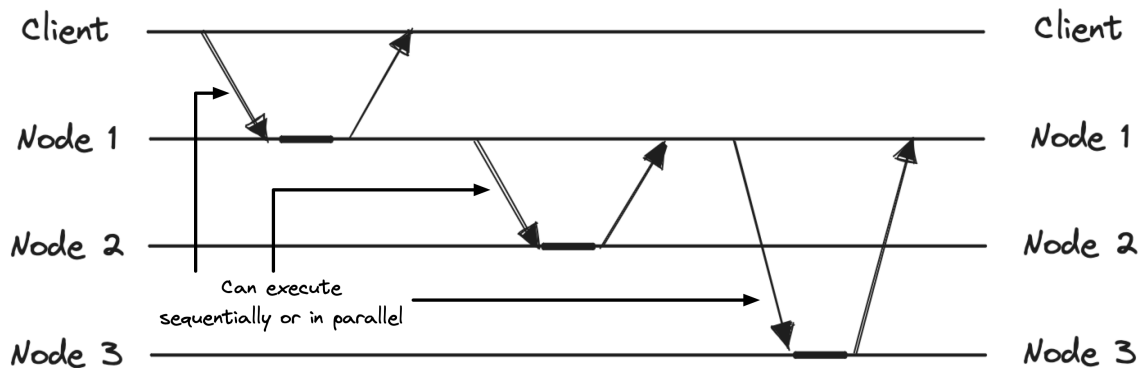


Figure 9. Asynchronous Replication

Synchronous replication ensures immediate consistency across all nodes, but it may impact latency and availability due to its dependence on every node's timely response.

In practice, many systems employ a hybrid strategy for replication: When a change is replicated, a majority of replicas is required to acknowledge the operation. This majority is often called the *quorum*. Essentially, replication in the quorum happens in the foreground. However, once the quorum is achieved, changes to replicas outside of the quorum can be asynchronous. Replication outside the quorum may happen in the background.

State-based vs Log-based Replication

Assuming that a system is a deterministic state machine, we have two primary options to replicate change: We can replicate the current state of the system, or we can replicate the sequence of operations that leads to the state.

```
step : State → Operation → State
```

For this discussion, the state machine *must* be deterministic. That means multiple replicas of the same state machine, beginning in the same start state and receiving the same operations in the same order, will arrive at the same state.

For example, consider a counter with a single `increment-by` operation that starts at state `counter = 0`. For this example, we assume the client issues a `increment-by(2)` followed by a `increment-by(3)` operation.

In state-based replication, changes are replicated by sending (a diff of) the current state across the wire — regardless of the

series of operations that led to the current state. So in this example, change is propagated as a sequence of `counter = 2` and `counter = 5` messages.

In log-based replication, changes are replicated by sending the sequence of operation across the wire. So in this example, change is propagated as a sequence of `increment-by(2)` and `increment-by(3)` messages.

In summary, state-based replication focuses on propagating the current state, while log-based replication focuses on propagating the sequence of operations leading to the current state. In practice, log-based replication is emerging as the industry favorite.

Single-Leader, Multi-Leader, Leader-Less

Replication strategies can be distinguished by the number of leader nodes. In systems with leadership, one or more leader nodes are authorized to accept and handle requests, whereas follower nodes are restricted and may not process requests directly.

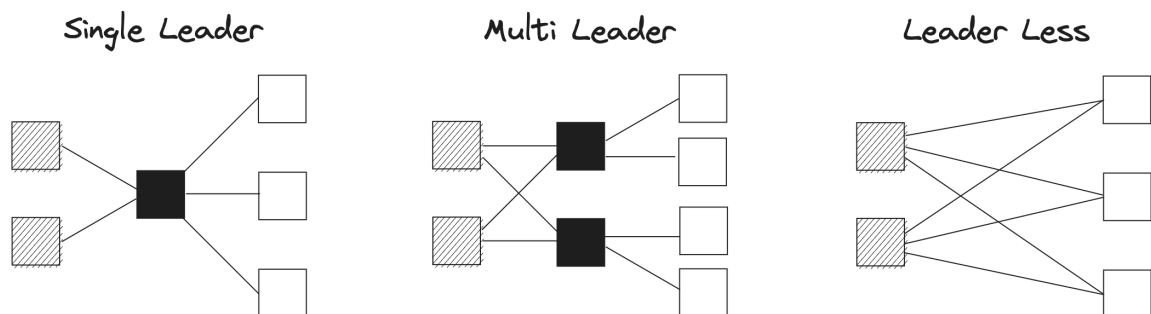


Figure 10. Single-Leader, Multi-Leader, and Leader-Less

In single-leader systems, there is one dedicated leader node that coordinates operations. Only the leader node accepts operations

and propagates changes to its followers. This creates a “chain of command,” but can become a single point of failure, almost negating the reason for replication.

In multi-leader systems, more than one dedicated node can act as a leader that coordinates operations. Any leader node can accept an operation. This avoids a single point of failure, but without a “chain of command,” concurrent operations may *conflict*, requiring *conflict resolution*.

In leader-less systems, there is no designated leader node that coordinates operations. All nodes are able to accept any operation. Conflicts that arise from concurrent operations are again resolved using conflict resolution.

A wide array of conflict resolution strategies exist, ranging from timestamp-based methods like “last write wins” to techniques using Conflict-Free Replicated Data Types (CRDTs). While these offer varied solutions for managing conflicts, their practical application is not straightforward. For instance, the seemingly simple “last write wins” can result in unexpected overwrites from a client's perspective. This highlights the need for careful consideration and understanding of the chosen conflict resolution strategy.

AHA! Moment • Leaders are for writes & follower are for reads?!

Sometimes authors oversimplify leader-based replication and state that *leader nodes process writes and follower nodes process reads*. While that is indeed a common configuration, that statement neglects the issue of replication lag.

Even though distributing reads across follower nodes can optimize performance, these nodes might not always reflect the most recent data due to the inherent delay in propagating updates. To ensure access to the most up-to-date data, reads must be directed to the leader node.

Consistency in Distributed Systems

In the context of software engineering, the term *consistency* is an overloaded term with *many* different interpretations.

For example, in the context of database systems, consistency refers to the guarantee that a transaction transitions the database from one consistent state to another consistent state (see Chapter 5).

In another example, in the context of the CAP theorem, consistency refers to Linearizability, a guarantee that a replicated system behaves observably equivalent to a non-replicated system (see Chapter 9).

In this section, we will define the term consistency in the context of distributed systems.

System Model

Here, a distributed system is a set of concurrent *processes* and a set of *objects*. A process is a sequence of steps, that is, a sequence of operations on objects. An operations is not instantaneous, instead an operation is delineated by its invocation and completion.

The history of a system is the sequence of operations including their concurrent structure, that is, the sequence of invocation and completion events order by their real-time occurrence.

```
H = [I•A.1, I•B.1, C•A.1, I•A.2, C•B.1, C•A.2]
```

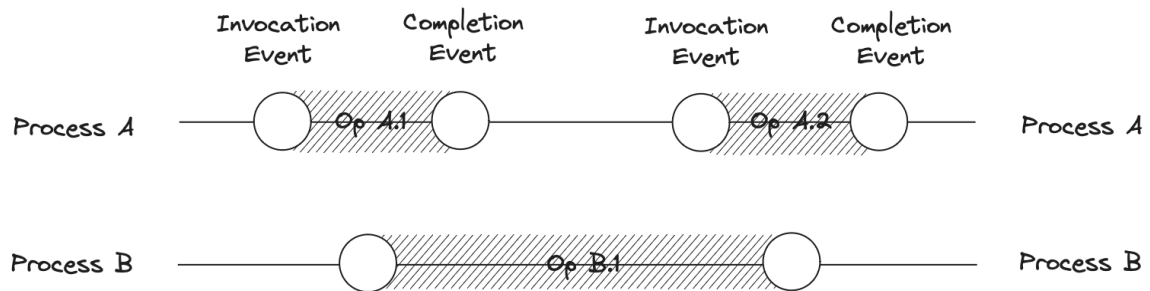


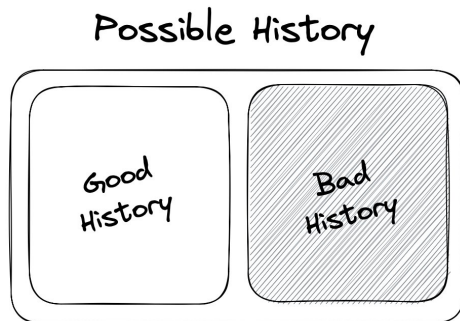
Figure 11. System Model

Consistency Model

A consistency model is a set of histories: A consistency model defines which histories are considered *good*, *legal*, or *valid* and which histories are considered *bad*, *illegal*, or *invalid*.

Typically we use consistency models to specify the guarantees a system makes, for example “This system is serializable” or “This system is linearizable”.

In other words, a consistency model is a *predicate* on a history, that is, a predicate on a sequence of invocation and completion events .



Consistency models may be disjoint (Figure 13, 1), overlapping (Figure 13, 2), or subsets (Figure 13, 3).

In the context of consistency models, those that are subsets (i.e., include fewer valid histories) are referred to as stronger, while those that are supersets (i.e., include more valid histories) are considered weaker. This creates a hierarchy between consistency models based on the subset-superset relationship.

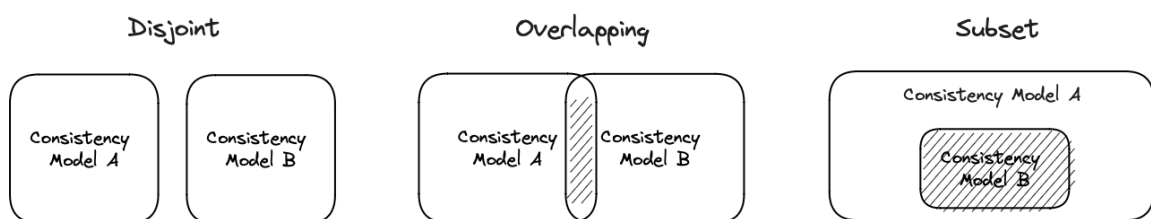


Figure 13. Disjoint, Overlapping, and Subsets

AHA! Moment • Violating Consistency?!

A common misconception among software engineering is the notion of "violating consistency." When we talk about a system violating consistency, what we really mean is that the system has failed to meet the guarantees of a specific *consistency model*.

In short: A system cannot violate consistency as a concept, a system can only violate the terms of a specific consistency model!

Common Consistency Models

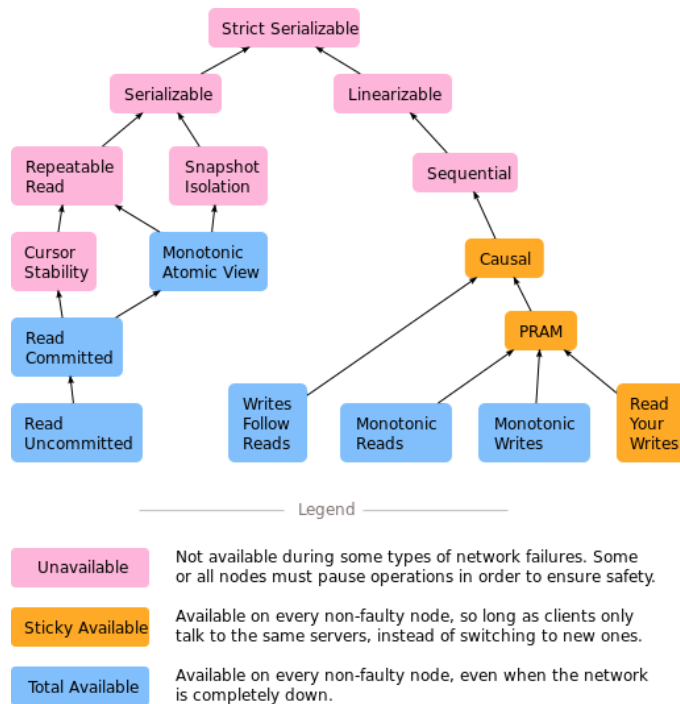


Figure 14. Common Consistency Models and their relationship from <https://jepsen.io/consistency>

Recall from Chapter 1 that correctness is application specific: As a software engineer, you have the ability to define the guarantees of your system and decide what behavior is desirable, tolerable, or intolerable. This includes consistency models! You have the ability to define which histories are good and which histories are bad. However, over the last few decades, a set of common, well-defined consistency models has emerged, each with different virtues and limitations. (Figure 14.)

Virtues and Limitations

How do you choose between different consistency models? What are the trade offs between stronger consistency models and weaker consistency models?

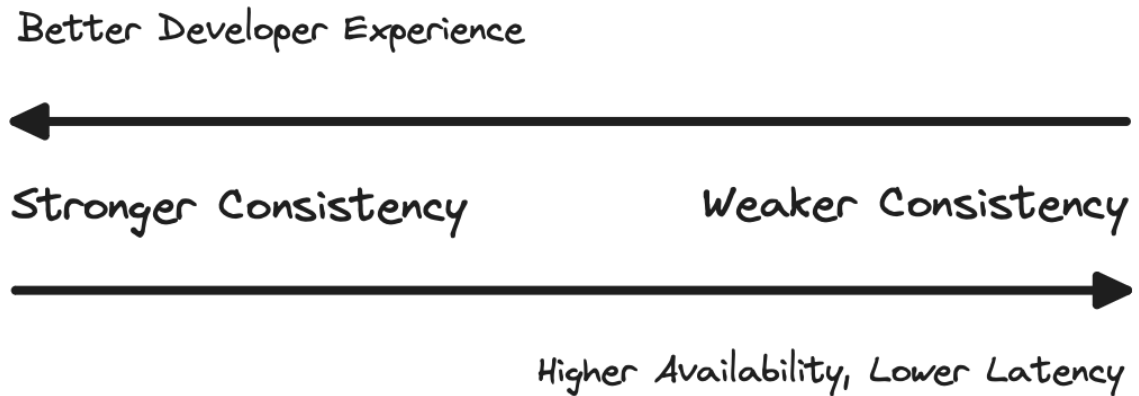


Figure 15. Stronger vs Weaker Consistency Models

When you shift to the left, that is, when you commit to a stronger consistency model, you lose availability and latency. But what benefits are gained in exchange? Conversely, when you shift to the right, that is, when you commit to a weaker consistency model, you gain better availability and latency. But what might be compromised in return?

In essence, you have to trade off availability and latency for developer experience. High availability and low latency come at the cost of being unable to prevent issues such as Lost Update and Write Skew, as well as provide recency bounds.

On one hand, a stronger consistency model improves the developer experience by mitigating more of the effects of replication. However, this mitigation requires more coordination between replicas, which limits availability and increases latency.

On the other hand, a weaker consistency model exposes more of the effects of replication but requires less coordination between replicas. This allows for better availability and lower latency.

You have to strike a balance that aligns with your expectations and requirements.

Conclusion

Redundancy aims to improve the reliability of a system, growing beyond the reliability limits of a single resource.

Redundancy refers to the duplication and the coordination of subsystems so that an increase in the duplication factor translates into an increase in reliability.

A common implementation of duplication is replication, the employment of multiple instances of “the same thing,” where one logical object is represented by multiple identical physical objects.

However, the question of sameness is a nuanced question, leading to many different answers, also known as consistency models. A consistency model defines which histories are considered *good*, *legal*, or *valid* and which histories are considered *bad*, *illegal*, or *invalid*.

Consistency models trade off availability and latency for developer experience, requiring you to strike a balance that aligns with your expectations and requirements.

Outlook

In this chapter, we laid the foundations to think about replication and consistency in distributed systems. In the next chapter, we will visit some of the more advanced topics around replication and consistency, including a detailed discussion of CAP theorem.

REPLICATION & CONSISTENCY



In the last chapter we explored replication and consistency. We saw that a stronger consistency model improves the developer experience by mitigating more of the effects of replication, but requires more coordination between replicas, which limits availability and increases latency. A weaker consistency model exposes more of the effects of replication but requires less coordination, which allows for better availability and lower latency.

In this chapter, we will explore two consistency models at the ends of the spectrum, *Linearizability* as one of the strongest and *Eventual Consistency* as one of the weakest. Additionally, we will explore the CAP Theorem, perhaps the most well-known principle in distributed systems that attempts to formalize the conflict between consistency and availability.

Objectives:

- Understand Linearizability
- Understand Eventual Consistency
- Understand the virtues and limitations of the CAP Theorem

Introduction

As outlined in the previous chapter, consistency models range from stronger consistency models that offer a better developer experience to weaker consistency models that offer higher availability and lower latency.

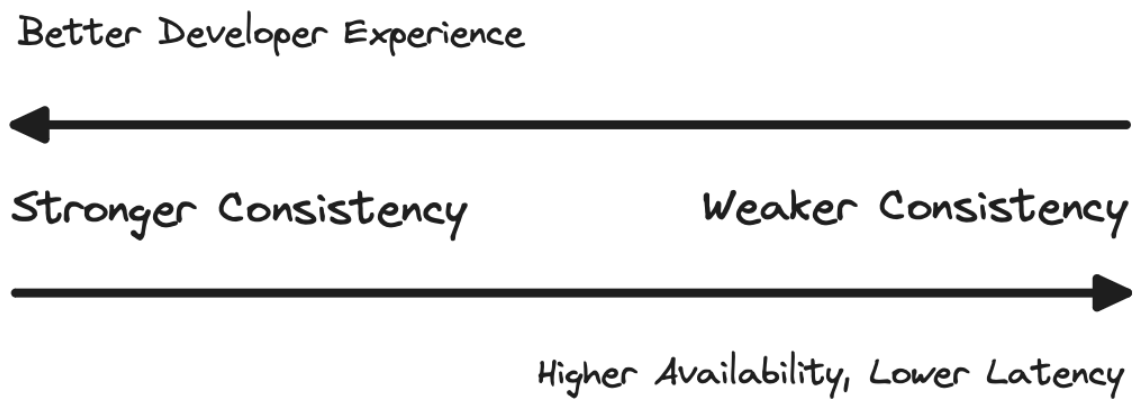


Figure 1. Stronger vs. Weaker Consistency Models

While challenging to quantify, a good developer experience is one that is intuitive and easy to reason about, leaving no room for unexpected and especially unwanted outcomes.

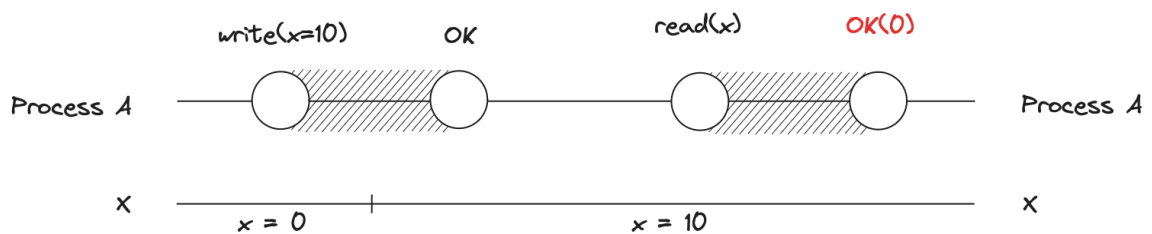


Figure 2. Unexpected & Unwanted Surprises

Figure 2 illustrates a process `A` accessing a single read write register `x`. Initially, `x` has the value `0`. `A` sets `x` to the value of `10`. At that point, we expect all subsequent reads of `x` to yield

the value `10`. However, in this example, the read of `x` yields the value of `0`.

Is this the correct behavior? The answer to that question depends on our definition of correctness.

- If we expect to read *the most recent value* then the answer is no, this behavior is incorrect.
- If we expect to read *any previous value* then the answer is yes, this behavior is correct.

Reading the most recent value is formalized by *Linearizability*, while reading any previous (more accurately, intermittent) value is formalized by *Eventual Consistency*.

Modern systems offer the developer a range of consistency levels to choose from. Figure 3 illustrates the available consistency levels in Azure CosmosDB, Microsoft's globally distributed, multi-model database management system.

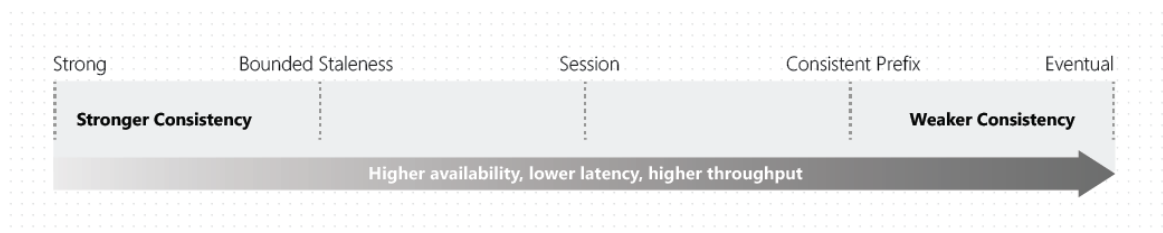


Figure 3. Consistency Levels of Azure CosmosDB
(<https://learn.microsoft.com/en-us/azure/cosmos-db/consistency-levels>)

Linearizability

On one hand, *Linearizability*, also known as *Atomic Consistency*, stands out as one of the strongest consistency models.

Linearizability provides a *recency guarantee*: as soon as an

operation completes, all subsequent operations will witness its effects.

Linearizability was initially introduced in the context of concurrent systems, providing a formal framework for specifying the semantics of shared objects in a multiprocessor environment. When applied to replicated, distributed systems, linearizability can provide a formal framework for specifying the semantics of shared objects in a replicated, distributed environment.

In essence, linearizability ensures that a replicated, distributed system behaves like a non-replicated, non-distributed system. In other words, linearizability ensures replication transparency.

System Model

Here, a distributed system is a set of concurrent *processes* and a set of *objects*. A process is a sequence of steps, that is, a sequence of operations on an object.

Object type defines the set of possible values and the set of possible operations to create and manipulate instances. An operation is not instantaneous; instead, an operation is delineated by its invocation and completion. We model an operation on an object as an invocation and a completion pair.

```
Operation = Invocation • Completion
```

Processes are strictly sequential, that is, each process issues a sequence of operations to objects, alternately issuing an

invocation and then receiving the associated response.

The history of a system is the sequence of operations of all processes including their concurrent structure, that is, the sequence of invocation and completion events ordered by their real-time occurrence.

```
H = [I•A.1, I•B.1, C•A.1, I•A.2, C•B.1, C•A.2]
```

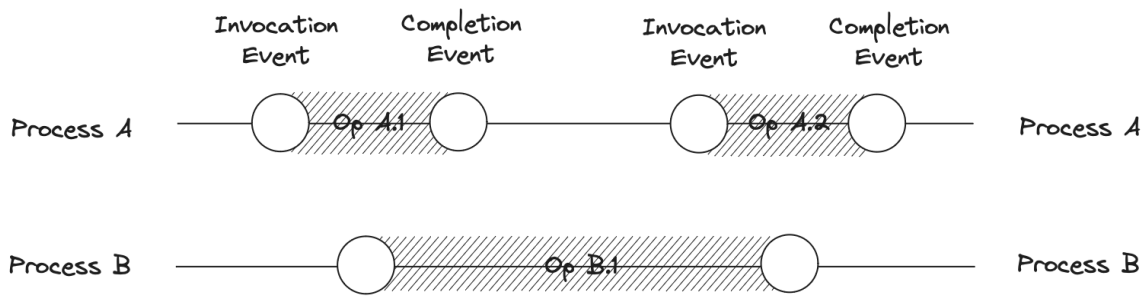


Figure 4. System Model

Linearizability is a real-time guarantee about single operations (one operation at a time) on single objects (one object at a time) that is defined per object type (tailored to the semantics of the object type).

Let's explore a queue and a stack example.

Queue & Stack

Recall from Chapter 1 that correctness is application specific: As a software engineer, you have the ability to define the guarantees of your system and decide what behavior is desirable, tolerable, or intolerable.

What you may consider to be broken, I may consider to be acceptable.

Chapter 1

This idea also applies to object types. Here, we will define two types, a *queue* and a *stack*, both defining an `insert` and a `remove` operation.

Queue

A queue is an object type where the two operations `insert` and `remove` preserve a first-in first-out order on its elements. Here, `insert` is often referred to as `enqueue`, `remove` as `dequeue`.

For a queue, Table 1. shows a valid trace and an invalid trace: When a process enqueues value a, then enqueues value b, we expect a dequeue operation to yield a and a subsequent dequeue operation to yield b.

Table 1. Valid and Invalid Trace

Valid

Invalid

Valid

Invalid

1. insert(a) • OK

2. insert(b) • OK

3. remove • OK(a)

4. remove • OK(b)

1. insert(a) • OK

2. insert(b) • OK

3. remove • OK(b)

4. remove • OK(a)

Stack

A stack is an object type where the two operations `insert` and `remove` preserve a first-in last-out order on its elements. Here, insert is often referred to as push, remove as pop.

For a stack, Table 2. shows a valid trace and an invalid trace: When a process pushes value a, then pushes value b, we expect a pop operation to yield b and a subsequent pop operation to yield a. In contrast to the case of a queue, the valid and invalid traces are reversed.

Table 2. Valid and Invalid Trace

Valid

Invalid

Valid

Invalid

1. insert(a) • OK

2. insert(b) • OK

3. remove • OK(b)

4. remove • OK(a)

1. insert(a) • OK

2. insert(b) • OK

3. remove • OK(a)

4. remove • OK(b)

We define the correct behavior of an object in a single-process, single-object setting. Linearizability extends this by defining a consistency condition that preserves these single-process, single-object semantics across multiple processes interacting with multiple copies of the object.

Definition

A history of operations is either *sequential* or *concurrent*, depending on the relationships between its operations.

A history is said to be sequential if every invocation of an operation is immediately followed by its corresponding completion. In other words, operations do not interleave. In contrast, a history is said to be concurrent if it is not sequential, that is, if the invocation of one operation may occur before the completion of another. In other words, operations interleave.

Table 3. Sequential vs. Concurrent

Sequential

Concurrent

```
1. invoke(op1)
2. complete(op1)
3. invoke(op2)
4. complete(op2)
```

```
1. invoke(op1)
2. invoke(op2)
3. complete(op1)
4. complete(op2)
```

A history H is linearizable if:

- **Sequential Equivalence** There exists a sequential history H' such that H' is equivalent to H, that is, H' and H yield the same results.
- **Real-time Ordering** The ordering of operations in H' respects the real-time ordering of non-concurrent operations in H, that is, any operation that is invoked after a previous operation completes must witness the effects of the completed operation.

Although linearizability is defined on a per object type basis, meaning that it is customized to the semantics of the object type, the core principles of sequential equivalence and real-time ordering remain unchanged.

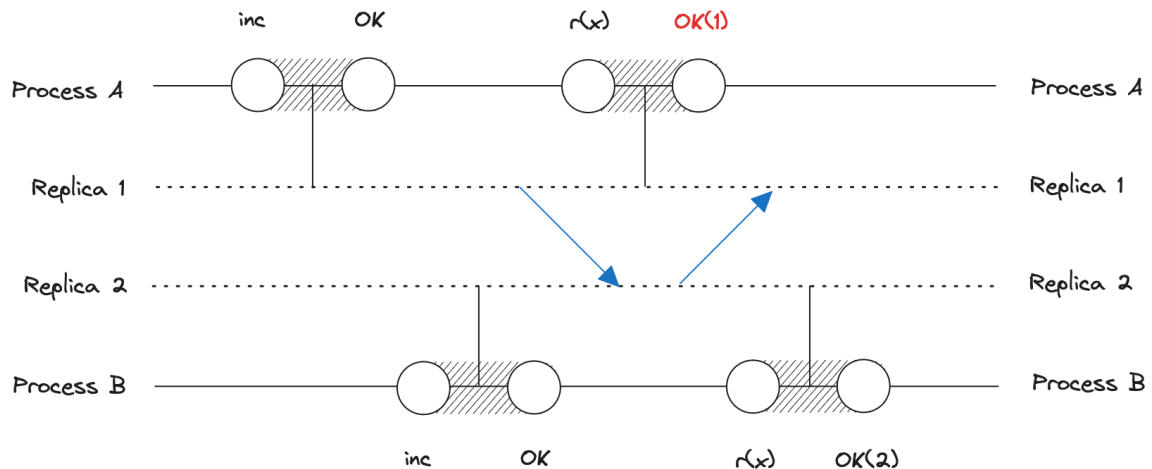


Figure 5. Non-linearizable history of an replicated, distributed increment only counter

Figure 5 illustrates a replicated, distributed increment-only counter starting at 0. The history is not linearizable: If the system was linearizable, the read operation of process A would have returned a value of 2 since the read operation of process A follows the increment operation of Process B. Instead, the system exposes the presence of multiple replicas, including its replication lag. As a result, the read operation of process A returned a stale value of 1.

AHA! Moment • Linearizable History vs. Linearizable System

Note how the term linearizability is used differently when applied to the history of an execution and when applied to a system.

If we say a history is linearizable, we assert that there exists an equivalent sequential history. In this case, the term linearizability denotes a *fact* about a history. If we say a system is linearizable, we assert that every possible history of the system is linearizable. In this case, the term linearizable denotes a *guarantee* about a system.

Implementation

How can we implement a replicated, distributed system that guarantees linearizability? In the next chapter we will look at one of the most popular options: Consensus algorithms.

Conclusion

Linearizability is one of the strongest consistency models. Linearizability ensures that a replicated system appears as if it was not replicated. However, achieving linearizability comes at the cost of availability and latency. This trade-off is exemplified by the most infamous principle in distributed systems: CAP.

CAP

CAP is a framework for understanding the trade-offs between (particular definitions of) **C**onsistency, **A**vailability, and **P**artition Tolerance — three key properties of a replicated, distributed system.

In this section, we will discuss and explore the CAP Theorem in great detail so that next time someone drops the CAP bomb on you, you'll be well prepared to withstand the metaphorical blast.

Introduction

CAP is perhaps the most well-known yet often misunderstood principle in distributed systems. When talking about CAP, we

have to make a difference between the original CAP Conjecture and the subsequent CAP Theorem.

In 2000, Eric Brewer introduced the CAP conjecture during his keynote address “Towards Robust Distributed Systems” at the PODC (Principles of Distributed Computing) conference. Brewer posited that a distributed system cannot achieve all three of the following properties simultaneously:

- **C**onsistency,
- **A**vailability, and
- **P**artition Tolerance.

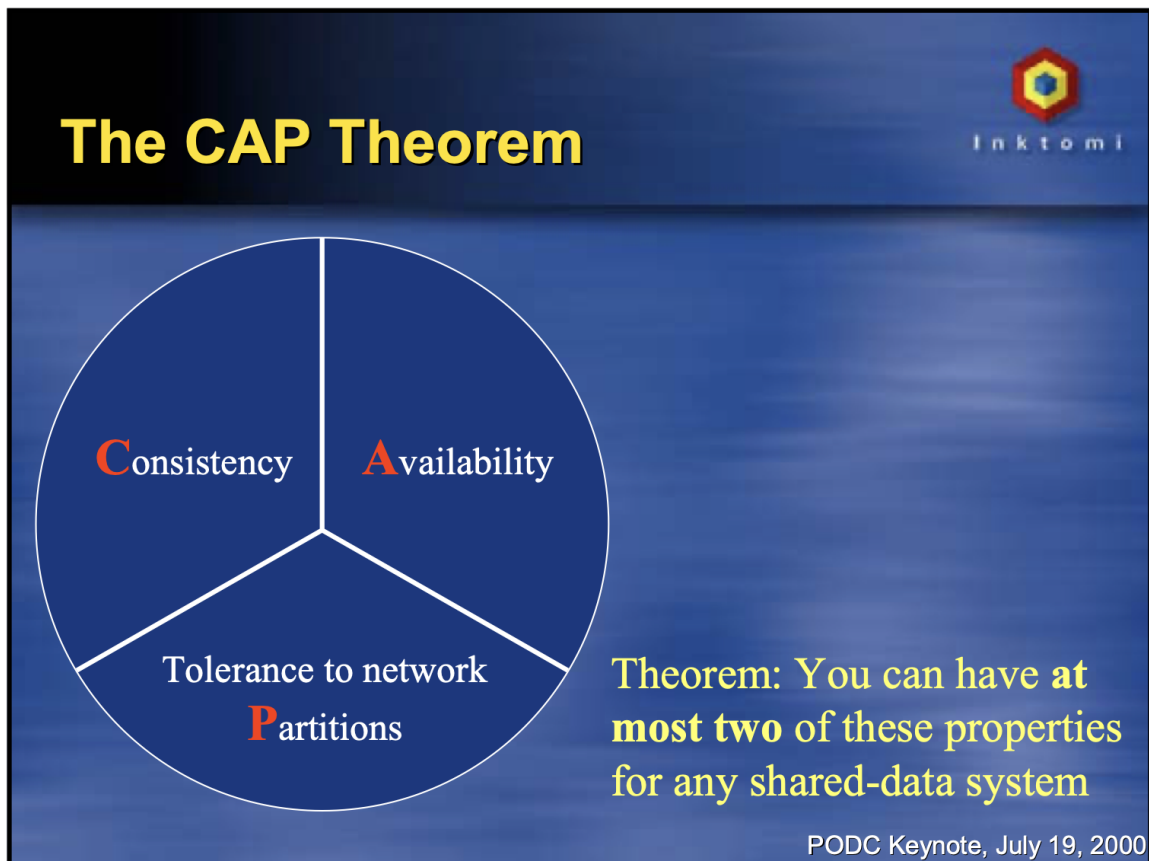


Figure 6. Eric Brewer, PODC Keynote “Towards Robust Distributed Systems”, 2000

While presented as a theorem, at that time, Brewer’s interpretation of CAP was merely conjecture, as Brewer did not

offer formal proof in its support.

In 2002, Seth Gilbert and Nancy Lynch published a formal proof in “Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services,” rendering their interpretation of CAP a theorem.

The CAP Conjecture by Eric Brewer is an attempt to *formulate* the conflict between Consistency and Availability while the CAP Theorem by Seth Gilbert and Nancy Lynch is an attempt to *formalize* the conflict between Consistency and Availability.

AHA! Moment • Pick 2 out of 3

CAP was originally presented as a “Pick 2 out of 3” framework, an interpretation that has since fallen out of favor. The “Pick 2 out of 3” interpretation implies that network partitions are optional, something you can “opt-out” of. However, as discussed in Chapter 2, network partitions are inevitable in a realistic system model. Therefore, the ability to tolerate partitions is a non-negotiable requirement, rather than an optional attribute.

Since you have to account for network partitions, you have to choose between consistency and availability if and when a partition occurs. In other words, the CAP divides the world into **CP** and **AP** systems.

The Claim

The CAP Theorem states that any replicated system can only guarantee *Consistency* or *Availability* under *Partitioning*.

However, the conjecture and the theorem are based on different interpretations (see Table 4).

Both the conjecture as well as the theorem define consistency as a safety property and availability as a liveness property of the system:

- **Consistency.** (*Informally*) Every read request receives a response indicating success and reflecting the value of the most recent write request or a response indicating failure.
- **Availability.** (*Informally*) Every read request eventually receives a response indicating success (not necessarily reflecting the value of the most recent write request).

Additionally, both the conjecture as well as the theorem define a network partition as a failure mode of the underlying system model:

- **Network Partition.** (*Informally*) A network partition divides the network into segments, messages sent from nodes in one segment to nodes in other segments are lost.

Table 4. Conjecture vs. Theorem

	The Conjecture	The Theorem
Consistency	Single Copy Serializability	Linearizability of r/w register

	The Conjecture	The Theorem
Availability	Some node responds timely	Any node responds eventually
Partitioning	Temporary	Permanent

The CAP Theorem

Seth Gilbert and Nancy Lynch published a theorem that provides a formal, mathematical articulation within the specific assumptions and constraints of their system model.

System Model

Gilbert's and Lynch's system model is illustrated in Figure 7. The system consists of three nodes, C, N_1 , and N_2 . Nodes do not have access to clocks and communicate by sending and receiving messages over an asynchronous network.

Let's consider that there are three nodes, C, N_1 , and N_2 . N_1 and N_2 start out with the same value, v_0 . Additionally, let's consider a permanent network partition between N_1 and N_2 . Communication is still possible between C and both N_1 and N_2 , but not between N_1 and N_2 .

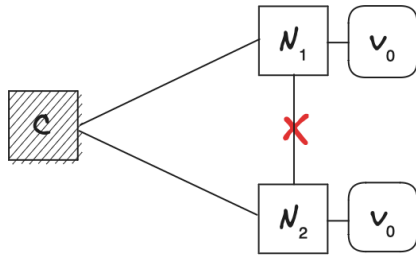


Figure 7. Gilbert's & Lynch's System Model

The Proof

To demonstrate the impossibility of achieving consistency, availability, and partition tolerance simultaneously, Gilbert and Lynch employed a proof by contradiction.

Assume, only for the sake of contradiction, that an algorithm exists allowing the system to be both consistent and available under this network partition.

The Algorithm's Failure

• Step 1

C sends a single write request to N_1 setting the value to v_1 .

```
send(w, v1) rcv(w, v1) send(ack) rcv(ack)
```

• Step 2

C sends a single read request to N_2 .

```
send(r) rcv(r) send(v0) rcv(v0)
```

Under these conditions, if a write occurs on N_1 and a read occurs on N_2 , the read operation will not be able to return the most recent value (v_1), thereby violating the assumed consistency.

Therefore, we reach a contradiction: our initial assumption that this system could be both consistent and available is proven false.

Note

Note that in the case of a permanent partition, no information can flow from one segment to another. Therefore, even the weakest form of consistency is not possible in a system with a permanent partition. A more careful formalization of CAP is required.

Critique

The conjecture and the theorem have key differences with far reaching consequences:

- Brewer's original interpretation of CAP is intuitive and practical, but it is not a theorem.
- Gilbert's and Lynch's subsequent interpretation of CAP is a theorem but not intuitive or practical.

Conclusion

The CAP Theorem has had both positive and negative effects on the distributed system community. On the positive side, CAP has taught us to think about the inherent trade-offs in distributed

systems. However, on the negative side, CAP is sometimes misused as a definitive argument to shut down a conversation, even when CAP may not be applicable to the design challenges at hand.

While many software engineers cite the CAP Theorem to justify their design decisions, a comprehensive understanding reveals that the CAP Theorem applies “in theory” yet may not apply to the design challenges at hand “in practice.”

Eventual Consistency

On the other hand, *Eventual Consistency* stands out as one of the weakest consistency models. Eventual Consistency provides a *convergence guarantee*: eventually all nodes are in the same state.

Werner Vogels, CTO of Amazon Web Services, popularized the term Eventual Consistency to describe the consistency model in Amazon's DynamoDB database system.

To understand the practical implications of eventual consistency, let's explore a real-world example often cited by Werner Vogels — the Amazon shopping cart.

The Shopping Cart

Consider the shopping cart illustrated in Figure 8. This cart is essentially a collection of items, with operations to `add` and `remove` items.

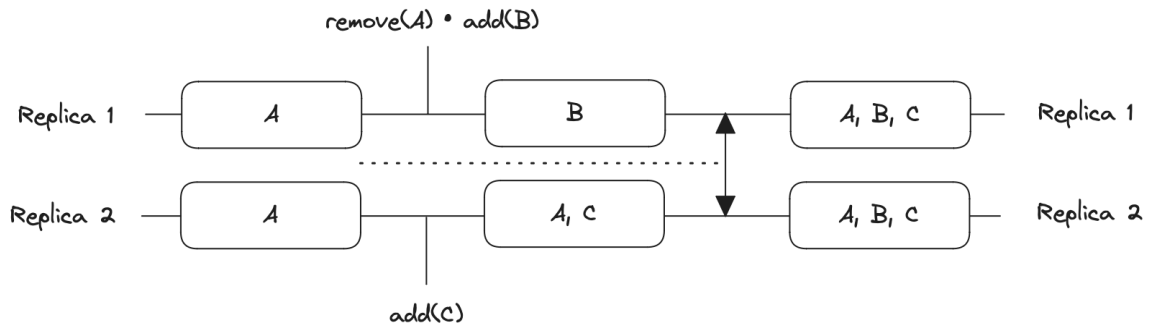


Figure 8. Eventual Consistent Shopping Cart

Initially, we have two identical replicas of the cart, each containing one item: **A**. Should a network partition occur, these replicas lose communication. Consequently, any change in one replica remains unseen by the other. In such a scenario, imagine a client removes item **A** and introduces item **B** to the first replica, while another client adds item **C** to the second. Upon restoration of the network connection, the replicas strive to reach a consistent state.

However, a challenge arises: The same cart has been altered differently across replicas, leading to a conflict requiring a resolution. Finding a resolution falls on the software engineers.

In Amazon's approach to this dilemma, the priority is clear: retain items added by the user over those they've removed. This method ensures no added items vanish, even though occasionally resurrecting removed ones.

The example highlights the trade-offs between availability on one the one hand and a delightful developer experience on the other hand—not to mention a delightful user experience devoid of surprises.

Variants

Eventual Consistency comes in two flavors, Basic Eventual Consistency and Strong Eventual Consistency.

Basic Eventual Consistency

Basic Eventual Consistency is defined in terms of two guarantees, *Eventual Delivery* and *Weak Convergence*.

- **Eventual Delivery**

An update made to one non-faulty replica is eventually delivered to every other non-faulty replica.

- **Weak Convergence**

After no further updates occur, all replicas will eventually converge to the same state.

This version does not constrain the system behavior when updates are continuously being made or the values that read operations may return prior to convergence.

Strong Eventual Consistency

Similarly, Strong Eventual Consistency is defined in terms of two guarantees, *Eventual Delivery* and *Strong Convergence*.

- **Eventual Delivery**

As in Basic Eventual Consistency.

- **Strong Convergence**

Any two replicas that have received the same set of updates are in the same state.

This version constrains the system's behavior such that two replicas will be in the same state if they have seen the same updates.

Implementation

Eventual Consistency necessitates the resolution of conflicts since clients can modify any replica at any time, even when the replicas are not in communication. So how are these concurrent updates reconciled?

Application-Specific Reconciliation

Historically, reconciliation has been managed at the application level. For instance, in the Amazon shopping cart example, the application gives precedence to newly added items over removed items during conflict resolution.

Algorithmic Reconciliation

Recently, two major families of algorithms have gained prominence in automated conflict resolution: Conflict-Free Replicated Data Types (CRDTs).

Conflict-Free Replicated Data Types (CRDTs): A CRDT is a data structure designed to allow independent and concurrent updates across multiple replicas, while ensuring eventual state convergence.

Operation-based CRDTs, also known as commutative CRDTs, replicate individual operations across replicas; replicas receive the updates and *apply* them locally. To guarantee convergence, the system must ensure that operations are not lost and each

operation is applied effectively once as well as the apply function is commutative.

State-based CRDTs, also known as convergent CRDTs, replicate the entire state across replicas; replicas receive updates and *merge* them locally. To guarantee convergence, the merge function is associative, commutative, idempotent.

A straightforward example of a CRDT is the Grow-Only Set, a set with one operation `add` :

Operation Based Grow-Only Set

```
// apply adds an element to  
// the set
```

```
on init do  
  set := { }  
end
```

```
on recv (add x) do  
  set := set u { x }  
end
```

State Based Grow-Only Set

```
// merge takes the union of  
// two sets
```

```
on init do  
  set := { }  
end
```

```
on recv (set') do  
  set := set u set'  
end
```

Today, CRDTs are an active area of research. Today, many modern distributed systems and databases, e.g. [Riak](#), incorporate CRDT principles to achieve high availability and low latency.

Conclusion

In this chapter, we explored Linearizability as one of the strongest consistency models and Eventual Consistency as one of the weakest consistency models.

Linearizability offers a delightful developer experience, akin to working with a single-node system; however, this comes at the cost of coordination, which results in lower availability and higher latency. In contrast, Eventual Consistency optimizes for availability and low latency but at the expense of having to navigate the difficulties of a replicated, distributed system like conflicts and temporary inconsistencies.

The CAP theorem provides a succinct formulation of the trade-offs between consistency and availability, serving as a potent reminder that there are no “perfect” solutions in distributed systems — we have to make choices based on the specific needs of each application.

Outlook

In the next chapter, we will explore distributed consensus, its application, and a popular implementation, the Raft algorithm.

DISTRIBUTED CONSENSUS



In the previous chapters, we explored the concepts of partitioning and replication, methods employed to overcome the limits of scalability and reliability of single nodes. As we venture further into the complexities of distributed systems, we arrive at a foundational abstraction: Distributed Consensus.

In this chapter, we will explore distributed consensus, state machine replication, and the Raft consensus protocol.

Objectives:

- Understand Consensus
- Understand State Machine Replication
- Understand the Raft Consensus Protocol

Motivation

Distributed Consensus lies at the heart of distributed systems. Long believed impossible to achieve, distributed consensus serves as a cornerstone for building reliable and scalable distributed systems.

Distributed consensus allows a group of redundant processes to advance in lockstep, to act as one. This allows that, at any time, some processes in the group can compensate for the failure of others.

Failing to reach consensus is often considered catastrophic for the application at hand. Did the transaction commit or abort? Did operation a happen before operation b? Was the lock acquired by component 1 or component 2? Any disagreement on these questions quickly results in incorrect behavior.

Therefore, the consensus problem has garnered outsized interest in both the theoretical and practical realms of software engineering.

AHA! Moment • The FLP Impossibility

The FLP Impossibility theorem, credited to its authors Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson, asserts that in an asynchronous distributed system without clocks, it is impossible to guarantee consensus under *all* circumstances, even if only a single node can fail.

Given the dire result of the FLP Impossibility theorem, one might wonder how we can realistically solve the consensus problem or expect consensus algorithms to succeed.

The FLP Impossibility theorem principally addresses the inherent limitations of ensuring liveness in asynchronous distributed systems without clocks. However, it does not rule out the possibility of achieving consensus under more constrained system models. For example, in an asynchronous system model with clocks, consensus is possible.

Introduction

At first glance, the concept of distributed consensus appears straightforward: *Distributed Consensus* requires a set of concurrent, communicating components—in this context often called processes—that communicate by sending and receiving messages over a network *to agree on a single value*.

However, while the problem statement of distributed consensus is simple, finding a solution in a realistic system model is notoriously difficult.

Consensus is trivial to achieve in a *theoretical system model*, where components are not subject to failure and the network delivers messages exactly once and in order. However, consensus is difficult to achieve in a *realistic system model*, where components may fail and the network may reorder, delay, lose, or duplicate messages.

In Chapter 6, "Distributed Transactions," we encountered a consensus problem in the form of Atomic Commitment: A set of resource managers must agree on whether to commit or abort a transaction. When we discussed the Two-Phase Commit Protocol and its behavior in the presence of failure, we realized that a failure of one component, the transaction coordinator, may block the protocol, guaranteeing safety but sacrificing liveness.

Consensus algorithms guarantee the safety of a system and aim to provide—but do not guarantee—the liveness of the system. No one component failure shall bring the system to a halt.

Definition & System Model

Formally, there is a set of processes `Processes` and a set of values `Values`. A process `p` may fail. Here, a failure is a crash failure, that is, a process halts.

A processes `p` may propose a value `v`, represented as `Propose(p, v)` and a process may decide on a value `Decide(p, v)`.

A consensus algorithm is any algorithm that ensures the safety properties of Validity, Integrity, Agreement, and the liveness property of Termination.

Table 1. Set Theory

Symbol	Meaning
$\forall$	For all
$\exists$	There exists

- Safety Properties

- **Validity**

If a process decides on value v , then value v was proposed by some process.

$$\forall p \in \text{Processes}, \exists q \in \text{Processes}, \forall v \in \text{Values}: \\ \text{Decide}(p, v) \Rightarrow \text{Propose}(q, v)$$

- **Integrity**

No process decides on a value v more than once.

$$\forall p \in \text{Processes}, \forall v, v' \in \text{Values}: \\ \text{Decide}(p, v) \wedge \text{Decide}(p, v') \Rightarrow v = v'$$

- **Agreement**

No two correct processes decide on different values.

$$\forall p, q \in \text{Processes}, \forall v, v' \in \text{Values}: \\ \text{Decide}(p, v) \wedge \text{Decide}(q, v') \Rightarrow v = v'$$

- Liveness Properties

- **Termination**

Every non-failed process eventually decides on a value v .

$$\forall p \in \{ p \in \text{Processes} \mid \text{alive}(p) \}, \exists v \in \text{Values}: \\ \Diamond \Box (\text{Decide}(p, v))$$

Agreement and Validity capture our intuition of consensus: All processes agree on the same value—and do not go back on their words.

The question arises, why is distributed consensus so significant in the context of distributed systems?

State Machine Replication

Distributed consensus forms the foundation for *State Machine Replication*, a technique that *mechanically* transforms processes into fault-tolerant groups of processes. Consensus algorithms enable processes to advance in lockstep, ensuring fault tolerance by allowing the group to compensate for any failed member.

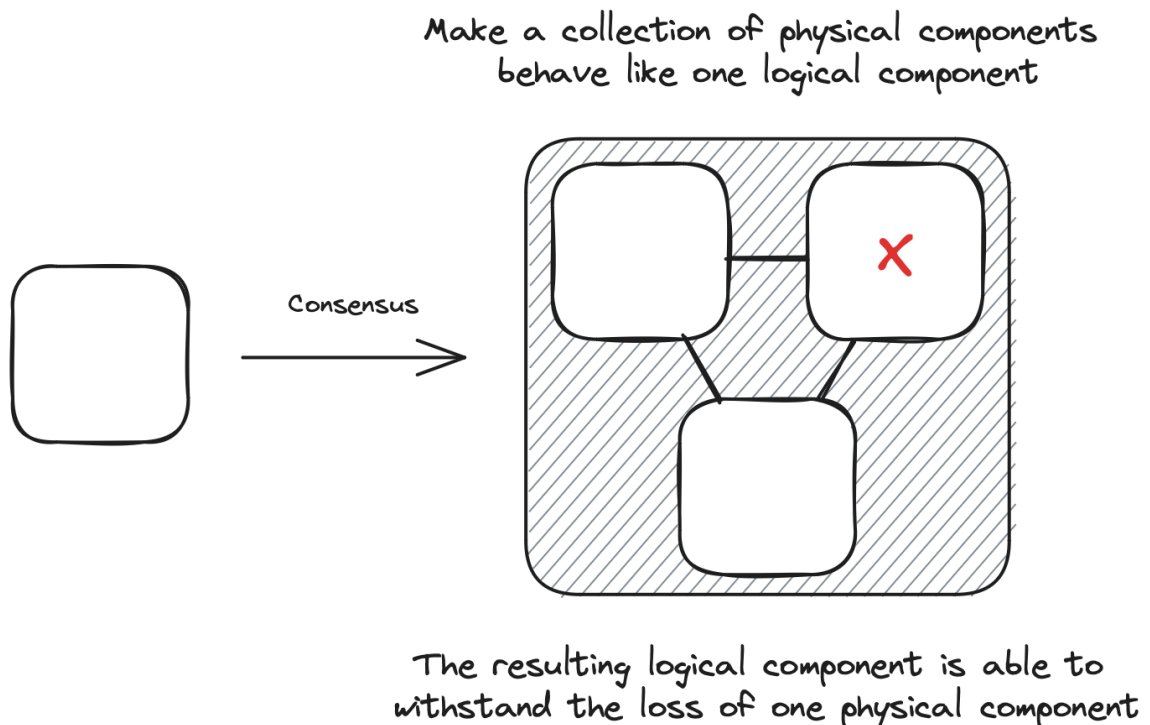


Figure 1. Transforming a process into a fault-tolerant process

State Machine Replication operates on a simple principle: If two identical, deterministic processes begin in the same state and receive the same inputs in the same order, they will produce the same output and reach the same state.

In other terms, the challenge of ensuring that multiple processes progress in lockstep can be reduced to the challenge of reaching consensus on the log that supplies their inputs.

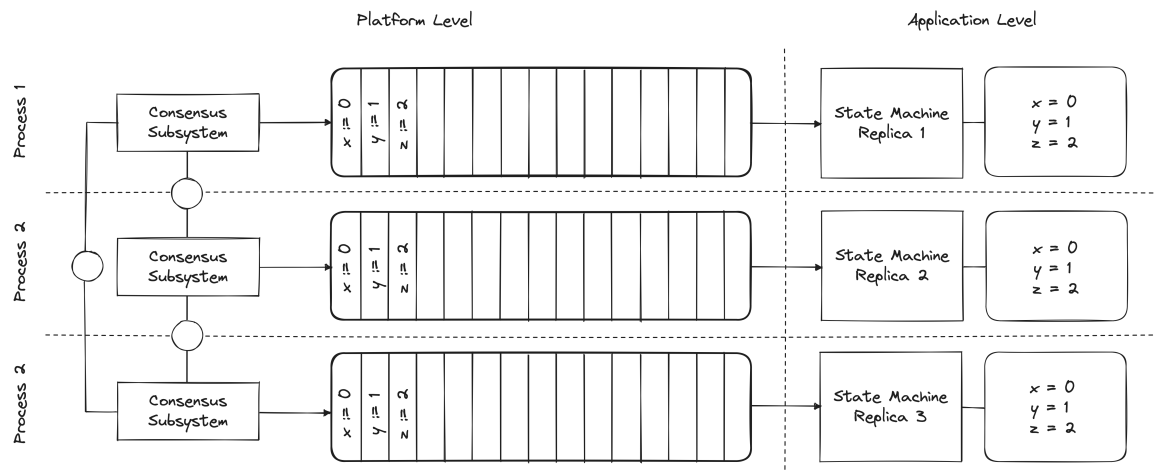


Figure 2. State Machine Replication in Action

Figure 2. illustrates State Machine Replication to implement a replicated key-value store. A consensus protocol ensures that every replica receives the same commands in the same order, resulting in a reliable key-value store that can compensate for the crash of a node.

History

Leslie Lamport, who was awarded the Turing Award in 2013 for his seminal contributions to the theory and practice of concurrent and distributed systems, initially set out to prove that

achieving fault-tolerant consensus in a distributed system was *impossible*.

However, in a remarkable twist of fate, instead of proving that fault-tolerant consensus is impossible, in 1989, Lamport devised and published what is often considered the first fault-tolerant consensus algorithm: the *Paxos Protocol*.

The protocol presents consensus as a single decision, ensuring that a set of write-once registers, the replicas, all get set to the same value, even in the face of failures and communication over an unreliable network.

While groundbreaking, Paxos has limitations. Leslie Lamport aptly demonstrated the fundamentals of consensus for a single decision, however, real-world distributed systems often require consistency across a series of decisions, not just isolated instances.

Multi-Paxos builds upon the principles of Paxos to achieve consensus on a series of decisions, rendering Multi-Paxos suitable of implementing State Machine Replication.

AHA! Moment • Viewstamped Replication

In 1988, Brian Oki and Barbara Liskov published the Viewstamped Replication algorithm in their paper *Viewstamped Replication: A New Primary Copy Method to Support Highly-Available Distributed Systems*.

Although Viewstamped Replication did not garner the same level of recognition and fame as Paxos and, the paper did not explicitly use the term "consensus," the algorithm presented addressed the same problem. It can, therefore, be regarded as

the first published algorithm that solved the problem of distributed consensus.

Implementing Consensus

Many well-known consensus algorithms, such as Multi-Paxos, Raft, or Viewstamped Replication, combine the idea of a *leader* with the idea of a *quorum*.

Leader-based Consensus

Achieving consensus is straightforward in a theoretical system model, where components never fail, and the network reliably delivers messages once and in order.

A simple approach is to appoint a process known as the Benevolent Dictator For Life (BDFL), where a single individual has the final say in decisions. This designated BDFL processes each request, makes decisions, and broadcasts these decisions to all other processes. In this ideal environment, where processes cannot fail and messages cannot get lost, duplicated, or reordered, consensus is guaranteed.

In contrast, achieving consensus is challenging in a realistic system model, where components may fail, and the network may lose, duplicate, or reorder messages.

Simply appointing a BDFL is untenable: If the BDFL fails, the entire system is unable to make any decisions. Furthermore, network inconsistencies—even with a functioning BDFL— a lost

message could prevent decisions from being communicated to all processes, resulting in system inconsistencies.

We need a more robust approach.

Quorum-based Consensus

A common approach to implementing consensus involves quorum-based algorithms. In a quorum-based algorithm, consensus is reached only when a majority of nodes, known as a quorum, acknowledge a given decision. These algorithms operate on the principle of majority rule to ensure that decisions are both made by and known to a majority of processes.

In the context of distributed systems, a quorum is defined as a subset of processes where the cardinality of the subset is strictly greater than half of the total number of processes, $Q > N/2$. **This ensures that any two quorums will always have at least one process in common**, a property critical for maintaining system consistency.

In practical terms, if we require to compensate for f failed processes, we need at least $f+1$ non-failed processes to establish a quorum. Summing these, we get $N = f + (f+1)$, which simplifies to $N = 2*f+1$, the often cited formula that represents the minimum total processes needed to tolerate f failures and still maintain the ability to achieve consensus.

The overlap of quorums serves to avoid split-brain conditions. In a split-brain scenario, two or more subsets of the nodes in a distributed system operate simultaneously but independently, leading to inconsistency. The quorum intersection property guarantees that at least one node will be aware of the decisions

made in either of the overlapping quorums, thus serving as a single point of coordination to prevent split-brain conditions.

Possible Quorums for p_1, p_2, p_3

$\{ p_1, p_2, p_3 \}$

$\{ p_1, p_2 \} \{ p_3 \}$

$\{ p_1, p_3 \} \{ p_2 \}$

$\{ p_2, p_3 \} \{ p_1 \}$

Combining Leader & Quorum

In algorithms like Multi-Paxos, Raft, and Viewstamped Replication, the leader, elected through a quorum, proposes values. The leader awaits acknowledgments from a quorum of nodes before committing a value, ensuring that a majority is aware of and agrees on the proposed value.

This approach combines the decisiveness of a leader with the reliability of a quorum, addressing the challenges associated with unreliable processes and unreliable networks.

Raft

In this section, we explore the [Raft Consensus Algorithm](#), popular for its emphasis on understandability. However, despite its design focus on understandability, Raft remains a complex algorithm. To deepen our understanding of Raft, we will tackle three *Raft Puzzles* at the end of this section.

Raft is a consensus protocol designed for managing replicated logs across a cluster of nodes, forming the foundation of state machine replication. The protocol relies on a leader to orchestrate message sequencing (deciding the order of log entries) and a quorum to ensure message commitment (deciding the inclusion of entries into the log).

This section will discuss the Raft consensus algorithm as outlined in the original paper. Variants of Raft exist, and some of these variations may differ from the protocol discussed here and the puzzles presented at the end of this section.

The Log

Raft's objective is to guarantee log consistency across all nodes in the cluster. For any pair of nodes n_1 and n_2 , their logs, represented by l_1 and l_2 , are considered consistent if one log is a subsequence of the other up to the commit index.

Log consistency is a prerequisite for *State Machine Safety*, Raft's central safety guarantee. The State Machine Safety guarantee

ensures that committed log entries will never be lost and may safely be applied to the application-level state machine.

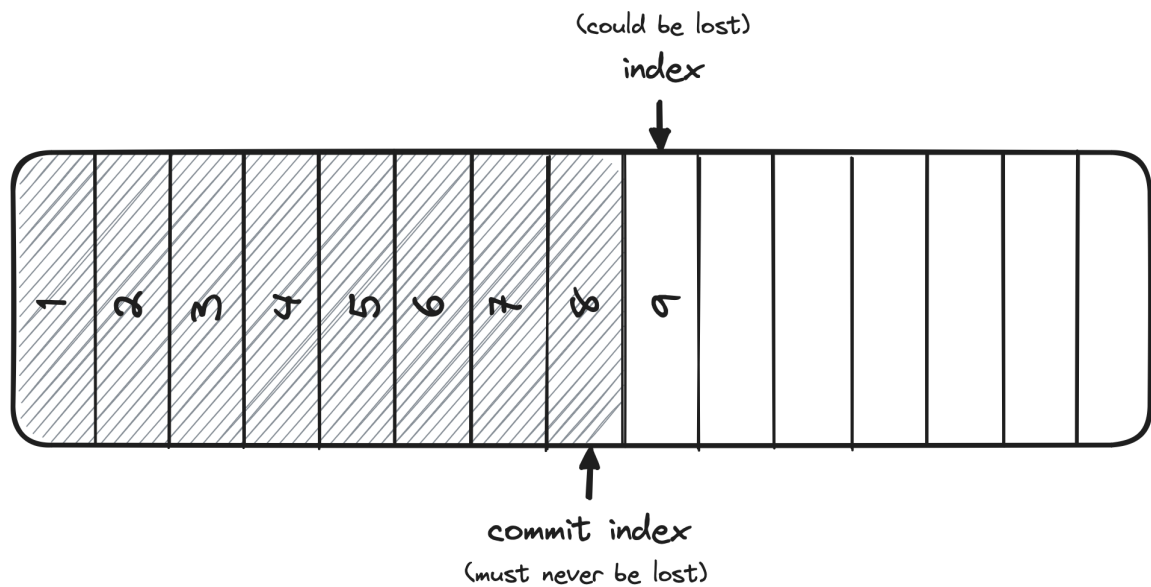


Figure 3. The Log

Raft uses the logs themselves to guarantee log consistency and State Machine Safety. Specifically, in case of a leader election, only candidates with the most up-to-date logs are eligible to become the leader.

AHA! Moment • Application Level vs. Platform Level

In the Raft consensus algorithm, there are essentially two layers of operation: the platform level and the application level. The platform level is tasked with running the core Raft protocol, which includes activities like leader election, log replication, and committing entries to the log. Once a log entry is committed, the platform level forwards or hands off this entry to the application level. The application level, in turn, possesses its own state machine, which applies the log entry to the application's state. In

this way, Raft ensures not just consensus at the platform level but also consistent application level behavior.

Terms

In Raft, time is delineated into monotonically increasing terms, creating a logical clock central for the protocol’s correctness. Each term consists of a *leader election phase* and a *log replication phase*:

- **Leader Election** (Request Vote Protocol)

The leader election sub-algorithm ensures that one and only one leader is active at any given time and a new leader is elected swiftly in the event of a leader failure.

Messages Exchanged	
Request	Response
Request Vote Request	Request Vote Response

- **Log Replication** (Append Entries Protocol)

The log replication sub-algorithm handles client requests and manages the propagation of log entries across the cluster.

Messages Exchanged

Request

Response

Append Entries Request

Append Entries Response

In summary, per term, first, Raft elects a leader. Then, Raft delegates authority and responsibility for managing the replicated log to the leader. The leader accepts log entries from clients, replicates the log entries to all other nodes, and determines when nodes may apply log entries to their local state machines.

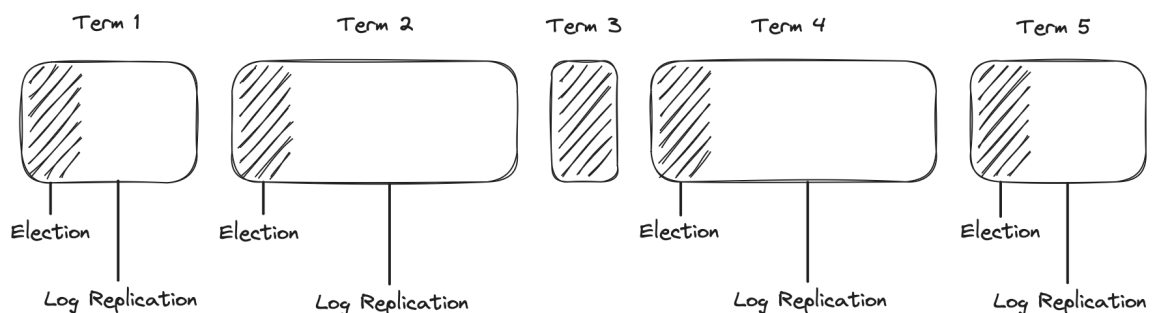


Figure 4. Terms, Election & Replication

AHA! Moment • Terms as Fencing Tokens

While we often assert that Raft guarantees that there is only one leader, we should clarify this statement. Raft does not guarantee that only one node *believes* to be the leader at any given time. However, Raft does guarantee that only one node may *act* as the leader.

If two nodes believe themselves to be the leader, a crucial distinction lies in their term numbers. A previous leader holds a term number lower than the current leader.

The term number acts as a fencing token. Nodes compare the term number of incoming messages with their own current term. Messages with lower term numbers are promptly rejected, preserving the guarantee that only one node may act as a leader at one point in time.

Term-based coordination is not only a technical detail but a fundamental cornerstone of Raft's ability to guarantee correctness.

Leader Election Protocol

In the Raft consensus algorithm, the Leader Election Protocol is responsible for ensuring that one leader is active at any given time and a new leader is elected in the event of a leader failure. We will explore the Leader Election Protocol from the point of view of one node. A Raft node can be in one of three states: Follower, Candidate, or Leader.

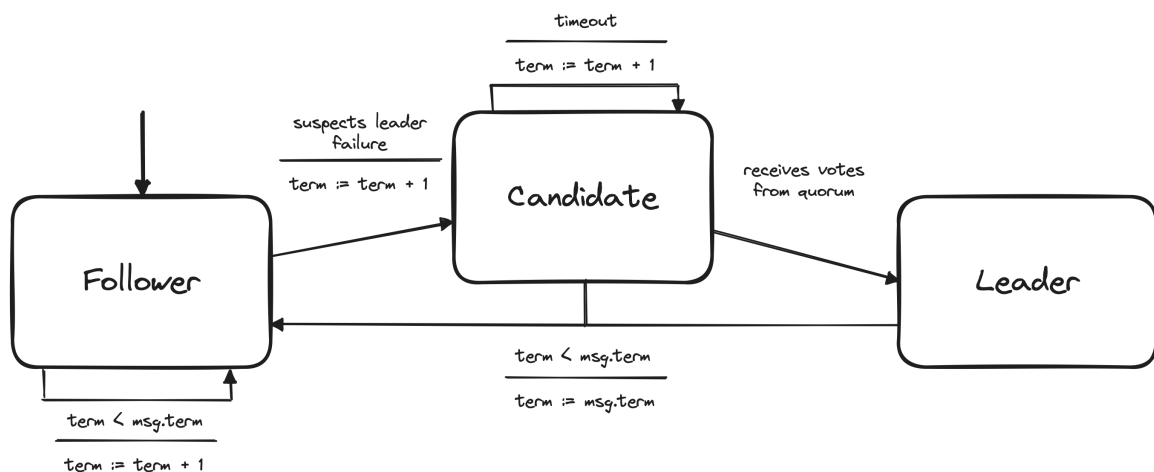


Figure 5. Raft node state transitions

As a Follower

When a Raft node boots, either for the very first time or after recovering from a crash, it starts as a Follower. In this state, the Follower listens for messages from the Leader.

Each Follower has an election timer set to a random timeout value. This timer resets whenever the Follower receives a message. When the timer expires, the Follower suspects that the Leader has failed and initiates an election.

As a Candidate

To initiate an election, a Follower increments its current term and transitions to the Candidate state. The Candidate votes for itself and issues `RequestVote` requests to each other node in the cluster.

Three possible outcomes can occur during the election:

- **Quorum Vote**

If the Candidate receives votes from a quorum of the nodes in the cluster, including itself, it becomes the Leader for the term.

- **Split Vote**

If the Candidate does not receive votes from a quorum of nodes in the cluster during the election, it does not become the Leader for the term. Instead, the candidate initiates a new election, and the process repeats until the candidate becomes the Leader or learns of a new term and Leader, transitioning back to being a Follower.

- **"Demotion"**

If the Candidate learns of a new term and Leader, it transitions back to being a Follower.

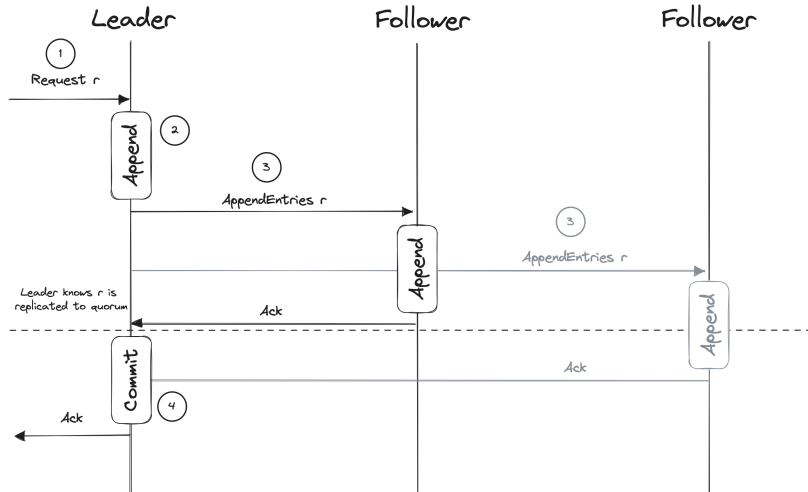
As a Leader

When a Raft node believes it is the Leader and then learns about a new term and a new Leader, it transitions back to being a Follower.

Log Replication

In the Raft consensus algorithm, the Log Replication Protocol is responsible for handling client requests and manages the propagation of log entries across the cluster. We will explore the Log Replication from the point of view of one node.

1. **Accept.** The leader accepts a new log entry from a client.
2. **Append.** The leader appends the new log entry to its own log.
3. **Propagate.** The leader propagates the log entry to other nodes in the cluster via `Append Entries` requests.
4. **Commit.** Followers receive the `Append Entries` request, append the entries to their logs, and send an acknowledgement response to the leader. When the leader has received acknowledgement responses from a quorum (including itself), the leader advances the commit index.



State Machine Safety

The key safety property for Raft is *State Machine Safety* : If a process has applied a log entry at a given index to its state machine, no other process will ever apply a different log entry for the same index.

Raft guarantees State Machine Safety by placing restrictions on which process may be elected leader: Only a candidate with the most up-to-date log may be elected leader.

Raft determines which of two logs is more up-to-date by comparing the index and term of the last entries in the logs:

- If the logs have different terms for their last entries, then the log with the later term is considered more up-to-date.
- If the logs have the same term for their last entries, then the longer log is considered more up-to-date.

Given these rules, if a quorum of processes have a log entry, no candidate without that log entry can become the leader.

Consequently, once an entry is added to the logs of a quorum,

the entry will always be present in the logs of any future leaders for that term or any higher term. This ensures that once a log entry is committed, the entry will never be lost or overwritten.

Puzzle #3 illustrates this section.

Raft Puzzles

In this final section, we will deepen our understanding of the Raft consensus protocol by exploring three Raft Puzzles. Notably, grasping the importance of the third puzzle will elevate your comprehension of Raft to that of a true Raft Wizard. ✨

In each puzzle, we assume a Raft cluster with three nodes, the cluster is on its third term.

AHA! Moment • Thinking in Algorithms

Regardless how much you study the brick, it will not help you in understanding the building.

Personally, I enjoy thinking in systems, but I do not enjoy thinking in algorithms. For me, thinking in algorithms is akin to being so focused on the brushstrokes that I fail to see the painting.

Thinking in distributed system is not only about understanding the algorithms a system uses. It is also about understanding how these algorithms contribute to the system's overall behavior. Many systemic behaviors are implicit consequences of an

algorithm and its interaction with the broader system, which may not be apparent solely from studying the algorithm itself.

Puzzle III is a great example.

Puzzle I

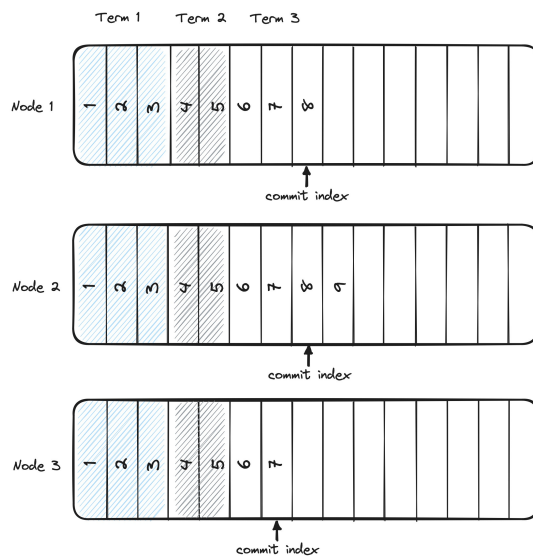


Figure 7. Puzzle I

Figure 7. illustrates the current state of the cluster. Which node is the leader for term 3 and what indicates its leadership?

Answer

Node 2 is the leader for term 3. This can be inferred from the fact that the leader is responsible for accepting client requests, adding them to its log, and then propagating the request to the followers. In this case, Node 2's log is more advanced or later term, indicating that it must be the leader for term 3.

Puzzle II

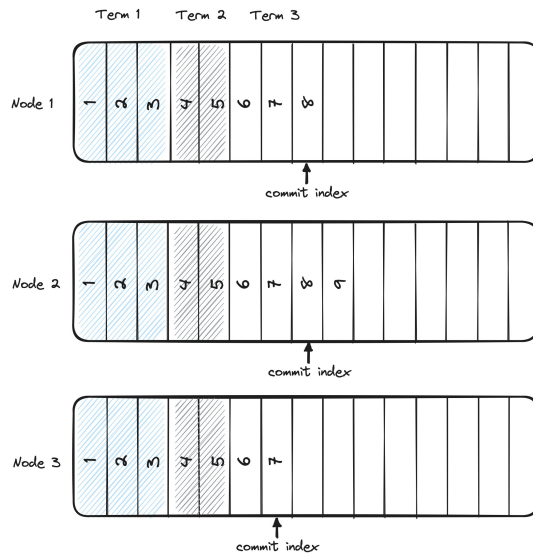


Figure 8. Puzzle II

Figure 8. illustrates the current state of the cluster. Node 2 is the leader in term 3 and has successfully accepted client request #9, which has been added to its log. However, node 2 has not yet propagated this request to any follower. Is there a possibility that request #9 could be lost?

Answer

Yes. If Node 2 loses leadership for any reason (e.g. node 2 crashes), the new leader in term 4 does not have knowledge of request #9. Consequently, if leadership changes, request #9 would be lost. However, losing request #9 does not violate State Machine Safety as the request has not been committed yet.

Puzzle III

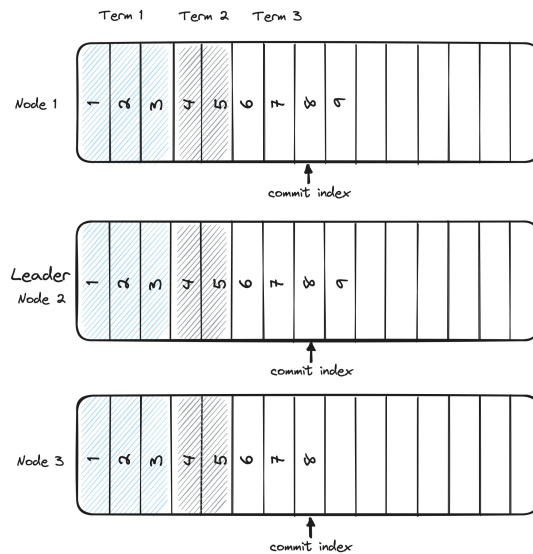


Figure 9. Puzzle III

Figure 9. illustrates the current state of the cluster. Node 2 is the leader in term 3 and has successfully accepted client request #9, which has been added to its log. Node 2 has propagated this request to node 1, which has also been added to its log. Node 1 has sent an acknowledgment, but node 2 has not received the acknowledgment yet and has not advanced its commit index. Node 2 crashes. Is there a possibility that request #9 could be lost?

Answer

No. Although losing request #9 does not violate State Machine Safety as the request has not been committed yet, *request #9 must not be lost.*

To understand why, we must assume the point of view of node 1: Node 1 has sent an acknowledgment and cannot know if node 2 crashed before or after advancing its commit index and sending an acknowledgment to the client. Therefore, in order to guarantee State Machine Safety in all scenarios, node 1 must assume request #9 has been committed.

When electing a new leader, only node 1 will be eligible to assume the leadership role as node 1 has the most up-to-date log. After assuming leadership, node 1 will propagate the missing entries to node 3 and advance its commit index, including request #9.

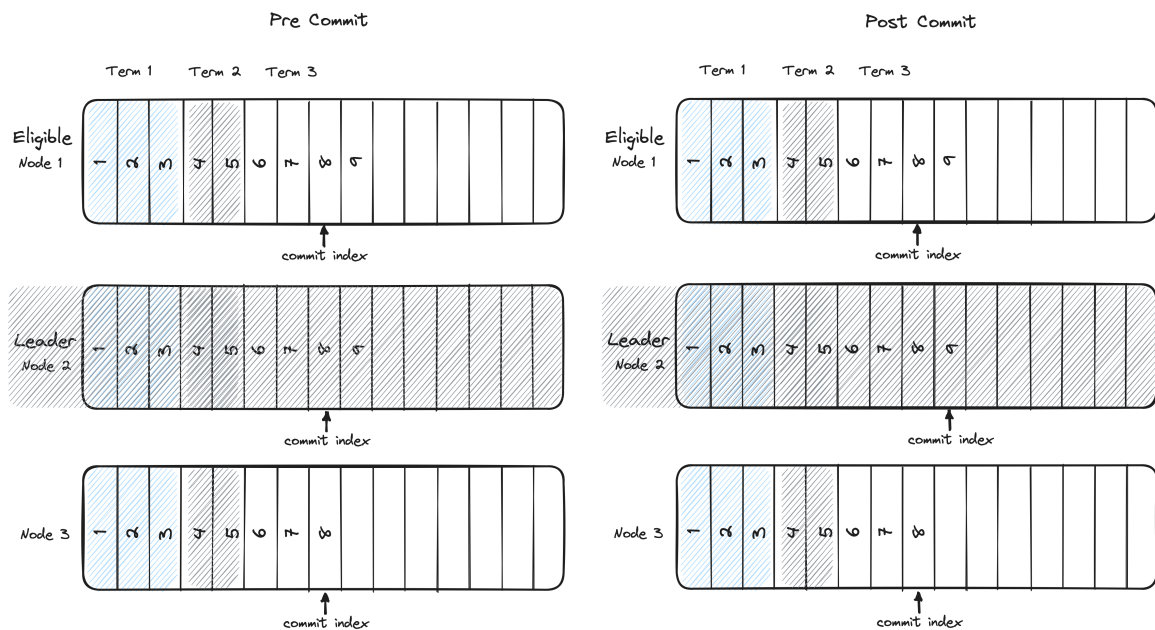


Figure 10. Left: Leader crashes before commit. Right: Leader crashes after commit.

Conclusion

In this chapter, we explored distributed consensus, state machine replication, and the Raft consensus protocol. Distributed consensus lies at the heart of distributed systems: Distributed consensus allows a group of redundant processes to advance in lockstep, to act as one, also known as state machine replication. The Raft protocol is a popular consensus protocol, often praised for its emphasis on understandability.

However, Distributed consensus remains a foundational challenge and Raft remains a complex algorithm.

Outlook

In the previous chapters, we explored the fundamental concepts of correct, scalable, and reliable distributed systems. In the next chapter, we will explore an emerging concept in distributed systems, Durable Executions.

DURABLE EXECUTIONS



In the previous chapters, we explored the fundamental concepts of correct, scalable, and reliable distributed systems. In this chapter, we will explore an emerging concept in distributed systems, Durable Executions.

Durable Executions are to distributed systems what transactions are to databases: An abstraction concealing the possibility of failure.

Objectives:

- Understand Short Running vs. Long Running Processes
- Understand Failure-Free Definitions
- Understand Failure-Tolerant Executions
- Understand Sagas vs. Durable Executions

In the presence of partial failure, even the most basic rules for reasoning about computations do not hold.

- An Equational Theory for Transactions

Motivation

Imagine a user registering for a streaming platform, for example, for video or music streaming. During the registration process, the platform handles the user's credit card payment and then grants access to its content library. Listing 1 displays the steps involved in the signup function.

```
async function signup(user) {  
  const account = await Account.create({  
    ...  
  });  
  
  // 💀 Potential Crash-Stop  
  
  const charge = await Payment.create({  
    ...  
  });  
  
}
```

At first glance, the function may appear to be fine. However, upon closer inspection, we notice a problem: the function only handles the *Happy Path*, naively ignoring the possibility of failure. If the function crashes after charging the credit card but before updating the database, that is, if the function executes partially, the user will be charged but will not have access.

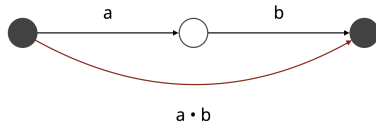


Figure 1. The sequential composition of two atomic actions is not itself atomic

We ran into a fundamental problem in software engineering: The sequential composition of two atomic actions is not itself atomic, leaving us vulnerable to crash failure, resulting in a partial execution.

```
t | < ≡ prefix t
```

Ideally, the execution of a process is failure-transparent, meaning that a failure-free execution is indistinguishable from a failed execution that was subsequently recovered.

```
t | < ≡ t or nothing
```

AHA! Moment • Atomicity

There are two different flavors of *Atomicity*, *Concurrency Atomicity* and *Failure Atomicity*:

- **Concurrency Atomicity.** A process is failure atomic if the process executes observably equivalent to *uninterrupted*. Intermediary states are **not** observable by other processes. (**Note** In the world of databases, this property is referred to as isolation).
- **Failure Atomicity.** A process is failure atomic if the process executes observably equivalent to *all-or-nothing*. However,

intermediary states are observable by other processes.

While concurrency atomicity is often *desirable*, failure atomicity is often *required* to transition the system from a correct state to a correct state. So in this chapter, we will use the term *atomic* synonymous to *failure atomic*

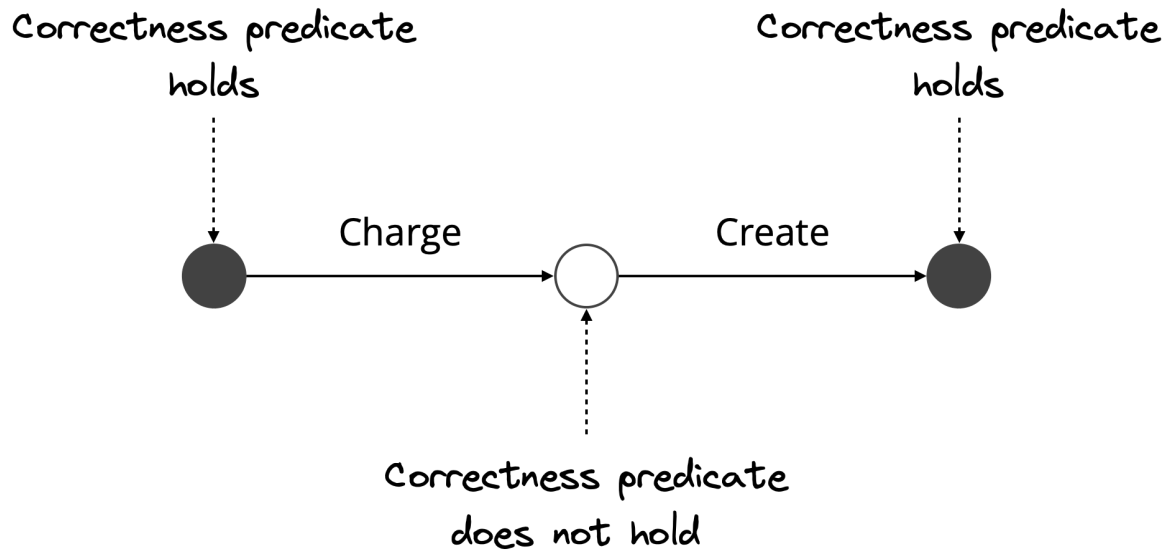


Figure 2. State transition from correct state to correct state via an incorrect intermediary state

Failure Transparency

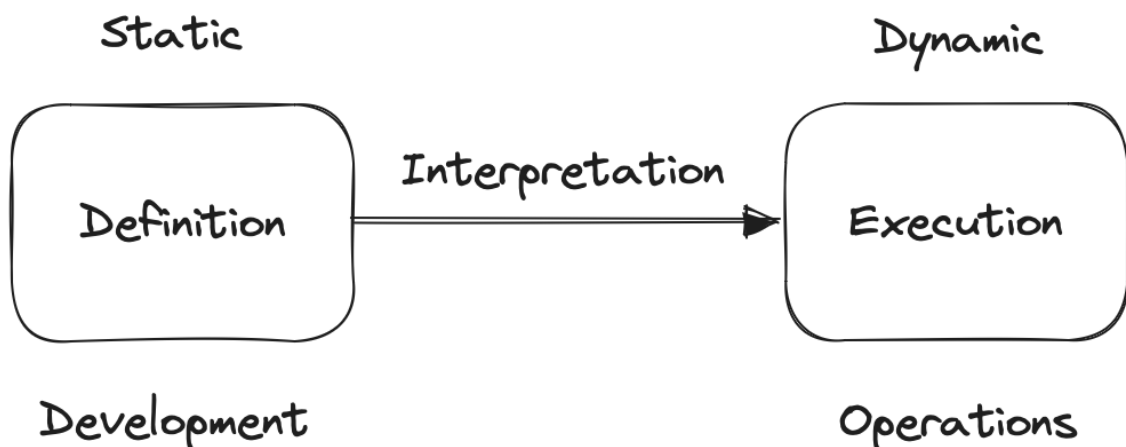
Recovery of a process is failure-transparent if there exists a sequence of events produced by a failure-free execution that would result in a sequence of events *equivalent* to the sequence of events produced by a failed, recovered, and continued execution. In other words, a transient or intermittent failure cannot prevent an execution to run to completion (Of course, a permanent failure can always prevent an execution to run to completion).

System Model

Although the programmer's activity ends when [they have] constructed a correct [definition], the[execution] taking place under control of [their definition] is the true subject matter of[their] activity, for it is this [execution] that has to accomplish the desired effect.

E. W. Dijkstra

In our day-to-day conversations, we often blur the lines between definition (the code) and execution (the running code). For instance, when talking about a 'function', we don't always specify whether we mean the 'function definition' or the 'function execution'. However, in order to have clear mental models, it is important to distinguish between the two. For example, being failure-aware or failure-agnostic is related to definitions, while failure-free, failure-tolerant, or failure-transparent is associated with executions.



Process Definition

In this chapter, we define a process definition P as a sequence of steps A , B , etc, where each step is a failure atomic action.

$$P = A \cdot P \mid \varepsilon$$

- $A \cdot P$ represents a process that performs an atomic action A and then behaves as its successor process.
- ε represents a process that terminates, essentially only a successful termination.

Process Execution

For a process definition P , we define a process execution as the trace t where each step A of P is an observable event a of t .

We assume Crash-Stop failures, that is, a process execution may stop at an arbitrary moment in time.

$$P = A \cdot B \cdot \varepsilon$$

$$t = \times \mid a \cdot \times \mid a \cdot b \cdot \checkmark$$

- $\checkmark$ represents a successful execution.
- $\times$ represents a crash failure.

Trivially, if a process definition consists of a *single* step, a process execution “inherits” the failure atomicity of its step (Figure 4).

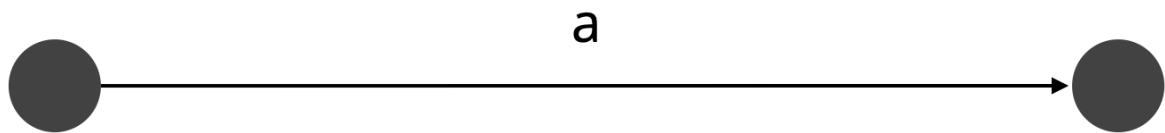


Figure 4. A process P consists of one step a

However, if a process definition consists of *multiple* steps, a process execution does not “inherit” failure atomicity. Failure atomicity does not compose (Figure 5).

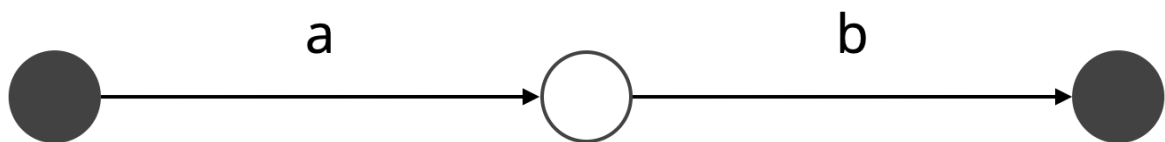


Figure 5. A process P consists of two steps a and b

Now that we have a model for process definitions and process executions, let's discuss how executions recover from failures.

AHA! Moment • Short vs. Long Running Executions

In their paper [Sagas](#), Hector Garcia-Molina and Kenneth Salem introduce the concept of short-running and long-running executions. Molina and Salem define short-running or short-lived as executions that consist of a single step, and long-running or long-lived as executions that consist of multiple steps. Therefore, short-running or long-running is not a statement about physical time but a statement about logical time, where time is delineated by steps. (Figure 6).

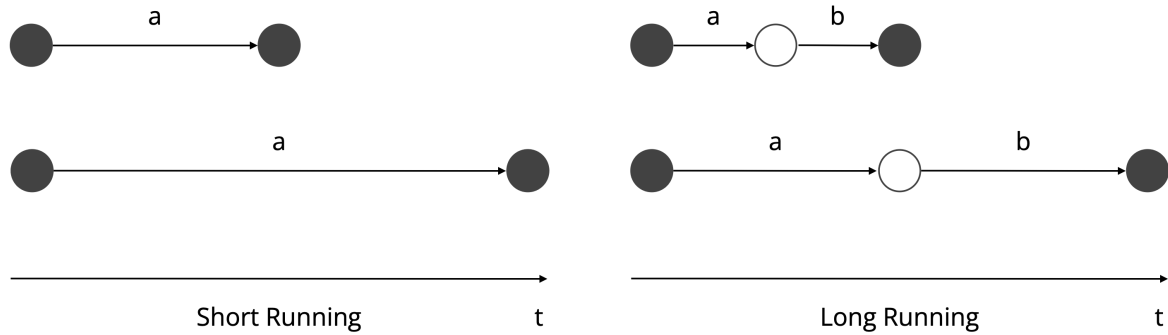


Figure 6. Short Running vs Long Running

Failure-Transparent Recovery

In this chapter, we will only consider forward recovery. For a discussion of backwards recovery and compensation, see Chapter 4.

Recovery of a process is considered failure-transparent if there is a sequence of events, generated by a failure-free execution, that would lead to a sequence of events equivalent to the sequence produced by a failed, recovered, and continued execution.

All discussion of failure-transparent recovery occurs in the context of a specific equivalence function. The equivalence of two sequences of events is determined by an *equivalence function*. This function is specific to the application, and as software engineers, we have the authority to decide when two sequences of events are equivalent.

Let's consider a function that prints the letters `a` to `f`, with each `print` statement being a step.

```
function alphabet() {
  print("a");
```



```
print("b");  
print("c");  
print("d");  
print("e");  
print("f");  
}
```

- The process definition is

$$P = A \cdot B \cdot C \cdot D \cdot E \cdot F \cdot \epsilon$$

- A failure-free execution is

$$t = a \cdot b \cdot c \cdot d \cdot e \cdot f \cdot \checkmark$$

Equivalence & Equivalence Functions

A failure-transparent execution is one that is equivalent to a failure-free execution, as determined by the chosen equivalence function.

Ideally, the equivalence function is the identity function, meaning that the sequence of events produced by a failed, recovered, and continued execution is the same as the sequence of events produced by a failure-free execution.

$$t = a \cdot b \cdot c \cdot \text{💣} \cdot d \cdot e \cdot f \cdot \checkmark$$

However, the identity function amounts to exactly once processing and as we saw in Chapter 4, exactly once processing is not always achievable.

A popular and apt equivalence function is the *valid to duplicate last event* equivalence function:

```
t = a • b • c ✨ c • d • e • f • ✓
```

Another popular and apt equivalence function is the *valid to duplicate last n events* equivalence function:

```
t = a • b • c ✨ b • c • d • e • f • ✓
```

The most extreme variant of *valid to duplicate last n events* equivalence function is the *restart* equivalence function:

```
t = a • b • c ✨ a • b • c • d • e • f • ✓
```

Strategies of Failure-Transparent Recovery

In practice, to ensure that a system behaves correctly in case of event duplication, their actions must be idempotent, see Chapter 4.

When a failure interrupts a process execution, the system must be able to recover and continue the process execution. There are two prevalent patterns for recovering and continuing an execution: restart and resume.

Restart

Process executions recover from failures by *restarting the process execution from the beginning*. Process restart is simple to implement. However, restart has some limitations.

For example, in the initial user signup example, we intend to create the account and collect payment immediately.

```
async function signup(user) {  
  const account = await Account.create(...);  
  const payment = await Payment.create(...);  
}
```

However, if we intend to create the account but delay collecting payment, a simple restart will not be sufficient. If the system restarts the entire signup process due to a failure, the restart will reset the one-week delay for the payment.

```
async function signup(user) {  
  const account = await Account.create(...);  
  setTimeout(() => {  
    const payment = await Payment.create(...);  
  }, OneWeek);  
}
```

Another example where restarting may not be ideal is non-deterministic action. For example, the difference between retrieving the time of day or generating a random number may or may not yield the correct behavior.

Restarting *may* result in incorrect behavior when a process involves non-deterministic actions, such as retrieving the current time or generating a random number, potentially leading to a different outcome.

Resume

Process executions recover from failures *by restarting the process execution from their most recent save point*. By capturing sufficient information between process steps, a process execution effectively resumes after a failure (see Sagas below)



Figure 7. Resume

Implementation of Failure-Transparent Recovery

Implementing failure-transparent recovery can be achieved at two different levels: the application-level and the platform-level. Each approach has its own implications for how processes are defined and executed.

Failure Handling Ladder

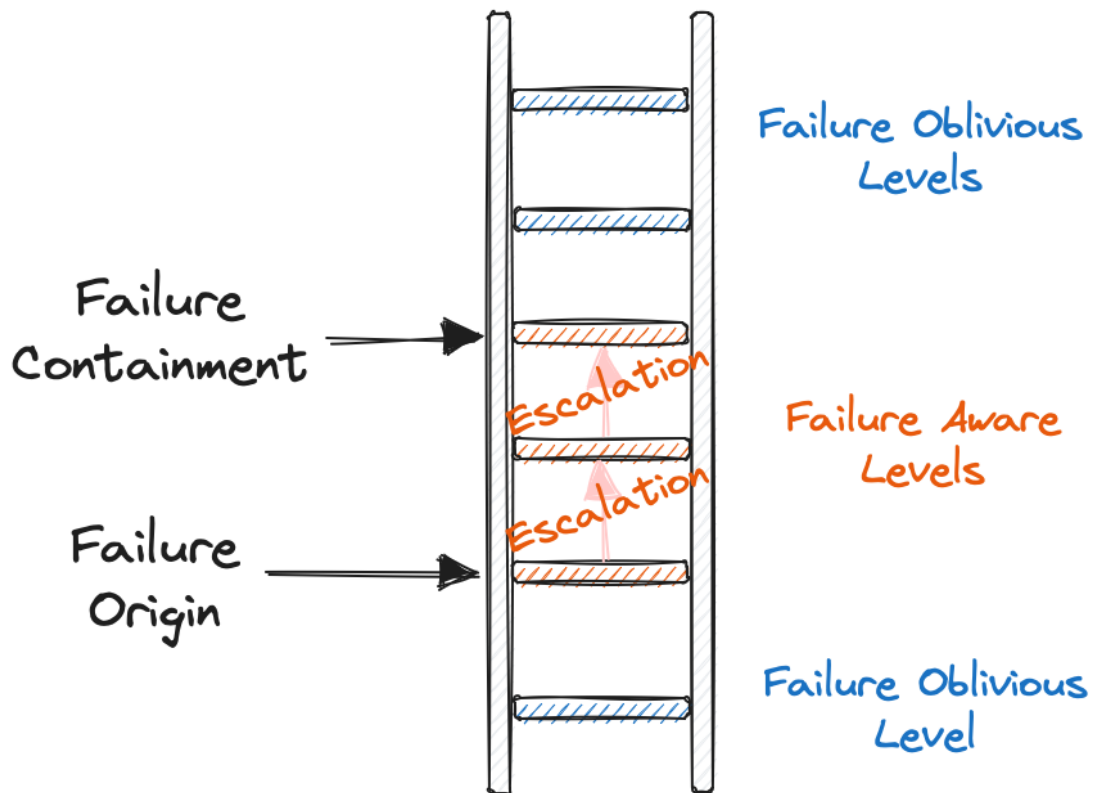


Figure 8. Failure Handling

Application-Level Implementation • Sagas

When failure transparency is implemented at the application-level, the result is *failure-aware* process definitions. In other words, the developer of the process is responsible for failure detection and mitigation and the resulting process definition—the code—accounts for the possibility failure.

Sagas are failure-aware process definitions resulting in failure-transparent process executions

While there is no canonical approach to implementing sagas, state-machine-based implementations are a common practice. In this approach, the state of the saga and the trigger for the next step are persisted after each step, enabling eventual execution.

```
async function signupSaga(state) {
  switch (state.label) {
    // Step 1: Create Account
    case 'createAccount':
      const account = await AccountCreate();
      return { label: 'createPayment', account };

    // Step 2: Create Payment
    case 'createPayment':
      const payment = await PaymentCreate();
      return { label: 'done', payment };

    // Handle unknown states
    default:
      throw new Error('Unknown state');
  }
}
```

In my opinion, this failure-aware implementation, accounting for the possibility of failure, obfuscates the core business logic of the signup process, making it harder to understand and maintain.

In contrast, Durable Executions do not change the nature of your definitions as they do not need to account for failure explicitly.

Platform-Level Implementation • Durable Executions

When failure transparency is implemented at the platform-level, the result is failure-agnostic or failure-oblivious process definitions. In other words, the developer of the process is not responsible for failure detection and mitigation and the resulting process definition, the code, does not account for the possibility failure.

Durable Executions are failure-agnostic process definitions resulting in failure-transparent process executions.

Today, there are two implementation strategies for Durable Executions, log-based implementations and state-based implementations.

Log-based Implementations

In log-based implementations, the system records the output of each step in a durable log to *replay* the process execution. In the event of an execution failure, the runtime restarts the execution but deduplicates previously executed events.

Consider the example of a the signup process:

```
function* signup(user) {  
  
  const account = yield function() {  
    Account.create({...});  
  });  
  
  // 💀 Potential Crash-Stop
```



```
const payment = yield function() {  
  Payment.create({...});  
});  
  
}
```

In this example, the function `signup` is a coroutine function that yields two steps: account creation and payment processing.

```
async function runtime(coroutine, args, durableLog) {  
  
  const execution = coroutine(args);  
  let stepResult = execution.next();  
  let i = 0;  
  
  while (!stepResult.done) {  
  
    let val;  
  
    if (durableLog[i] !== undefined) {  
      stepResult = execution.next(durableLog[i]);  
    } else {  
      durableLog[i] = await stepResult.value();  
      execution.next(durableLog[i]);  
    }  
  
    i++;  
  
  }  
}
```

Here, the `runtime` function takes a coroutine, arguments for the coroutine, and a durable log. The runtime iteratively executes each step of the coroutine, storing the result in the durable log. If

a step has been executed previously (indicated by a non-undefined entry in `durableLog`), the runtime reuses that value.

In effect, in case of recovery, the runtime replays and fast forwards to the failure point.

This implementation is simple and does not require any specific language or runtime support. However, there are notable drawbacks and burdens for software developers when using log-based implementations. Firstly, log-based implementations only work with deterministic executions, meaning that on replay, the execution must follow the exact same path for the execution and the log to match. Secondly, the log grows continuously, and for executions with many steps, the size of the log may become a limiting factor.

State-based implementations

In state-based implementations, the system records the state of the execution, or more accurately, its continuation, after each step. In the event of an execution failure, the runtime restores the continuation and continues execution.

```
async function runtime(coroutine, args, durableState) {
  let execution;

  // If a durable state exists, load and resume the coroutine,
  // otherwise start a new one.
  if (durableState) {
    execution = load(durableState);
  } else {
    execution = coroutine(args);
  }

  // Execute the coroutine steps.
  let stepResult = execution.next();
```

```
while (!stepResult.done) {  
    // Save the current state.  
    durableState = save(execution);  
  
    // Execute the current step and get the result.  
    const value = await stepResult.value();  
  
    // Move to the next step.  
    stepResult = execution.next(value);  
  
}  
}
```

Here, the runtime function takes a coroutine, arguments for the coroutine, and a durable state. The runtime begins execution by either starting a new coroutine or resuming a previously saved one. As each step is executed, the runtime serializes and saves the entire coroutine state to `durableState`.

In effect, in the case of recovery, the runtime doesn't need to replay or fast-forward; it simply reloads the saved state and resumes execution right where it left off before the failure.

This implementation avoids the need for determinism and a growing log. However, this implementation requires language and runtime support in the form of serializable continuations — **a requirements that most runtimes today do not fulfill.**

I expect that with growing popularity of durable executions, over time many mainstream runtimes will offer serializable continuations, either natively, via a compiler extension or via libraries. Early examples include [GrainLang](#) and [UnisonWeb](#).

Conclusion

Failure transparency is a desirable property of any software system: Failure transparency refers to the property of a system that failure free executions are indistinguishable from failed and subsequently recovered executions. The users of a failure transparent system enjoy the illusion that failure does not exist.

Implementing failure transparency can be achieved at two different levels: the application-level and the platform-level. Each approach has its own implications for how processes are defined and executed:

- In case of the application-level, failure-aware process definitions result in failure transparent process executions.
- In case of the platform-level, failure-agnostic process definitions result in failure transparent process executions.

Platform-level failure transparency leads to a delightful developer experience: The developer gets to write code as if failure does not even exist.

Outlook

In the last chapter of this book, we will explore how to think about cloud computing, cloud native computing, serverless, and microservices.

CLOUD & SERVICES



In the landscape of contemporary distributed systems, cloud computing, cloud native computing, serverless computing and microservices have risen to prominence. However, we encounter a paradox of popularity: Despite the ubiquity of the terms, their exact meanings remain somewhat vague and unclear.

Therefore, in the final chapter, I will present my mental models for cloud computing, cloud native computing, serverless computing and microservices.

Objectives:

- Understand Cloud Computing
- Understand Cloud Native Computing
- Understand Serverless Computing
- Understand Services

A Word of Warning

Accurate and concise definitions are essential for effectively conveying complex concepts. However, in software engineering,

there's a frequent issue with definitions that are *neither accurate nor concise*, even for commonly encountered concepts.

With this in mind, the definitions and descriptions in this chapter strive to be accurate and concise while being consistent with current industry understanding. However, it is important to note that they do not necessarily reflect industry standards.

A Brief & Personal History of Cloud Computing

Back in 2008, when I started working at SAP, one of the world's largest software companies, acquiring a single virtual machine was a *manual* and *slow* process. The process involved submitting a request for a virtual machine through a support ticket on the SAP portal. After this, I had to wait for management approval, followed by a waiting period for the provisioning of the machine. Even once the machine was online, it demanded continuous configuration and maintenance until I eventually requested its decommissioning a few months later.

By the time I left SAP a decade later, the landscape had dramatically changed. Acquiring a set of virtual machines had become an automated and nearly instantaneous process. I could quickly create virtual machines on SAP's cloud platform, only constrained by my allocated budget. Machines were no longer permanent fixtures that needed configuration and maintenance. Instead, new machines appeared and disappeared on an as-needed basis.

The transition from static to dynamic resource allocation has redefined our approach to system design: Systems became

elastic. Elasticity refers to the ability of a system to achieve scalability and reliability by acquiring and releasing resources *on demand to meet demand*.

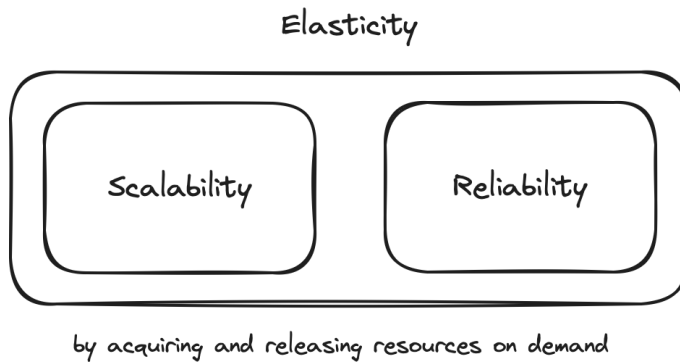


Figure 1. Elasticity in terms of Scalability and Reliability

From Anticipation to Adaptation

Before the advent of the cloud, our systems consisted of a limited number of resources that were long-lived and well-known. These resources had long-lived and well-known properties, including long-lived and well-known identities and network addresses. Software engineers tried to *predict* load and failure requirements and address them *proactively*.

In other words, the topology of our systems was static.

Increasingly, our systems consist of many, short-lived resources with short-lived properties, most prominently short lived network addresses. Software engineers attempt to *observe* load and failure requirements and address them *reactively*, that is, systems are able to self regulate.

In other words, the topology of our systems is dynamic.

This dynamic environment serves as not only the *backdrop* but also the *bedrock* of cloud computing.

What is Cloud Computing?

At its essence, cloud computing separates the world into *resource consumers* and *resource providers*: a resource consumer is able to *acquire* and *release* virtually unlimited resources on demand from a resource provider, the cloud platform.

For example, a cloud platform may offer the resource consumer to acquire compute resources like virtual machines or storage resources like hard disk drives. After a resource is no longer needed, the resource consumer may release the resource.

A cloud platform can be a *public cloud platform*, that is, a resource provider that offers resources outside its own organization, or a *private cloud platform*, that is, a resource provider that offers resources only inside its own organization.

Well-known examples of public cloud platforms include [Amazon Web Services](#), [Google Cloud Platform](#), or [Microsoft Azure](#). Well-known examples of private cloud platforms include company-run [OpenStack](#) instances or [Cloud Foundry](#) instances.

In summary, the hallmark of cloud computing lies in its distinct division between the resource consumer and resource provider, coupled with the capability to acquire and release resources on demand. These fundamental aspects are the core primitives of cloud computing.

In the following sections, we will use these characteristics to establish accurate and concise definitions for cloud native and serverless computing.

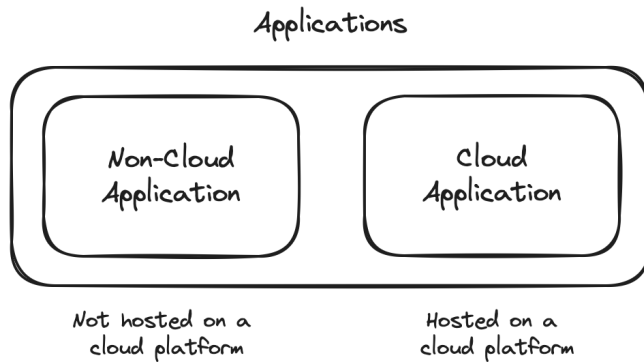


Figure 2. Non-Cloud Application vs Cloud Application

What is Cloud Native Computing?

A *cloud application* is any application hosted on a cloud platform. For example, according to this definition, a vanilla Wordpress installation hosted on a cloud platform is a cloud application. Yet, not every cloud application is a cloud native application.

A *cloud native application* is a cloud application that is scalable and reliable *by construction*, leveraging the cloud's capability of acquiring and releasing resources on demand.

The concept of "by construction" stands in clear contrast to "by requirement." Due to the design and implementation of a cloud native application, scalability and reliability are guaranteed, not merely desired. In other words, these qualities are assured rather than requested.

Cloud platforms provide a range of building blocks to enable scalability and reliability by construction such as supervisors,

scalers, and load balancers. A supervisor monitors a set of resources and acquires a new resource if an existing one is believed to have failed. A scaler monitors the set of resources and acquires or releases resources to match fluctuations in demand. Lastly, a load balancer manages the distribution of requests across available resources.

For example, in Kubernetes, a K8s Deployment can act as the supervisor, a K8s HorizontalPodAutoscaler as the scaler, a K8s Service as the load balancer.

However, merely placing an existing application onto a cloud platform equipped with tools for scalability and reliability doesn't automatically render the application scalable and reliable. As discussed throughout this book in topics like Failure Tolerance, Partitioning, and Redundancy, designing for scalability and reliability requires a holistic approach—it does not only happen on a platform-level but also on an application-level.

For example, according to this definition, the aforementioned vanilla Wordpress instance is not a cloud native application as it lacks load and failure detection and mitigation facilities.

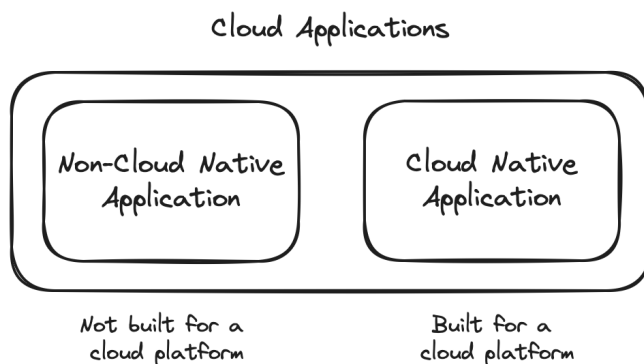


Figure 3. Non-Cloud Native Application vs Cloud Native Application

AHA! Moment • Lift & Shift

"Lift & Shift" describes the process of transferring an existing application, originally not hosted on a cloud platform, to a cloud environment. This migration typically involves little to no modifications to the application itself. However, the benefits of such a move can be limited. To fully capitalize on the capabilities of a cloud platform, especially its elasticity, an application needs to be specifically designed with elasticity in mind.

AHA! Moment • Cloud Native “Technologies”

Common definitions of the term cloud native emphasize that cloud native applications are container packaged, microservice-oriented, and dynamically orchestrated—decorating the result with terms like declarative and immutable.

While cloud native applications often possess these characteristics, I believe that these characteristics alone do not effectively define them. For instance, container packaging is a *technical* choice, while being microservice-oriented is an *architectural* choice. Choosing a different technology or different architectural style does not prevent a cloud application from being scalable and reliable by construction.

What is Serverless Computing?

Like the previous section, we will define serverless computing in terms of cloud computing and the cloud primitives.

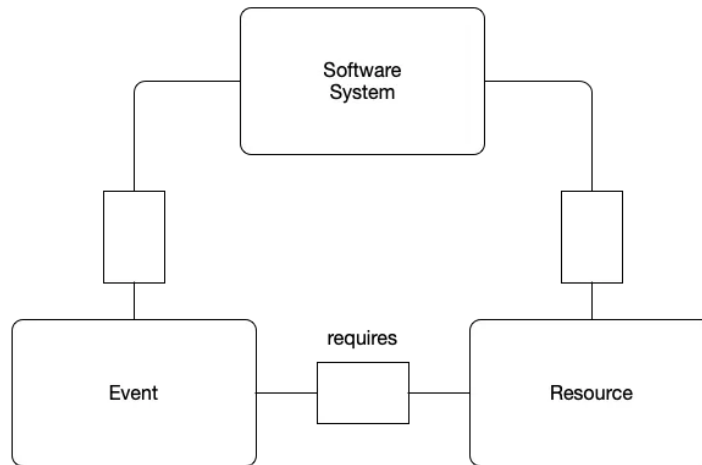


Figure 4. Minimal model of computation to reason about server less computing

We will base this paragraph on a minimal model of computation illustrated in Figure 4. A system is composed of a set of resources, such as computational resources, network resources, or storage resources. Additionally, a system handles a set of application events, here referred to as requests.

There are four relevant actions to consider: A system must be able to acquire and release a set of resources. In addition, a system must be able to receive events and process events.

In all cases, an event cannot be processed if the required resources are not yet acquired.

Traditional

In a traditional computing environment the *operator* of a system must acquire the necessary resources *a priori*, that is, *before* the application event enters the system.



Figure 5. Order of events in traditional computing

Serverless

In a *serverless computing environment* the system may acquire the necessary resources *on demand*, that is, *after* the application event entered the system.



Figure 6. Order of events in server less computing

Consequently, in a serverless environment, the system must be able to determine the necessary set of resources based on the application event itself.

Cold Path vs. Hot Path

Typical implementations of a serverless environment do not release resources after processing an application event immediately. Instead, once acquired, resources are held in anticipation of additional application events for some period of time.

Cold Path refers to the situation where receiving an event and processing an event are interleaved by acquiring resources. Hot Path refers to the situation where receiving an event and processing an event are not interleaved by acquiring resources.

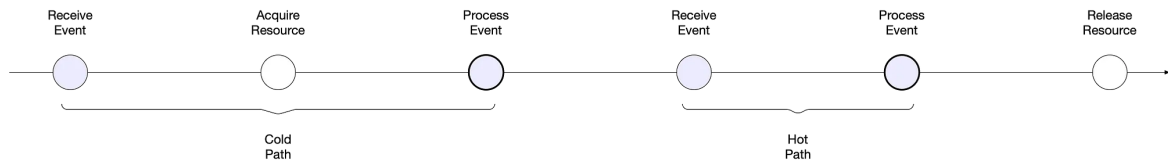


Figure 7. Cold path vs hot path

What is a Service?

In the landscape of cloud based, distributed systems, *microservices* have risen to prominence, eliciting as much enthusiasm as they do debate. However, when we encounter microservices, we encounter a paradox of popularity. Despite the ubiquity of the term microservices, we only have a fuzzy, nebulous mental model of what precisely constitutes a microservices.

Note

Invoking the terms 'microservices' and 'microservices-based architecture' often sparks intense discussions centered on the defining characteristics of *excellent* microservices and architectures. However, my objective is not to critique but to clarify—to discuss what fundamentally constitutes a microservices. Therefore, I will use the less emotionally charged term *service*.

Almost any discussion of a service-based architecture begins with a whiteboard session: rectangles to represent services, lines to represent interactions. A familiar ritual. However, as the discussion progresses, this clarity dissipates. Rectangles get duplicated, other rectangles with labels like load balancer or auto

scalar emerge. New shapes to represent databases, queues, or timer are added.

Stepping back, the whiteboard is no longer a model of simplicity but a complex tapestry that begs the question: where are the service boundaries?

Is each white rectangle truly an autonomous service? When multiple rectangles share the same label, do they collectively represent an autonomous service? Are databases, queues, and timers inside or outside service boundaries? What roles do load balancers and auto scalers play?

Let's demystify: What is a service?

Global View vs. Local View

In the preface of this book we briefly outlined the idea of a global view and a local view. Recall that we are considering the system from the perspective of an all-knowing observer, meaning that we have the ability to observe the state of the entire system, **we have a global view**. A component does not have the same luxury, a component can only observe its own state and its own channel to the network, **giving it a limited, local view**.

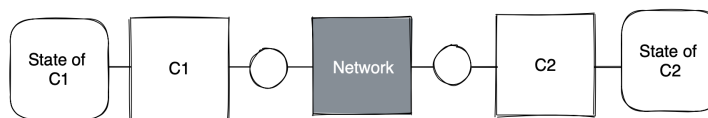


Figure 8. Global point of view, there is C1 and C2

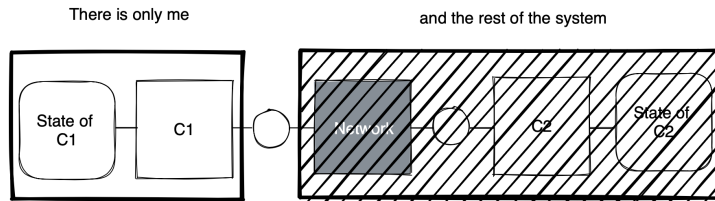


Figure 9. C₁'s point of view, there is only C₁ and the rest of the system

When taking a limited local view, the component C₁ does not actually talk to a service, C₁ talks to the rest of the system. Simply mapping a service to a component or set of components does not make sense from that point of view.

Instead, from the point of view of C₁, a service is a contract, specifying the interaction between C₁ and the rest of the system.

For example, let's consider an `echo` service: When a component sends an HTTP POST request to `http://echo.io` the component will receive a response containing the body of the request.

From our newly found perspective, the component is not addressing another component via `http://echo.io` but is invoking a system-wide contract. The component isn't communicating with a specific component located at `echo.io` but rather requesting a service from the system as a whole.

The service *is the contract* between the component and the rest of the system, the service *is not the set of components implementing the contract*. In other words, we focus on a logical instead of a physical point of view.

Example • Recommendation Service

To develop an intuition of a service, let's examine a *Recommendation Service* like a Movie Recommendation Service.

The contract of the Recommendation Service consist of one operation `Get` . `Get` accepts a single movie `id` and returns a list of movie `ids` .

```
Get: ID -> [ID]
```

Now how could we implement this service? We could implement this service as a Python process connecting to a database to retrieve recommendations specific to an `id` (Figure 10.).

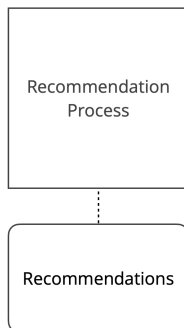


Figure 10. Initial model of the Recommendation Service

While this is functionally correct, this service is neither scalable nor reliable since one process is neither scalable nor reliable. To improve scalability and reliability we employ redundancy, that is, we duplicate the process (Figure 11).

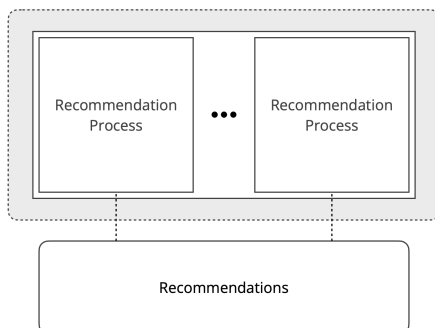


Figure 11. Redundant implementation of the Recommendation Service

However, what can we do if the database is overloaded or experiencing a failure? To address this, we can deploy two different groups of processes: a primary and a backup.

- Primary Group: returns optimal recommendations in case the database is healthy.
- Backup Group: returns static recommendations in case the database is not healthy.

So while the primary group retrieves recommendations from a database that is sometimes unavailable, the secondary group retrieves recommendations e.g. from a local file, that is always available (Figure 12).

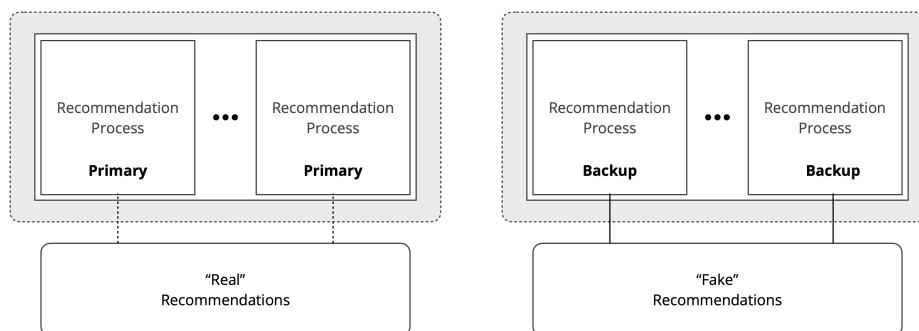


Figure 12. Primary and Backup

In order to distribute requests among processes, we employ a load balancer. However, since we are dealing with two process groups, not all processes are equal. Therefore, we need to employ a smart load balancer. For example, the load balancer may default a request to a process in the primary group, but if that process returns an error, retry the request to a process in the backup group. Additionally, in order to mitigate load and failure both process groups employ autoscaling (Figure 13).

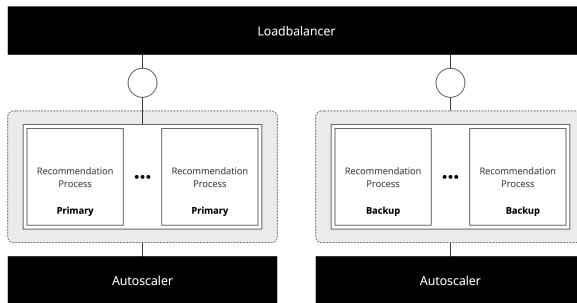


Figure 13. Loadbalancer and Autoscaler

However, there's a challenge: the load balancers and autoscalers themselves may not be immune to load issues or failures, a situation we once again address with redundancy (Figure 14).

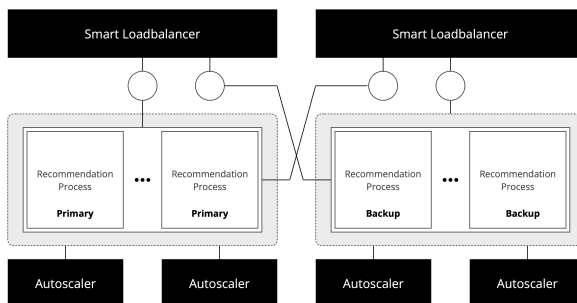


Figure 14. Redundant Loadbalancers and Autoscalers

However, Figure 14. suggests a 1:1 relationship between parts of the system that may not actually exist. For example, some instances of an autoscaler may be responsible for more than one process group, even across service boundaries (Figure 15).

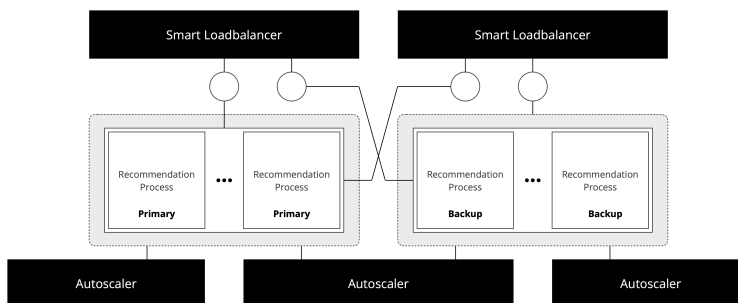


Figure 15. Final model of the Recommendation Service

The Recommendation Service is implemented by a dynamic, ever changing set of components their interactions. Some of these components contribute solely to the implementation of the

Recommendation Service, while others contribute to the implementation of multiple services. Yet, the service is still perfectly embodied by its contract of returning a set of recommendations—the contract as the only constant.

The consumer may ignore all of the complexity and simply request a set of recommendations **from the rest of the system**—the collective entity implementing *all* services available in the system.

Conclusion

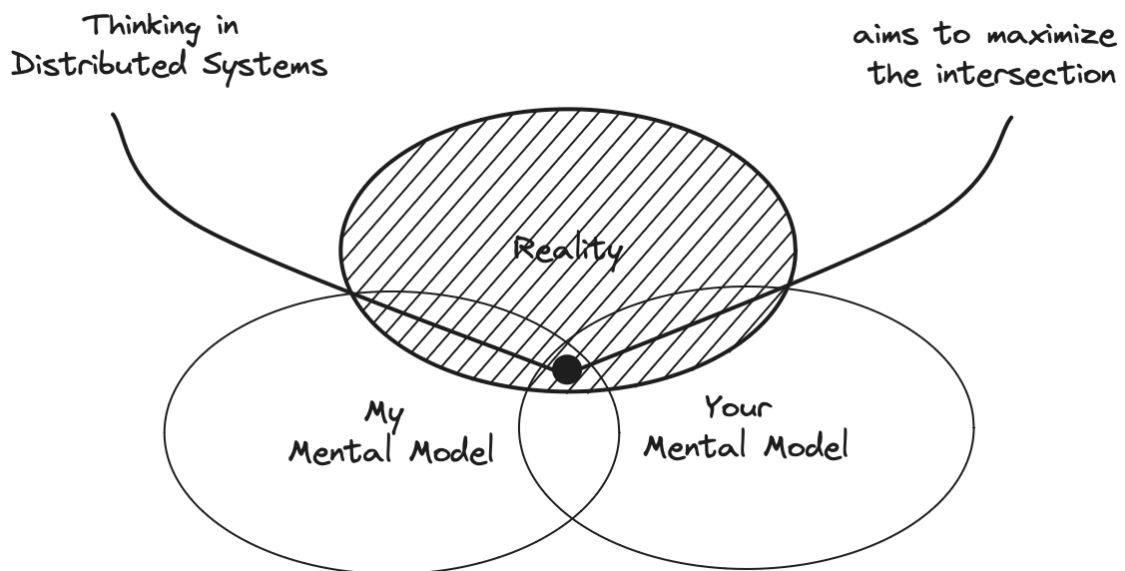


Figure 16. Thinking in Distributed Systems

Imagine a band where each musician interprets the same musical sheet differently. When one musician reads a note as a 'C' and another as an 'F', their combined efforts, meant to produce harmony, creates dissonance.

This analogy mirrors our experience with technology terms like 'cloud', 'cloud native', 'serverless', or 'microservices'. Despite their frequent use, our understanding of these concepts tends to remain shallow and is not aligned with the understanding of our peers. This limited understanding can undermine our effectiveness, both as engineers and communicators.

I never feel more empowered, more confident—neither as an engineer nor as a communicator—than when I feel I fully understand something. Therefore, I encourage you to relentlessly explore the core of any concept you encounter.

Never stop digging

Dominik Tornow